

Ising Graphical Model

Dmitry Kamenetsky

A thesis submitted for the degree of
Doctor of Philosophy at
The Australian National University

May 2010

© Dmitry Kamenetsky

Typeset in Palatino by $\text{\TeX}$ and $\text{\LaTeX}$ 2 ϵ .

Except where otherwise indicated, this thesis is my own original work.

Dmitry Kamenetsky
18 May 2010

Acknowledgements

It is the end of a long and difficult journey and I would like to thank everyone who has made it possible. A big thank you must go to all my supervisors: Nic Schraudolph, Scott Sanner, Tiberio Caetano and Gideon Dror. Nic you have inspired me with your skill and knowledge. You have taught me well and now I feel ready to tackle any research challenge. Scott and Tiberio, thank you for your support and guidance. Gideon thank you for your kindness and many great memories.

I also want to thank all the past and present staff of our Statistical Machine Learning group who have made me appreciate both the power and beauty of Machine Learning: Douglas Aberdeen, Marconi Barbosa, Olivier Buffet, Wray Buntine, Li Cheng, Kishor Gawande, Simon Günter, Markus Hegland, Marcus Hutter, Adam Kowalczyk, Kee Siong Ng, Petteri Nurmi, Brian Parker, Mark Reid, Sylvia Richter, Conrad Sanderson, Alex Smola, Peter Sunehag, S. V. N. Vishwanathan, Christfried Webers and Markus Weimar.

A huge thank you to all my fellow students: Justin, Robby, Julian, Joel, James, Novi, Javen, Le, Choon Hui, Owen, Jin and Xinhua. You have made this journey fun and bearable. You have all been great friends and I hope to stay in touch no matter where life takes us. For those who haven't finished, remember that I feel your pain and that the end is near. Justin you have been a good friend and taught me how to appreciate nice tea. Robby I can't wait to have some fine Belgian beer with you and discuss the aesthetic value of skimpy bikinis. Julian, Le and Xinhua thank you for helping me out on so many occasions, I hope I wasn't too much of a pest. Joel it has been a pleasure working with you and I think we made a great team — one day we will make the best Go program. James, I guess I am now a doubly-finished man, which reminds me that we must finish our theory on the meaning of life. Novi thank you for making me laugh and putting a smile on my face. Javen I still remember our fun roadtrip and dancing lessons, we should do these again someday. Choon Hui don't give up on TopCoder and remember to never underestimate yourself. Owen you must teach me how to make such great photos and I really miss playing poker with you. Jin I am looking forward to going camping and fishing with you; good luck in your new position.

I would like to thank all my friends for distracting me from work and helping me to relax, in particular: Chris, Simon, Desmond, Ayman, Alexey, Linda, Caroline, Lairi, Rowan. Chris thank you for teaching me wonderful things about matrices, helping me with primes and for being a fatherly figure to me. Simon thank you for being such a great friend both at work and outside. I've missed your jokes and our never-ending debates. Desmond thank you for introducing me to skiing, I promise to get better. Ayman you have opened my mind and taught me many things about life, so I say thank you. Alexey I greatly enjoyed everything we did together and I am looking forward to many more good times ahead. Linda thank you for your support and encouragement. Caroline thank you for the fun times and your great help in job hunting. Lairi I enjoyed our tennis games and friendship. Rowan I will greatly miss your hot chocolates, our crazy dancing, our conversations about nothing, watching Tour de France and

infomercials at 2am, and playing board games. You have been the best man in every sense of that word. Stay strong and your freedom will come soon. Thank you also to all my inter-state friends who couldn't be here, but always supported me morally.

I would like to thank all the staff in the Department of Computing Science at the University of Alberta for their warm welcome, despite the cold weather. In particular, I thank Dale Schuurmans and David Silver for organizing my visit.

I thank NICTA and the Australian National University for financing my studies and giving me a nice office. I also thank Michelle Moravec, Deb Pioch, Diane Kossatz and Sue Van Haeften for keeping me on track and helping me with all the complex forms. My thank you goes to our Turkish corner shop for cooking tasty pizzas that helped me to get more work done.

Finally I would like to thank the people who are closest to me — my family. Without your continuous support, encouragement and words of wisdom I wouldn't get very far. My parents, you have always believed in my abilities and have helped me in difficult times. Last, but not least I would like to thank my dear wife Kate. Thank you for your patience and unconditional love. Your delicious dinners have given me the strength to continue and your love has given me complete happiness.

Abstract

The Ising model is an important model in statistical physics, with over 10,000 papers published on the topic. This model assumes binary variables and only local pairwise interactions between neighbouring nodes. Inference for the general Ising model is NP-hard; this includes tasks such as calculating the partition function, finding a lowest-energy (ground) state and computing marginal probabilities.

Past approaches have proceeded by working with classes of tractable Ising models, such as Ising models defined on a planar graph. For such models, the partition function and ground state can be computed exactly in polynomial time by establishing a correspondence with perfect matchings in a related graph.

In this thesis we continue this line of research. In particular we simplify previous inference algorithms for the planar Ising model. The key to our construction is the complementary correspondence between graph cuts of the model graph and perfect matchings of its expanded dual. We show that our exact algorithms are effective and efficient on a number of real-world machine learning problems.

We also investigate heuristic methods for approximating ground states of non-planar Ising models. We show that in this setting our approximative algorithms are superior than current state-of-the-art methods.

Contents

Acknowledgements	v
Abstract	vii
1 Introduction	1
1.1 Thesis Contribution	1
1.2 Thesis Outline	2
2 Background	3
2.1 Mathematical Notation	3
2.2 Graph Notation	3
2.2.1 Connectivity and Biconnectivity	4
2.2.2 Planar Graphs	4
2.3 Dimer Problem	7
2.3.1 Counting Dimer Coverings	8
2.3.2 Pfaffian	9
2.3.3 Constructing the Kasteleyn Matrix	11
2.3.4 Pfaffian Graphs	12
2.3.5 Examples	15
3 Review of the Ising Model	17
3.1 Introduction	17
3.1.1 Brief History	19
3.2 Previous Work on the Partition Function of the Ising Model	20
3.2.1 Ising Problem to Even Subgraphs	20
3.2.2 Even Subgraphs to Perfect Matchings	22
3.2.3 Non-Planar Ising Models	25
3.3 Previous Work on the Ground States of the Ising Model	27
3.3.1 Work by Bieche et al	27
3.3.2 Work by Barahona	29
3.3.3 Work by Thomas and Middleton	30
4 Review of Graphical Models	31
4.1 Introduction	31
4.2 Undirected Graphical Models	32
4.2.1 Conditional Independence Property	32

4.2.2	Factorization Property	33
4.3	Inference Algorithms	34
4.3.1	Variable Elimination	34
4.3.2	Belief Propagation	35
4.3.3	Loopy Belief Propagation	36
4.3.4	Junction Tree Algorithm	37
4.3.5	Graph Cuts	38
4.3.6	Tree Reweighting	39
4.3.7	Work by Globerson and Jaakkola	41
5	Planar Ising Graphical Models: Inference	43
5.1	Introduction	44
5.1.1	Energy Minimization via Cuts	46
5.1.2	Planarity Constraint	47
5.1.3	Connectivity	48
5.1.4	Biconnectivity	48
5.2	Computing Ground States	51
5.2.1	Expanded Dual Graph	52
5.2.2	Complementary Perfect Matchings	53
5.2.3	Computing the Ground State	56
5.3	Computing the Partition Function and Marginal Probabilities	56
5.3.1	Plane Triangulation	57
5.3.2	Odd Edge Orientation	59
5.3.3	Constructing the Kasteleyn Matrix	60
5.3.4	Factoring Kasteleyn Matrices	62
5.4	Conclusion	66
6	Planar Ising Graphical Models: Experiments	67
6.1	Parameter Estimation	67
6.1.1	Maximum Likelihood	67
6.1.2	Maximum Margin	68
6.2	Synthetic Binary Image Denoising	69
6.2.1	Noise Level	71
6.2.2	Parameter Estimation	71
6.2.3	Reconstruction	72
6.3	Boundary Detection	75
6.4	Territory Prediction in Go	77
6.4.1	The Game of Go in Machine Learning	77
6.4.2	Go Positions as Graph Abstraction Hierarchies	80
6.4.3	Conditional Random Field Model	82
6.4.4	Experiments	86
6.4.5	Outlook and Discussion	89

6.5	Conclusion	90
7	Non-Planar Ising Graphical Models	91
7.1	Edmond's Blossom-Shrinking Algorithm	91
7.1.1	Application to Non-Planar Graphs	92
7.2	Heuristics	93
7.2.1	Low-Genus Embedding	94
7.2.2	Minimax Spanning Tree	94
7.2.3	Node Quenching with Implicit Lookahead	95
7.2.4	Tree Quenching	97
7.3	Experiments	99
7.3.1	Experimental Setup	99
7.3.2	Comparison with the Ground State	100
7.3.3	Overall Comparison	104
7.4	Conclusion	108
8	Conclusion	109
8.1	Future Work	110
	Bibliography	111

Introduction

From the beginning of time, man has been trying to understand the world. To help us in this complex task, we have created statistical models that aim to capture all the relevant information of a system. We have models that explain many types of natural phenomena, ranging from gravity and movement of tectonic plates to human behaviour and spread of disease. Models help us to generalise concepts and transfer information between concepts. We can use models to answer many important questions about a system, such as what is its most likely state or what is the probability of a particular event occurring in that system. The procedure used to answer these questions is called *statistical inference* or simply *inference*. There are many types of inference, some are suitable for many types of models and questions, while others are specific to certain models and questions. Machine learning is the study of statistical models and inference methods that are best suitable for these models.

One important model in statistical physics is the Ising model. This model assumes binary variables and only local pairwise interactions between neighbouring variables. Despite its simplicity, the Ising model has been successfully used to describe many physical systems, such as ferromagnetism, fluid flow, neural activity, protein folding and many others. Inference in the general Ising model has been shown to be NP-hard [2, 54], however it becomes tractable when certain conditions are met.

This thesis investigates tractable inference algorithms for the Ising model and discusses their application to machine learning.

1.1 Thesis Contribution

Due to its origins, the Ising model is often described in the language of statistical physics. Here we aim to describe it in the context of machine learning and assume no prior knowledge of statistical physics. The contributions of this thesis are the following:

1. We provide a thorough survey of the Ising model, focussing on the computation of its partition function and ground states.
2. We introduce the Ising graphical model: an undirected graphical model that obeys the constraints of the Ising model, *i.e.*, a binary-labeled energy function that is a sum of edge disagreement costs. We show that the Ising graphical model with an additional bias node is equivalent to any binary-labeled pairwise undirected graphical model.
3. We describe an exact and efficient framework for computing the partition function, worst margin violators, marginal edge probabilities and lowest-energy (ground) states of a planar Ising

graphical model. By doing so, we simplify previous inference algorithms and provide important theoretical connections. In particular, we give the precise complementary connection between graph cuts of the model graph and perfect matchings of its expanded dual. We explain the role of planarity (*resp.* triangulation) in making that relation a surjection (*resp.* bijection).

4. We describe how our algorithms can be used in popular parameter estimation frameworks of maximum-margin and maximum-likelihood. We show that our inference algorithms are efficient and effective on a number of real-world machine learning problems.
5. We demonstrate that our algorithms for computing the ground state of a binary-labeled pairwise Markov Random Field outperform current state-of-the-art algorithms.

1.2 Thesis Outline

It is recommended that this thesis is read in chronological order, *i.e.*, from Chapter 2 to Chapter 8. Chapters 2, 3 and 4 introduce the necessary background material, while Chapters 5, 6 and 7 describe the contribution of this thesis. The following is a brief outline of the thesis:

- Chapter 2 introduces key definitions and notations used throughout the thesis. The chapter also describes the solution to the Dimer problem, which is necessary for understanding Chapters 3, 5 and 6.
- Chapter 3 describes the Ising model explains previous approaches for the computation of the partition function and ground states in this model.
- Chapter 4 presents a review of graphical models and their most popular inference algorithms.
- Chapter 5 introduces the Ising graphical model, which is a binary-labeled undirected graphical model based on the Ising model. The chapter focusses on planar Ising graphical models and describes a framework for exact and efficient inference algorithms in these models. The material presented in this chapter is based on [110, 111]; the contribution of each author is assumed to be equal, unless explicitly stated.¹
- In Chapter 6 we apply the algorithms from Chapter 5 to real-world machine learning problems. We provide an experimental comparison of two popular parameter estimation methods: maximum-margin and maximum-likelihood. The material presented in this chapter is based on [57, 58, 110, 111]; the contribution of each author is assumed to be equal, unless explicitly stated.¹
- Chapter 7 discusses non-planar Ising graphical models and proposes heuristic algorithms for approximating ground states in these models. We compare these heuristic algorithms to the state-of-the-art algorithms described in Chapter 4 on binary-labeled pairwise Markov Random Fields. The work described in this chapter is in preparation for Journal of Machine Learning Research.¹
- Chapter 8 contains a concise summary of our work and ideas for future work.

¹C++ code implementing the algorithms described herein is available from <http://nic.schraudolph.org/isinf/>

Background

This chapter introduces some of the key definitions and notations that are used throughout the thesis. In Section 2.3 we introduce the Dimer problem, which is a key concept in understanding the next chapter on the Ising model.

2.1 Mathematical Notation

We will use bold lowercase letters for vectors and bold uppercase for matrices. Calligraphic letters will refer to a set and an element of that set will be represented by the corresponding lowercase letter. Scalar quantities will be typeset in normal font. For example, $\mathbf{a} := [a_1, a_2, \dots, a_n]$ is a n -dimensional row vector, while a is scalar. $\mathbf{B} := [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m]$ is a $n \times m$ matrix, which can also be written as $\mathbf{B} = [b_{ij}]_{n \times m}$. $\mathcal{E}$ is a set and $e \in \mathcal{E}$ is an element of that set.

The meaning of $|\cdot|$ varies depending on the type of the argument. $|\mathbf{B}| = \det \mathbf{B}$ is the determinant of matrix $\mathbf{B}$, $|\mathbf{b}| = n$ is the length of vector $\mathbf{b}$, while $|b|$ is the absolute value (magnitude) of the scalar b . If $\mathcal{S}$ is a set then $|\mathcal{S}|$ is the number of elements in $\mathcal{S}$, i.e., its cardinality. $\|\mathbf{b}\| := \sqrt{b_1^2 + \dots + b_n^2}$ is the L_2 -norm of $\mathbf{b}$. $\llbracket \cdot \rrbracket$ denotes the indicator function, its value is 1 if its argument is true and 0 otherwise, e.g., $\llbracket a > b \rrbracket$ is 1 if $a > b$ and 0 if $a \leq b$. We will write sets using the curly brackets, i.e., $\{\cdot\}$. We will write permutations using the curved brackets, i.e., $(2, 3, 1, 5, 4)$ is a permutation of the first 5 natural numbers.

$\neg$ denotes Boolean negation, e.g., $\neg 1 = 0$ and $\neg 0 = 1$. e is the Euler's constant and $\ln x$ is the natural logarithm (base e) of x . Sometimes we will write e^x as $\exp(x)$.

$\mathbb{P}(A)$ is the probability of event A , while $\mathbb{P}(A|B)$ is the probability of event A , given that event B occurred.

2.2 Graph Notation

This thesis uses a fair amount of graph theory, some of it standard and some of it not. Here we will describe the representation of graphs, graphical structures and common definitions.

Informally, a graph is an object which can be visualized as a collection of lines. These lines are the edges of the graph, they can be either undirected or directed. A graph is directed if all its edges are directed; undirected if all its edges are undirected. The endpoints of the edges are called the nodes (or

the vertices) of the graph.

Formally, we will write a graph as $G(\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ is the set of *vertices* and $\mathcal{E}$ is the set of *edges* of G . A vertex (or node) i will be represented as $i \in \mathcal{V}$. If G is undirected then an edge between node i and node j will be represented as $(i, j) \in \mathcal{E}$ and $(i, j) = (j, i)$. If G is directed then an edge from node i to node j will be represented as $(i, j) \in \mathcal{E}$. For directed graphs G , we will usually assume that if $(i, j) \in \mathcal{E}$ then $(j, i) \notin \mathcal{E}$. Often we will work with *weighted* graphs, *i.e.*, graphs whose edges are associated with weights. We also need the definition of a *clique*:

Definition 1 A clique is a set of nodes, such that every pair of nodes in that set is connected by an edge. A maximal clique is a clique which is not a proper subset of another clique. A clique with n nodes is often called K_n .

Finally, we will define a *cycle* and a *component*:

Definition 2 A cycle of a graph $G(\mathcal{V}, \mathcal{E})$ is a closed path of its vertices $v_1, v_2, \dots, v_k$, such that $(v_k, v_1) \in \mathcal{E}$ and for every $i < k$: $(v_i, v_{i+1}) \in \mathcal{E}$. The set of edges of the cycle is $\mathcal{O} := \{(v_1, v_2), \dots, (v_i, v_{i+1}), \dots, (v_k, v_1)\}$. A cycle is called simple if it has no repeated edges or vertices.

Definition 3 A component of a graph with respect to a given property is a maximal subgraph that has the property.

2.2.1 Connectivity and Biconnectivity

Most of the algorithms in this thesis assume that the graph is connected:

Definition 4 An undirected graph $G(\mathcal{V}, \mathcal{E})$ is connected if $\mathcal{E}$ contains at least one path between any two nodes of $\mathcal{V}$. If this is not the case then the graph is disconnected.

For example the graph in Figure 2.1(a) is connected, while the graph in Figure 2.1(b) is not. A connected component (Definition 3) is the maximal subgraph in which all nodes are connected. In Figure 2.1(b) there are 2 connected components. Clearly if a graph is connected then it must contain exactly one connected component. A related, but stronger requirement is that of biconnectivity:

Definition 5 A graph is biconnected iff it is connected and does not have any articulation vertices. An articulation vertex or a cut vertex is a vertex whose removal (along with any incident edges) disconnects the graph.

The graph in Figure 2.1(a) is connected, but not biconnected, because the central vertex is an articulation vertex. However, the graph in Figure 2.1(c) is biconnected.

2.2.2 Planar Graphs

Definition 6 A graph is planar if it can be drawn in the plane $\mathbb{R}^2$ (infinite flat surface) without edge intersections. The regions into which such a plane drawing partitions $\mathbb{R}^2$ are the faces of the drawing; the unbounded region is the external face.

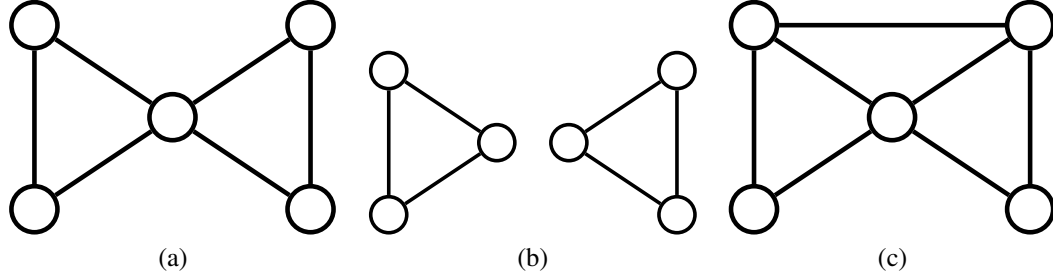


Figure 2.1: (a) connected graph. (b) disconnected graph. (c) biconnected graph.

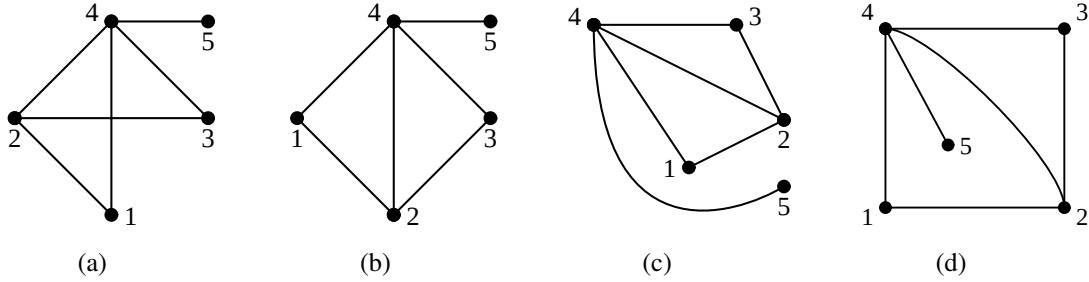


Figure 2.2: (a) a non-plane drawing of a planar graph; (b) a plane drawing of the same graph; (c) a different plane drawing of same graph, with the same planar embedding as (b); (d) a plane drawing of the same graph with a different planar embedding.

This definition might suggest that our algorithms must produce (or have access to) a plane drawing of the graph. In practice, the drawing of the graph contains information that is not useful for us, such as: the precise location of the vertices, and the exact shape of the edges. All we care about is the cyclic (say, clockwise) *ordering* of the edges incident upon each vertex. In topological graph theory, this is formalized in the notion of a *rotation system* [138, p. 21f]:

Definition 7 Let $G(\mathcal{V}, \mathcal{E})$ be an undirected, connected graph. For each vertex $i \in \mathcal{V}$, let $\mathcal{E}_i$ denote the set of edges in $\mathcal{E}$ incident upon i , considered as being oriented away from i , and let π_i be a cyclic ordering of $\mathcal{E}_i$. A rotation system for G is a set $\Pi = \{\pi_i : i \in \mathcal{V}\}$.

For example for vertex 4 in Figures 2.2(b) and 2.2(c), $\mathcal{E}_4 = \{(4, 3), (4, 1), (4, 2), (4, 5)\}$ and $\pi_4 = ((4, 5), (4, 3), (4, 2), (4, 1))$. With some abuse of notation, let $\pi_i(i, j)$ denote the successor edge of (i, j) in the cyclic ordering π_i . For example, $\pi_4(4, 2) = (4, 1)$ and $\pi_4(4, 1) = (4, 5)$. Informally, this means that if we are located at vertex 4 and move clockwise from $(4, 2)$ then we will reach edge $(4, 1)$. To define the sets $\mathcal{E}_i$ of oriented edges more formally, construct the *directed* graph $G(\mathcal{V}, \mathcal{E}')$, where $\mathcal{E}'$ contains a pair of directed edges (known as *edgelets*) for each undirected edge in $\mathcal{E}$, that is, $(i, j) \in \mathcal{E}' \iff [(i, j) \in \mathcal{E} \vee (j, i) \in \mathcal{E}]$. Then $\mathcal{E}_i = \{(j, k) \in \mathcal{E}' : i = j\}$. Rotation systems directly correspond to topological graph embeddings in orientable surfaces:

Theorem 8 (White and Beineke [138]) Each rotation system determines an embedding of G in some orientable surface S such that $\forall i \in \mathcal{V}$, any edge $(i, j) \in \mathcal{E}_i$ is followed by $\pi_i(i, j)$ in (say) clockwise orientation, and such that the faces $\mathcal{F}$ of the embedding, given by the orbits (cycles) of the mapping

$(i, j) \rightarrow \pi_j(j, i)$, are 2-cells (topological disks).

Note that while in graph visualisation “embedding” is often used as a synonym for “drawing”, in modern topological graph theory it stands for “rotation system”. We adopt the latter usage, which views embeddings as equivalence classes of graph drawings characterized by identical cyclic ordering of the edges incident upon each vertex. The graphs in Figures 2.2(b) and 2.2(c) have the same embedding as evident from the equivalence of their π_i . The embedding in Figure 2.2(d) differs, since we have $\pi'_4 = ((4, 5), (4, 1), (4, 3), (4, 2))$ and $\pi'_4(4, 2) \neq \pi_4(4, 2)$. A sample face in Figure 2.2(b) or 2.2(c) is given by the orbit

$$(4, 1) \rightarrow [\pi_1(1, 4) = (1, 2)] \rightarrow [\pi_2(2, 1) = (2, 4)] \rightarrow [\pi_4(4, 2) = (4, 1)].$$

The same starting edge in Figure 2.2(d) gives the orbit

$$(4, 1) \rightarrow (1, 2) \rightarrow (2, 4) \rightarrow (4, 5) \rightarrow (5, 4) \rightarrow (4, 1).$$

Definition 9 The genus g of a surface is the maximum number of non-intersecting closed curves which one can draw on the surface without disconnecting it.

Intuitively, the genus g of the embedding surface S is the number of “holes” in S . So if S is a plane or a sphere then $g = 0$. If S is a torus (donut) then $g = 1$. The genus can be determined from the Euler characteristic

$$|\mathcal{V}| - |\mathcal{E}| + |\mathcal{F}| = 2 - 2g, \quad (2.1)$$

where $|\mathcal{F}|$ is found by counting the number of orbits in the rotation system, as described in Theorem 8. Since planar graphs are exactly those that can be embedded on a surface of genus $g = 0$ (a topological sphere), we arrive at a purely combinatorial definition of planarity:

Definition 10 A graph $G(\mathcal{V}, \mathcal{E})$ is planar iff it has a rotation system Π producing exactly $2 + |\mathcal{E}| - |\mathcal{V}|$ orbits. Such a system is called a planar embedding of G , and $G(\mathcal{V}, \mathcal{E}, \Pi)$ is called a plane graph.

A nice characterization of planar graphs was given by Kuratowski [78]:

Theorem 11 A graph is planar iff it does not contain a subgraph that is homeomorphic to $K_{3,3}$ or K_5 (see Figure 2.3 and Definition 1).

Theorem 11 implies that the graphs in Figure 2.3 are not planar. Indeed it is not possible to draw them on a plane without any crossing edges. The graph in Figure 2.3(c) is homeomorphic to the graph in Figure 2.3(b). A graph G_1 is homeomorphic to a graph G_2 if a sequence of allowed steps transform it into G_2 . The allowed steps are:

1. Edge (i, j) can be subdivided into two edges (i, w) and (w, j) via the addition of vertex w .
2. Two edges (i, w) and (w, j) can be merged into a single edge (i, j) via the removal of vertex w , provided that the degree of w is 2.

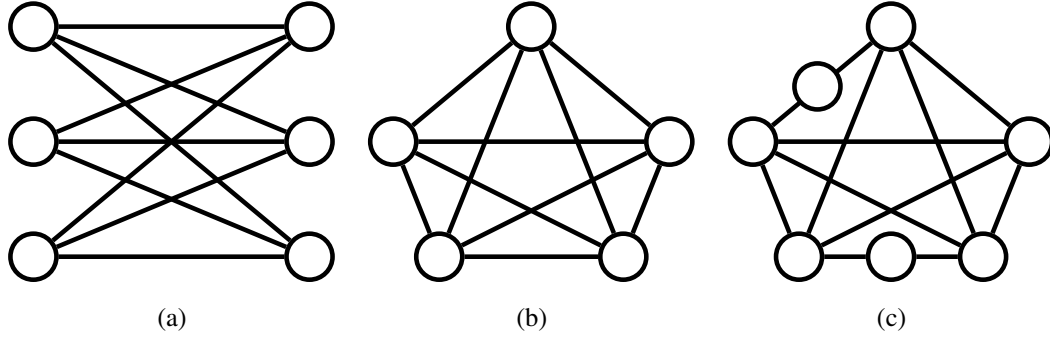


Figure 2.3: Examples of non-planar graphs. (a) Utility graph $K_{3,3}$. (b) Complete graph of 5 nodes or K_5 . (c) A graph that is homeomorphic to K_5 .

All planar graphs have a corresponding *dual* graph:

Definition 12 The dual $G^*(\mathcal{F}, \mathcal{E})$ of an embedded graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ has a vertex for each face of G , with edges connecting vertices corresponding to faces that are adjacent (i.e., share an edge) in G .

G^* is a *pseudograph*, i.e., it may contain loops and multiple edges (see Figure 2.4). It is important to note that G^* has the same set of edges as G : a dual edge crosses its corresponding edge in G . We will use this fact to simplify many algorithms.

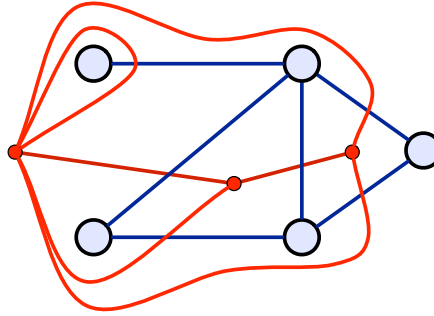


Figure 2.4: The original graph G (blue) with its corresponding dual graph G^* (red). Every edge in G is crossed by an edge in G^* and vice versa.

2.3 Dimer Problem

The dimer problem, also known as the domino tiling problem, is a fundamental problem in graph theory, which underpins many of the definitions and algorithms described in this thesis. We will begin by introducing the problem, followed by a detailed description of its solution.

Informally the dimer problem asks us in how many ways can we cover the edges of a graph with dimers. A *dimer* is a graph that consists of a single undirected edge. For example, there are 3 ways to cover the graph in Figure 2.5(a) with dimers, shown in Figures 2.5(b, c, d).

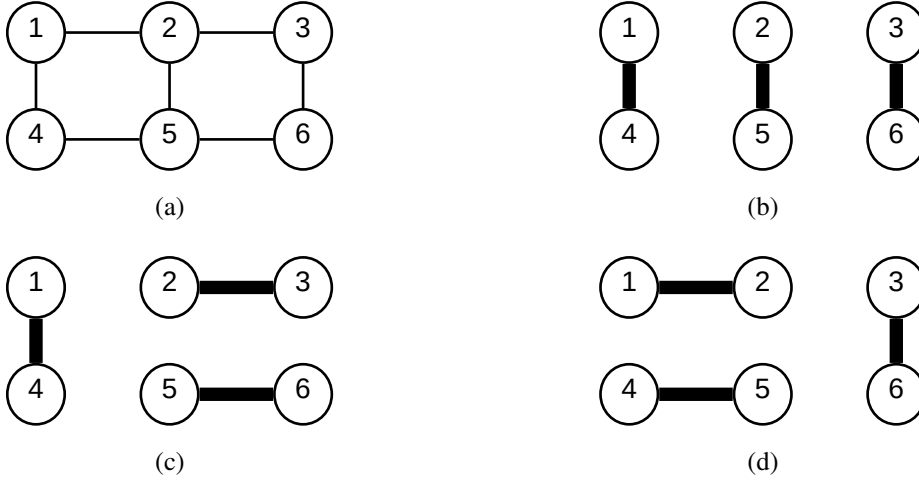


Figure 2.5: A graph (a) and all its 3 corresponding dimer coverings (in bold) shown in (b), (c) and (d).

More formally, the problem is to count the number of *dimer coverings* (Definition 13) or equivalently the number of *perfect matchings* (Definition 14) in a graph:

Definition 13 A dimer configuration of a graph G is a subgraph of G , whose connected components (Definitions 3, 4) are dimers. A dimer covering of G is a dimer configuration which spans all the vertices of G .

Definition 14 A perfect matching of a graph $G(\mathcal{V}, \mathcal{E})$ is a subset $\mathcal{M} \subseteq \mathcal{E}$ of edges wherein exactly one edge is incident upon each vertex: $\forall v \in \mathcal{V}, |v| = 1$ in $G(\mathcal{V}, \mathcal{M})$. Its weight $w(\mathcal{M})$ is the sum of the weights of its edges.

It is important to note that not all graphs have a perfect matching. For example, graphs with an odd number of nodes cannot have a perfect matching. A dimer covering can be represented as a set of edges (dimers) that are included in the covering. For example, a dimer covering of Figure 2.5(b) can be written as $\{(v_4, v_1), (v_3, v_6), (v_2, v_5)\}$. Note that there are many ways to write the same dimer covering, since we can interchange the vertices within each dimer, as well as, the order of the dimers. In the next section, we will present a canonical representation of dimer coverings.

2.3.1 Counting Dimer Coverings

Let $G(\mathcal{V}, \mathcal{E})$ be a graph that admits a perfect matching, where $\mathcal{V} = \{v_1, \dots, v_{2n}\}$ is the set of $2n$ vertices and $\mathcal{E}$ is the set of edges. Remember that a dimer covering must span all the vertices, thus a dimer covering of G will contain exactly n dimers. As discussed in the previous section, the order of vertices and dimers in a dimer covering does not matter. Thus each dimer covering can be represented in $2^n n!$ ways. From these representations we would like to select a single (unique) representation.

We will begin by defining a canonical representation of a dimer covering. Define the k^{th} dimer in the representation to be the edge connecting nodes v_{i_k} and v_{j_k} . For uniqueness we will write this edge as (v_{i_k}, v_{j_k}) where $i_k < j_k$. We can define a total order over dimers based on their first vertices: for

each pair of dimers, let $(v_{i_k}, v_{j_k}) < (v_{i_m}, v_{j_m})$ iff $i_k < i_m$. Now we can simply sort the dimers in a dimer covering and obtain its unique representation:

$$\{(v_{i_1}, v_{j_1}), (v_{i_2}, v_{j_2}), \dots, (v_{i_n}, v_{j_n})\}, \quad (2.2)$$

where $i_k < i_{k+1} \forall k$. For example, using this representation the dimer covering in 2.5(b) becomes $\{(v_1, v_4), (v_2, v_5), (v_3, v_6)\}$. In summary we have partitioned the set of vertex indices $\{1, 2, \dots, 2n\}$ into n pairs: $\{(i_1, j_1), (i_2, j_2), \dots, (i_n, j_n)\}$ such that $i_k < j_k$, $i_k < i_{k+1} \forall k$. Now to count the dimer coverings, all we need is a sum over the set of such partitions. It turns out that the form of this sum looks very similar to the *Pfaffian* of a particular matrix K .

2.3.2 Pfaffian

In the previous section we have hinted that the number of dimer coverings can be computed via a Pfaffian of a particular matrix K . Now we will provide a formal definition of the Pfaffian, which will be followed by the construction of K .

Consider a generic $2n \times 2n$ matrix $A = [a_{ij}]_{2n \times 2n}$. Let α be a partition of the set $\{1, 2, \dots, 2n\}$ into n pairs: $\alpha := \{(i_1, j_1), (i_2, j_2), \dots, (i_n, j_n)\}$ such that $i_k < j_k \forall k$ and $i_1 < i_2 < \dots < i_n$. P is the set of all such partitions α . Let

$$\pi_\alpha := \begin{pmatrix} 1 & 2 & 3 & 4 & \dots & 2k-1 & 2k & \dots & 2n-1 & 2n \\ i_1 & j_1 & i_2 & j_2 & \dots & i_k & j_k & \dots & i_n & j_n \end{pmatrix} \quad (2.3)$$

$$= (i_1, j_1, i_2, j_2, \dots, i_n, j_n) \quad (2.4)$$

be the permutation² corresponding to α , so $\pi_\alpha(1) = i_1$, $\pi_\alpha(2) = j_1$, $\dots$, $\pi_\alpha(2k-1) = i_k$, $\pi_\alpha(2k) = j_k$ and so on. Notice that (2.4) corresponds precisely to the representation of dimer coverings in (2.2). The Pfaffian of A is then given by

$$\begin{aligned} \text{Pf } A &:= \sum_{\alpha \in P} \text{sgn } \pi_\alpha A_{i_1, j_1} A_{i_2, j_2} \dots A_{i_n, j_n} \\ &= \sum_{\alpha \in P} \text{sgn } \pi_\alpha A_{\pi_\alpha(1), \pi_\alpha(2)} A_{\pi_\alpha(3), \pi_\alpha(4)} \dots A_{\pi_\alpha(2n-1), \pi_\alpha(2n)} \\ &= \sum_{\alpha \in P} \text{sgn } \pi_\alpha \prod_{k=1}^n A_{\pi_\alpha(2k-1), \pi_\alpha(2k)}, \end{aligned} \quad (2.5)$$

where $\text{sgn } \pi_\alpha$ is the *signature* of π_α and is defined as +1 if π_α is *even* and -1 if π_α is *odd*. A permutation is even if it can be expressed as a composition of an even number of *transpositions*, and otherwise it is odd. A transposition (i, j) is a function that maps i to j and vice-versa. For example, the permutation that turns the list $\{1, 2, 3, 4, 5\}$ into $\{3, 4, 5, 2, 1\}$ is odd because it can only be written as a composition of an odd number of transpositions: $(1, 3, 5)(2, 4) = (1, 3)(1, 5)(2, 4)$.

²We use two notations for permutations: two-line notation (2.3) and the cycle notation (2.4).

Skew-Symmetry

A *skew-symmetric* matrix is a matrix A such that $A = -A^\top$ (or $A_{ij} = -A_{ji}$ for all i, j). The Pfaffian of a skew-symmetric matrix has some nice properties that we can exploit. Consider a single term of such a Pfaffian computation $t_\pi := \text{sgn } \pi A_{\pi(1), \pi(2)} \dots A_{\pi(2n-1), \pi(2n)}$. The following modifications of $\pi = (i_1, j_1, \dots, i_n, j_n)$ leave t_π unchanged:

1. Interchange of any two partners: i_k and j_k . $\text{sgn } \pi$ changes sign, but then $A_{\pi(2k-1), \pi(2k)}$ becomes $A_{\pi(2k), \pi(2k-1)} = -A_{\pi(2k-1), \pi(2k)}$, and thus t_π remains unchanged.
2. Interchange of any two pairs of partners: (i_k, j_k) and (i_m, j_m) . This does not affect $\text{sgn } \pi$ and only interchanges $A_{\pi(2k-1), \pi(2k)}$ with $A_{\pi(2m-1), \pi(2m)}$, which leaves t_π unchanged.
3. Obviously any combination of 1 and 2.

In 1849 Cayley [15] proved that the Pfaffian of any skew-symmetric matrix A can be computed via its determinant:

$$\text{Pf}^2 A = \det A. \quad (2.6)$$

Note that the Pfaffian of an odd $n \times n$ skew-symmetric matrix is defined to be zero, as the determinant of an odd skew-symmetric matrix is zero. This can be seen as

$$\det A = \det A^\top = \det(-A) = (-1)^n \det A.$$

When n is odd, this implies that $\det A = -\det A$, *i.e.*, $\det A = 0$. (2.6) is a very useful property, because it means that we can compute Pfaffians in polynomial time in the number of elements. From now on when we refer to the Pfaffian of a matrix, we will assume that the matrix is even and skew-symmetric.

Example

We now present a detailed example of a Pfaffian computation. Consider the matrix A defined as

$$A := \begin{pmatrix} 0 & a & b & c \\ -a & 0 & d & e \\ -b & -d & 0 & f \\ -c & -e & -f & 0 \end{pmatrix}.$$

The following table shows all the possible allowed partitions α of the set $\{1, 2, 3, 4\}$. For each α , π_α is the associated permutation, $\text{sgn } \pi_\alpha$ is its signature and the right-most column is the contribution to the Pfaffian sum.

α	π_α	$\text{sgn } \pi_\alpha$	contribution
$\{(1, 2), (3, 4)\}$	I	+1	$+af$
$\{(1, 4), (2, 3)\}$	$(2, 4)(2, 3)$	+1	$+cd$
$\{(1, 3), (2, 4)\}$	$(2, 3)$	-1	$-be$

Summing the contribution column we obtain the final answer: $\text{Pf } A = af - be + dc$. Now consider the determinant of A . Ignoring the terms that are 0, we have:

$$\begin{aligned}\det A &= 2afdc - 2afbe - 2bedc + a^2f^2 + b^2e^2 + d^2c^2 \\ &= (af - be + dc)^2 = \text{Pf}^2 A.\end{aligned}$$

2.3.3 Constructing the Kasteleyn Matrix

We now discuss what K looks like and state precisely its relationship to G . We will refer to K as the *Kasteleyn matrix*, due to its founder Kasteleyn [59, 61].

First of all, we know that the row and column indices of K must correspond to the vertices of G , *i.e.*, K is a $|V| \times |V|$ sized matrix. We also know that the terms in $\text{Pf } K$ that do not correspond to dimer coverings must be 0. To ensure this, we set $K_{ij} := 0$ if there is no edge between v_i and v_j . This must also include $K_{ii} := 0$, since we are not allowed to have any self-loops.

Since we want to evaluate $\text{Pf } K$ via (2.6) we must make K skew-symmetric. This is achieved by letting $K_{ij} := -K_{ji}$, which still leaves us with a lot of freedom.

We now have that all non-zero terms in $\text{Pf } K$ correspond to actual dimer coverings. In order to count the dimer coverings correctly, these terms must contribute $+1$ to the sum in Equation 2.5. Also they can all be -1 , since $\text{Pf}^2 K = (-\text{Pf } K)^2 = \text{Pf}^2 (-K)$ will still produce a valid real-valued determinant. To summarize, we want all non-zero terms in $\text{Pf } K$ to have the same sign. This implies that the product of any pair of non-zero terms, corresponding to two dimer coverings, should be positive:

$$\text{sgn } \pi_\beta \text{sgn } \pi_\gamma \prod_{k=1}^n K_{\pi_\beta(2k-1), \pi_\beta(2k)} K_{\pi_\gamma(2k-1), \pi_\gamma(2k)} > 0, \quad (2.7)$$

where $\beta, \gamma \in P$ are the partitions corresponding to the two dimer coverings. Although β and γ are fixed, we have freedom in selecting π_β and π_γ , provided that $\text{sgn } \pi_\beta$ and $\text{sgn } \pi_\gamma$ remain unchanged.

Let us try to interpret the product in (2.7). To facilitate further discussion it is convenient to define a *superposition* of two dimer coverings. The superposition $S(D_i, D_k)$ of two dimer coverings D_i and D_k is a union of D_i and D_k whose edges alternate between D_i and D_k .

We now describe how a superposition is constructed referring to the example in Figure 2.6. Let D_β (Figure 2.6 bold) and D_δ (Figure 2.6 dashed) be the dimer coverings that are generated with partitions $\beta := \{(1, 4), (2, 3), (5, 6)\}$ and $\gamma := \{(1, 4), (2, 5), (3, 6)\}$ respectively. We begin with an arbitrary starting node, say v_1 . This node (as all other nodes) is adjacent to exactly one edge in D_β . Following this edge we arrive at v_4 . v_4 will be adjacent to a single edge in D_γ . Following that edge, we will either arrive at our initial node (which is the case here) or a new node. Since there is a finite number of nodes, we will always arrive at our starting node, thus completing a cycle. Once we have completed a cycle we select a new arbitrary starting node (that is not already in a cycle) and repeat the procedure. At the end of this procedure, each node will belong to exactly one cycle, called a *superposition cycle*. The superposition cycles visit every node and therefore constitute a cycle covering, in which every cycle has an even length. In Figure 2.6 the superposition cycles of $S(D_\beta, D_\delta)$ are: 1-4-1 and 2-3-6-5-2. The permutation of this superposition can be conveniently written as a product of even cyclic groups corresponding to each superposition cycle, *i.e.*, $(1, 4)(2, 3, 6, 5)$.

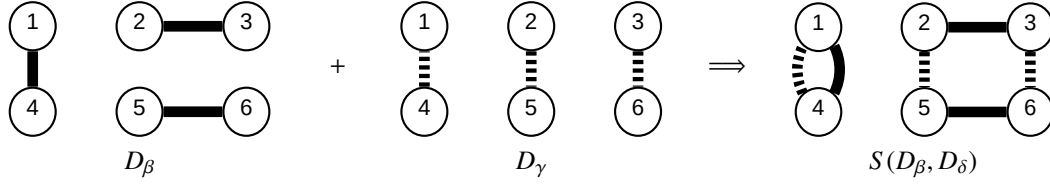


Figure 2.6: Superposition $S(D_\beta, D_\delta)$ of two dimer coverings D_β and D_γ .

It should now become apparent that it is possible to select π_β and π_γ such that the product in (2.7) corresponds to a superposition of D_β and D_δ . In particular, $\text{sgn } \pi_\beta \text{sgn } \pi_\gamma$ can be written as $\text{sgn } \pi$ where π is a product of even cyclic groups. Each such group thus contributes a factor of (-1) to $\text{sgn } \pi$. We now need to select terms in $\mathbf{K}$ to cancel out these minus signs.

The simplest case is when the cyclic group has length 2. In this case the minus sign is automatically compensated, because the factors contributing to (2.7) take the form $\mathbf{K}_{ij}\mathbf{K}_{ji} = -\mathbf{K}_{ij}^2$, which is negative.

For all other cyclic groups we require an odd number of negative terms to cancel out the minus sign of the signature. Let us represent the sign of a matrix element $\mathbf{K}_{ij}$ by an orientation of the edge (v_i, v_j) . In particular if $\mathbf{K}_{ij} = +1$ then the edge is oriented from v_i to v_j , otherwise it is oriented from v_j to v_i . An odd number of negative terms now corresponds to an odd number of edges oriented in the same direction. We can now formalize the requirement for the signs of the matrix elements:

Lemma 15 *Each cycle occurring in a superposition of two dimer coverings must contain an odd number of edges oriented in the same direction.*

Here it is convenient to introduce the *orientation* of the graph:

Definition 16 *An orientation of an undirected graph $G(\mathcal{V}, \mathcal{E})$ is a set $\mathcal{E}'$ of oriented edges with $|\mathcal{E}'| = |\mathcal{E}|$ such that for each $(i, j) \in \mathcal{E}$, $\mathcal{E}'$ contains either (i, j) or (j, i) , but not both.*

An orientation of the edges of G satisfying Lemma 15 is called an *admissible orientation* or a *Pfaffian orientation*. We say that a graph G is *Pfaffian* if there exists a Pfaffian orientation for G . Notice that we do not require that every even cycle of G has an odd orientation parity, but only those cycles that are a result of a superposition of two dimer coverings. This is a real restriction, without which the condition would be difficult to satisfy. In the next section we show which graphs are Pfaffian and how to construct an admissible orientation.

2.3.4 Pfaffian Graphs

The complete characterization of Pfaffian graphs remains an unsolved problem. However, Kasteleyn [60, 61] made considerable progress by showing the following fundamental theorem

Theorem 17 (Kasteleyn Theorem) *Every planar graph has a Pfaffian orientation.*

To prove Theorem 17 we need to show that every planar graph can be oriented such that every superposition cycle has an odd number of edges oriented in the same direction. The direction of an edge in a cycle refers to the direction of that edge in reference to the center of the cycle. For convenience we

chose the clockwise direction as our reference direction. Thus we require that every superposition cycle has an odd number of edges oriented clockwise, *i.e.*, each of its cycles is *clockwise odd*. We call a face *clockwise odd* if the cycle on its perimeter is clockwise odd. We begin by showing that

Lemma 18 *Every plane graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ can be oriented such that every face (except possibly one) is clockwise odd. This orientation is called the clockwise odd orientation.*

Proof (based on [61, p. 93]) We will prove this Lemma constructively. Select an arbitrary face $f_1 \in \mathcal{F}$ and orient all its edges except one arbitrarily. We can always orient the last edge, such that f_1 is clockwise odd. Now select an adjacent face f_2 , one that shares an edge with f_1 . f_2 must have one or more other edges. We orient all these edges except one arbitrarily. Again we can choose the last edge to make f_2 clockwise odd and move on to an adjacent face. We continue this process until all faces have been oriented. This process encounter problems if orienting the last edge of f_i completes the orientation of two faces at once. If this occurs while orienting the very last edge then the Lemma 18 still holds, since we would have already oriented $|\mathcal{F}| - 1$ faces.

However we need to consider carefully if this occurs earlier (Figure 2.7). This will happen only if we select faces in such way that they form a region that is not simply connected, *i.e.*, a region that contains “holes” (such as face 5 in Figure 2.7). We can overcome this problem by working with the dual graph $G^*(\mathcal{V}^*, \mathcal{E}^*)$. Each dual node $i_k \in \mathcal{V}^*$ correspond to a face f_k of G . Starting from an arbitrary node $i_n \in \mathcal{V}^*$, we traverse G^* (in any order) and output the list of visited nodes in postorder $i_1, i_2, \dots, i_n$. We now give a clockwise odd orientation to the faces of G in the order $f_1, f_2, \dots, f_n$. It may or may not be possible to give a clockwise odd orientation to the last face f_n , however Lemma 18 still holds.

The reason this algorithm works is the following. If we are orienting face f_i with $i < n$ then there must exist $k > i$ such that f_k borders f_i and f_k hasn’t been oriented. This means that f_i has at least one unoriented edge — the edge shared by f_i and f_k . We can choose how to orient this edge and thus make f_i clockwise odd. Therefore, the first i such that f_i cannot be given a clockwise odd orientation is $i = n$, in which case $n - 1$ faces have a clockwise odd orientation and Lemma 18 holds. For further explanation and pseudocode see Section 5.3.2. ■

Lemma 19 *In a plane graph $G(\mathcal{V}, \mathcal{E}, \Pi)$, if all faces are clockwise odd then every cycle enclosing an even number of vertices is clockwise odd.*

Proof (based on [87, p. 321]) For every cycle, the number of clockwise edges and the number of vertices that it encloses must have opposite parity. Let C be any cycle in G . Suppose there are f faces inside C and let c_i be the number of clockwise edges on the boundary of face i for $i = 1, \dots, f$. Since all faces are clockwise odd, each c_i is odd and hence

$$f \equiv \sum_{i=1}^f c_i \pmod{2}. \quad (2.8)$$

Let v be the number of vertices enclosed by C , e the number of edges inside C , k the number of edges on C and c the number of those edges that are clockwise. Thus the number of vertices in C is $v + k$, the

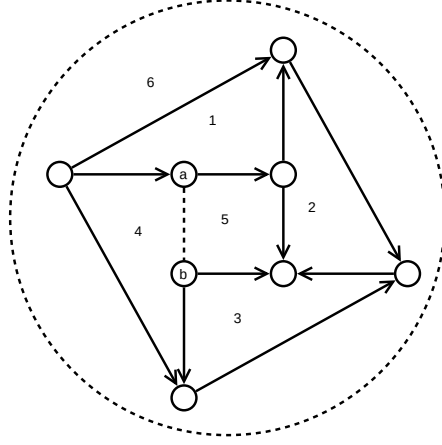


Figure 2.7: This graph is embedded on a sphere and now the “external face” 6 simply becomes another face. If the faces are oriented in the order 1, 2, 3, 4, 5, 6 then there is no way to orient the edge (a, b) such that both faces 4 and 5 are clockwise odd. Since face 6 is also not clockwise odd, there will be two faces that are not clockwise odd, thus violating the requirement of Lemma 18.

number of edges in C is $e + k$ and the number of faces in C is $f + 1$ (we also count the outer face). Thus by Euler’s formula we have

$$|\mathcal{V}| - |\mathcal{E}| + |\mathcal{F}| = v - e + f + 1 = 2,$$

and hence

$$e = v + f - 1. \quad (2.9)$$

Every edge inside C will border exactly two faces. In one of these faces it will be clockwise, while in the other it will be counter-clockwise. Thus counting all clockwise edges for each face is equivalent to counting all edges inside C , so we have $\sum_{i=1}^f c_i = c + e$. Combining this with Equation 2.9 we can now rewrite Equation 2.8 as

$$f \equiv \sum_{i=1}^f c_i = c + e = c + v + f - 1 \pmod{2}, \quad (2.10)$$

and hence $c + v - 1 \equiv 0 \pmod{2}$. This implies that $c + v$ is odd, meaning that c has an opposite parity to v . ■

The combination of Lemmas 18 and 19 tells us that every plane graph can be oriented such that every cycle enclosing an even number of points is clockwise odd. To complete the proof of Theorem 17 we need the following to hold

Lemma 20 *In a plane graph $G(\mathcal{V}, \mathcal{E}, \Pi)$, all superposition cycles enclose an even number of vertices.*

Proof (based on [61, p. 94]) By definition, a plane graph cannot contain any crossing edges. This implies that the superposition cycles cannot cross. Thus, each superposition cycle encloses either zero or

more complete superposition cycles. Since each superposition cycle contains an even number of vertices, we can conclude that each superposition cycle will enclose an even number of vertices (including zero). ■

Since the set of superposition cycles is a subset of all cycles, combining Lemma 20 with Lemmas 18 and 19 proves that every plane graph can be oriented such that every superposition cycle is clockwise odd. In other words we have shown that every planar graph is Pfaffian. Finally it must be noted that the construction used for Lemma 18 is a polynomial-time algorithm for constructing a Pfaffian orientation for plane graphs.

It turns out that the class of Pfaffian graphs is much larger than just those that are planar [127]:

- Every finite graph with no subgraph homeomorphic to $K_{3,3}$ is Pfaffian [85].
- A graph G is Pfaffian if it can be drawn in the plane (possibly with crossings) so that every perfect matching of G intersects itself an even number of times, *i.e.*, has an even *crossing number* [96].

Note that the above only show the existence of a Pfaffian orientation, while constructing the Pfaffian orientation in polynomial-time may still be impossible. For a general graph with genus g , Kasteleyn [61] noted that the number of perfect matchings can be computed by evaluating 4^g determinants. This result was independently proven by Galluccio and Loeb1 [36] and Tesler [125].

2.3.5 Examples

We now present a complete example of counting the number of perfect matchings in a graph. Consider the graph in Figure 2.5(a). We orient the edges of the graph, such that each face (except possibly the outer face) has an odd number of edges oriented clockwise (Figure 2.8).

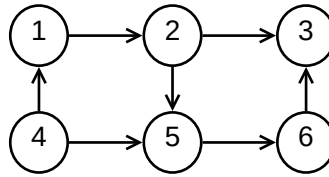


Figure 2.8: Pfaffian or clockwise-odd orientation of a graph. Notice that the outer face $1 - 2 - 3 - 6 - 5 - 4 - 1$ does not need to be clockwise-odd, although it is in this case.

Based on this oriented graph $G(\mathcal{V}, \mathcal{E}')$, we construct the corresponding skew-symmetric matrix $\mathbf{K}$ using the following rules:

$$\mathbf{K}_{ij} = \begin{cases} +1 & \text{if } (v_i, v_j) \in \mathcal{E}' \\ -1 & \text{if } (v_j, v_i) \in \mathcal{E}' \\ 0 & \text{if } i=j \\ 0 & \text{otherwise} \end{cases} \quad (2.11)$$

Using the above, we obtain the matrix

$$\mathbf{K} = \begin{pmatrix} 0 & +1 & 0 & -1 & 0 & 0 \\ -1 & 0 & +1 & 0 & +1 & 0 \\ 0 & -1 & 0 & 0 & 0 & -1 \\ +1 & 0 & 0 & 0 & +1 & 0 \\ 0 & -1 & 0 & -1 & 0 & +1 \\ 0 & 0 & +1 & 0 & -1 & 0 \end{pmatrix}.$$

Now, the number of perfect matchings can be computed as the Pfaffian of $\mathbf{K}$ using (2.6): $\text{Pf } \mathbf{K} = \sqrt{\det \mathbf{K}} = \sqrt{9} = 3$. This agrees with the total number of perfect matchings shown in Figures 2.5(b-d).

Kasteleyn [59], Temperley and Fisher [124] independently showed that the dimer problem on the $2n \times 2m$ lattice has a closed-form solution. A Pfaffian orientation of a lattice can be achieved by orienting all horizontal edges in the same direction, while orienting vertical edges in alternating directions as shown in Figure 2.9. Computing the Pfaffian of the corresponding matrix we find that the number of dimer coverings is

$$\prod_{i=1}^n \prod_{k=1}^m \left(4 \cos^2 \frac{\pi i}{2n+1} + 4 \cos^2 \frac{\pi k}{2m+1} \right). \quad (2.12)$$

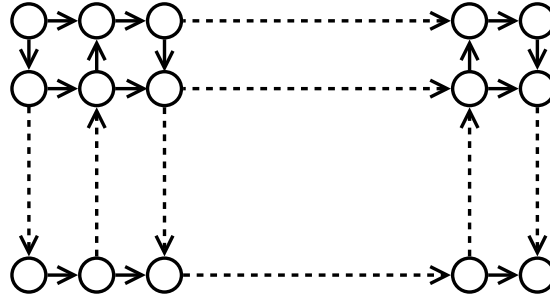


Figure 2.9: An example of a Pfaffian orientation of a lattice.

Review of the Ising Model

This chapter introduces the Ising model — a classical model in statistical physics. We give a description of the model and provide some motivation for its use. This is followed by a brief history of the main developments associated with the model. We then focus on two key problems in the Ising model: computation of the partition function (Section 3.2) and the ground state (Section 3.3).

3.1 Introduction

Consider a large lake. As winter approaches the air becomes cooler and so does the temperature of the water. Then at around 0° degrees the surface of the lake freezes. This occurs because the water molecules change their state from liquid to solid, known as a *phase transition*. Amazingly the lake freezes across the whole surface at roughly the same time. How do the molecules know at which temperature to freeze and how do they “communicate” with each other to achieve that feat? The same phenomenon is observed in other systems, such as when a cooling lump of iron suddenly becomes magnetic at a certain temperature.

In 1925, the German physicist Ernst Ising introduced a simple graphical model, called the *Ising model* [53], that can model such phenomena. The Ising model can be formulated on any graph as follows: consider an undirected graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, \dots, N\}$ is a set of N sites, and $\mathcal{E}$ is a set of edges representing the interactions between these sites. Every site i has a corresponding *spin* variable s_i . These spins are binary-valued³, taking values $+1$ for “up” or -1 for “down”. Two spins s_i and s_j may interact with each other. The energy of such an interaction depends on whether the values of the participating spins are the same or different: it is given by $-J_{ij}s_i s_j$, where J_{ij} is the strength of the interaction. The J_{ij} are independent identically distributed random variables. If $J_{ij} > 0$ the interaction is called *ferromagnetic*, if $J_{ij} < 0$ the interaction is called *antiferromagnetic*, and otherwise $J_{ij} = 0$ and the two spins do not interact. For each pair of *interacting* spins s_i and s_j (i.e., $J_{ij} \neq 0$) there exists a corresponding edge $(i, j) \in \mathcal{E}$. The *state* of the model s is an assignment of all N variables s_i , $1 \leq i \leq N$. The set of all possible configurations is $\Omega = \{-1, +1\}^N$. As well as pair-wise interactions there can also be an external field that affects each site i with energy $-h_i s_i$. Thus in the general case the energy of a

³The Potts model [101] is a generalization of the Ising model which allows the spins to have more than 2 states.

configuration $s \in \Omega$ is given by the so-called Edwards-Anderson Hamiltonian [25]:

$$\mathcal{H}(s) := - \sum_{(i,j) \in \mathcal{E}} J_{ij} s_i s_j - \sum_{i \in \mathcal{V}} h_i s_i. \quad (3.1)$$

When $h_i = 0 \forall i \in \mathcal{V}$ the system is said to have *no external field* (also called *zero magnetic field condition*), in which case the energy of the configuration s becomes

$$\mathcal{H}(s) = - \sum_{(i,j) \in \mathcal{E}} J_{ij} s_i s_j. \quad (3.2)$$

Note that negating all the spins s_i gives a different configuration s' whose energy is the same: $\mathcal{H}(s) = \mathcal{H}(s')$. From now on, we will regard s and its negation s' as a single state. The system prefers lower energy states, *i.e.*, those s that minimize $\mathcal{H}(s)$. Hence such states are more probable; the degree of preference depends on the temperature T of the system. Formally, the probability that the system with the energy (3.2) is in configuration s is given by

$$\mathbb{P}(s) = \frac{1}{Z} \exp\left(-\frac{\mathcal{H}(s)}{\kappa T}\right), \quad (3.3)$$

where κ is the Boltzmann constant. For the sake of simplicity we will define $\beta := 1/(\kappa T)$. Z is known as the *partition function* and is defined as

$$Z(\beta) := \sum_{s \in \Omega} \exp(-\beta \mathcal{H}(s)). \quad (3.4)$$

The partition function plays the role of a normalizing constant, ensuring that the probabilities add up to one. This can be easily seen if we compute the total probability:

$$\sum_{s \in \Omega} \mathbb{P}(s) = \frac{1}{Z} \sum_{s \in \Omega} \exp(-\beta \mathcal{H}(s)) = \frac{1}{Z} Z = 1. \quad (3.5)$$

In Equation 3.3 as the temperature T decreases the configurations with lower energy become more likely. In fact in the limit $T \rightarrow 0$ only the configurations of lowest energy have a non-zero probability of occurring. Hence an important task is to find such configurations s^* that minimize the energy of the system. Such a configuration is known as a *ground state* and is defined as

$$s^* := \operatorname{argmin}_{s \in \Omega} \mathcal{H}(s). \quad (3.6)$$

In Equation 3.2, if $J_{ij} > 0$ then having both s_i and s_j in the same state (*i.e.*, $s_i = s_j$) can only decrease the energy. Therefore if $J_{ij} > 0 \forall (i, j) \in \mathcal{E}$ then it is clear that s^* is the configuration where all spins are in the same state. However the problem becomes far from trivial when J_{ij} can be negative. The Ising model can be used to answer many other questions such as:

- What is the most likely spin at a given site?
- At what temperature does a phase transition occur?

- What is the correlation between spins at two given sites?

For some Ising models the above questions can be answered exactly, *e.g.*, when the underlying graph is planar (see Sections 3.2 and 3.3). In the general case, however, exact methods are too slow and one must resort to approximations. A plethora of heuristic methods have been used to study the Ising model, for example: Monte Carlo simulation [55, 80, 120], Simulated Annealing [63], Parallel Tempering [49, 88], Extremal Optimisation [10] and genetic algorithms [48].

Although the Ising model can be formulated on any graph, often a 2-dimensional rectangular lattice is used. When the size of the model lattice is too small, the behavior of the spins at the boundary of the lattice may produce erratic results. For this reason, the model is often given *periodic boundary conditions*, *i.e.*, nodes on one side of the lattice are connected to nodes on its opposite side. The resulting graph is a toroidal lattice.

3.1.1 Brief History

The original motivation for the model was the phenomenon of ferromagnetism. Iron is magnetic, so once it is magnetized it stays magnetized for a long time compared to any atomic time scale. Once the electron's spin was discovered, it became clear that magnetism is due to a large number of electrons spinning in the same direction. It was then natural to ask how the electrons all know which direction to spin, if they can only influence their neighbours. The Ising model was designed to investigate whether a large fraction of the electrons could be made to spin in the same direction using only local forces.

Interestingly, the Ising model was first proposed by Ernst Ising's research director Lenz [82] in 1920, but the model is more commonly attributed to Ising. In fact even Lenz himself has never made any attempt to claim credit for suggesting the model, and even his colleagues at Hamburg University were not aware of his contribution [13]. In his PhD thesis, Ising [52] solved the one-dimensional Ising model and showed that it does not exhibit a phase transition. On the basis of this result, he concluded that the Ising model does not have a phase transition in any dimension.

In 1936 Peierls proved Ising wrong, using a probabilistic argument to show that a phase transition can occur in the two-dimensional Ising model [100]. In 1941 Kramers and Wannier [74] showed that a matrix formulation of the model allows the partition function to be related to the largest eigenvalue of the matrix, and established the exact location of the phase transition—the *Curie point*. A breakthrough came in 1944 when Lars Onsager computed the partition function of the Ising model on a two-dimensional square lattice with no external field [97]. In 1952 Kac and Ward [56] attempted to provide a simpler alternative to Onsager's formula based on a combinatorial interpretation. Kac and Ward [56] viewed the partition function as a generating function for the problem of counting even subgraphs in an $n \times m$ lattice, which can be computed by finding the determinant of an appropriately constructed $4mn \times 4mn$ matrix. Their argument, however, was found to be incomplete, since some of the closed polygons were not being counted correctly. The clue to the correct approach was found by Feynman who conjectured a relation between functions of graphs and closed paths on lattices [45]. In 1960 Sherman [117] proved Feynman's conjecture, thus making the Kac-Ward method completely rigorous.

Also in 1960, Hurst and Green [50] used field theory techniques to arrive at a new method that combines the algebraic solution of Onsager [97] with the combinatorial solution of Kac and Ward [56], but is simpler than either. This was the first method that used the *Pfaffian* of a skew-symmetric matrix

to solve the Ising model. The Pfaffian method was developed further in the fundamental papers of Kasteleyn [59, 60] and Fisher [30, 31].

In 1982 Barahona [2] showed that finding the ground state of an Ising model defined on a square lattice with an external field is NP-hard. He also showed that finding the ground state of an Ising model defined on a two-level square lattice without an external field is NP-hard. Based on Barahona's work, in 2000 Istrail [54] proved that computing the partition function of a three-dimensional Ising model to be NP-complete. He explained that NP-completeness arises due to the non-planarity of the model involved, rather than its number of dimensions. Finally, Bertoni et al. [6] showed that the ground state of a three-dimensional Ising model is not approximable, meaning that there is no polynomial-time algorithm that approximates the energy of the ground state within a certain ratio.

3.2 Previous Work on the Partition Function of the Ising Model

The partition function of an Ising model can be viewed as a sum over its spin configurations. It is well known that the partition function of a planar Ising model with no external field (3.4) can be found exactly by computing the Pfaffian of a particular matrix derived from the model graph G [56]. The mapping to the Pfaffian is constructed in two steps: first the Ising partition function on G is mapped to a weighted sum over *even subgraphs* in some relevant graph G_0 , then the weighted sum over even subgraphs in G_0 is mapped to the dimer problem (Section 2.3), *i.e.*, a perfect matching on a *decorated* graph of G_0 , which we call G_d . In the following sections we describe these mapping steps.

3.2.1 Ising Problem to Even Subgraphs

As mentioned earlier, the Ising partition function on G can be mapped to a weighted sum over *even subgraphs* (Definition 22) in some relevant graph G_0 . There are two main methods for obtaining the required mapping: in the first, G_0 is the primal model graph G with transformed edge weights; in the second, G_0 is the dual graph G^* with unmodified edge weights.

Primal Graph Construction

For the sake of simplicity assume that the model graph G is a $L \times L$ square lattice with $N := L^2$ spins. We begin with an important Lemma:

Lemma 21 $e^{\pm x} = \cosh(x)(1 \pm \tanh(x))$, $\forall x \in \mathbb{R}$.

Proof

$$\begin{aligned} \text{RHS} &= \cosh(x)(1 \pm \tanh(x)) = \cosh(x) \pm \sinh(x) \\ &= \frac{e^x + e^{-x}}{2} \pm \frac{e^x - e^{-x}}{2} = e^{\pm x} = \text{LHS} \end{aligned}$$

■

Since $s_i s_j = \pm 1 \forall (i, j) \in \mathcal{E}$, we can apply Lemma 21 to the expression of the partition function (3.4):

$$\begin{aligned}
 Z(\beta) &= \sum_{s \in \Omega} \prod_{(i,j) \in \mathcal{E}} e^{\beta J_{ij} s_i s_j} \\
 &= \sum_{s \in \Omega} \prod_{(i,j) \in \mathcal{E}} \cosh(\beta J_{ij}) (1 + s_i s_j \tanh(\beta J_{ij})) \\
 &= \sum_{s \in \Omega} \left[\prod_{(i,j) \in \mathcal{E}} \cosh(\beta J_{ij}) \prod_{(i,j) \in \mathcal{E}} (1 + s_i s_j \tanh(\beta J_{ij})) \right] \\
 &= \prod_{(i,j) \in \mathcal{E}} \cosh(\beta J_{ij}) \left[\sum_{s \in \Omega} \prod_{(i,j) \in \mathcal{E}} (1 + s_i s_j \tanh(\beta J_{ij})) \right] \\
 &\propto \sum_{s \in \Omega} \prod_{(i,j) \in \mathcal{E}} (1 + s_i s_j \tanh(\beta J_{ij})) \tag{3.7}
 \end{aligned}$$

Thus we have transformed a sum over $e^{-\beta \mathcal{H}(s)}$ to a sum over polynomials in $\tanh(\beta \mathbf{J})$ [29, 106]. Expanding the product in (3.7) gives rise to 2^{2N} terms of varying order in $\tanh(\beta \mathbf{J})$. A k^{th} -order term in $\tanh(\beta \mathbf{J})$ has a coefficient of the form $(s_{i_1} s_{j_1}) \dots (s_{i_k} s_{j_k})$, where each pair appears only once. There is a 1:1 mapping by which each k^{th} -order term in $\tanh(\beta \mathbf{J})$ is associated with the set of k edges $\{(i_1, j_1), \dots, (i_k, j_k)\}$. We know that

$$\sum_{s=\pm 1} s^n = \begin{cases} 2 & \text{if } n \text{ is even} \\ 0 & \text{if } n \text{ is odd} \end{cases} \tag{3.8}$$

Based on (3.8) we can see that only those terms where each s_i appears an even number of times will contribute to $Z(\beta)$ in Equation 3.7, while all other terms will vanish. Diagrammatically this corresponds to a set of edges $\{(i_1, j_1), \dots, (i_k, j_k)\}$ in which every node i has an even number (0, 2 or 4 in case of a rectangular grid) of incident edges. We call such a set of edges (and nodes) an *even subgraph*:

Definition 22 An even subgraph is a subgraph in which every node has an even degree.

Examples of even subgraphs are shown in Figures 3.1 and 3.2(a). While “even subgraph” is a term that is generally accepted in modern graph theory, many other terms have been used to describe the same concept: *closed polygons* [31, 56, 59, 60], *closed subgraphs* [61], *quasi-cycles* [3], *Eulerian subgraphs* [37] and *loops* [126].

To summarize, we can map the Ising partition function on G to a weighted sum over even subgraphs in G_0 . In this case G_0 is the original graph G whose edge weights are $w_0(i, j) := \tanh(\beta J_{ij})$ for edge $(i, j) \in \mathcal{E}_0$.

Dual Graph Construction

In the primal graph construction there is no direct correspondence between individual even subgraphs and spin configurations. An alternative construction is to define G_0 to be the dual graph G^* and use unmodified weights, *i.e.*, $w_0(i, j) := -\beta J_{ij}$ for edge $(i, j) \in \mathcal{E}_0$. Every state s of the Ising model induces a *cut* in G ; this cut is an even subgraph of G^* :

Definition 23 The cut C of an Ising model $G(\mathcal{V}, \mathcal{E})$ induced by state $s \in \{-1, 1\}^{|\mathcal{V}|}$ is the set $C(s) = \{(i, j) \in \mathcal{E} : s_i \neq s_j\}$.

Figure 3.1 shows a section of a rectangular lattice G (white circles) with its corresponding dual graph G^* (red disks). Edges with opposing spins (*disagreement edges*) correspond to edges of the cut $C(s)$, which are the edges of the even subgraph in G^* (bold red).

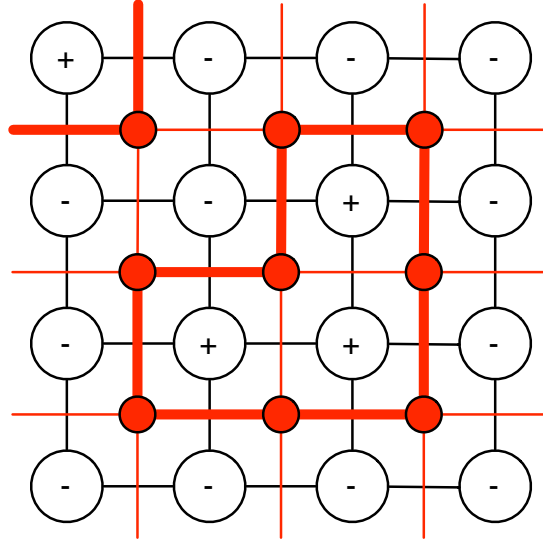


Figure 3.1: A section of a rectangular lattice G (white circles) with its corresponding dual graph G^* (red disks). The spin configuration at each site induces a cut in G , which is an even subgraph in G^* (bold edges).

Most work on the Ising partition function before the 1980s was primarily concerned with regular lattices, and it so happens that the dual graph of such lattices is the original lattice. Therefore, it is not clear whether prior work, such as [61], used the dual construction. The first time the dual graph is mentioned in relation to the Ising partition function is in Barahona [2]. However, in that work the correspondence is between sets of unsatisfied edges in G and paths in G^* joining pairs of frustrated faces of G (Section 3.3.2). It seems that Globerson and Jaakkola [38] were the first to explicitly use this dual construction for the Ising partition function (Section 4.3.7). We also use this construction in our work (see Chapter 5).

3.2.2 Even Subgraphs to Perfect Matchings

The computation on even subgraphs in G_0 can be mapped to the dimer problem on a *decorated* graph G_d . This mapping can be either direct or complementary, depending on whether edges of an even subgraph correspond to matched or unmatched edges of the perfect matching of G_d , respectively. Past approaches also differ in the construction of the decorated graph G_d . The decorated graph G_d is obtained by replacing every node of G_0 with a certain subgraph, called a *city*.

Direct Mapping

In the direct mapping, edges of an even subgraph in $G_0(\mathcal{V}_0, \mathcal{E}_0)$ correspond to matched edges of the perfect matching of $G_d(\mathcal{V}_d, \mathcal{E}_d)$. Recall that an even subgraph has an even number of edges incident on every node of G_0 . This implies that each city in G_d must have an even number of nodes. With this in mind, Kasteleyn [60, 61] replaces every node $i \in \mathcal{V}_0$ with a *Kasteleyn city*:

Definition 24 A Kasteleyn city of a node $i \in \mathcal{V}_0$ with degree d is a d -clique if d is even and a $(d+1)$ -clique if d is odd. This clique contains all original edges of G_0 , as well as the added edges that are internal to the clique.

To simplify the description of the mapping, we assume that G_0 is a 4-connected rectangular lattice. In this case, it is sufficient to replace every node $i \in \mathcal{V}_0$ with a 4-clique. The construction of G_d can be seen in Figure 3.2(b). We must add extra “dummy” nodes (black disks) to ensure that cities have an even number of nodes; these nodes do not have an incident original edge.

The direct mapping from even subgraphs in G_0 to dimer coverings in G_d is done as follows. With each edge of an even subgraph in G_0 (Figure 3.2(a)) associate a dimer on the original edge in G_d (bold dimers in Figure 3.2(b)). The remaining unmatched nodes in G_d come in pairs, as there are 4 nodes per clique with 2 nodes already matched earlier. These pairs of nodes can be matched via dimers on the internal edges of G_d . If 4 edges of an even subgraph meet (e.g., node 6 in Figure 3.2(a)), then there is only one way of mapping it to a dimer configuration in G_d (Figure 3.3(a)). This 1:1 correspondence also holds where 2 edges of an even subgraph meet (Figure 3.3(b)). However, if a node in G_0 has no incident edges of an even subgraph, such as the *isolated* nodes 1, 11 and 12 in Figure 3.2(a), then there are 3 ways to map it to a dimer configuration (Figure 3.3(c)).

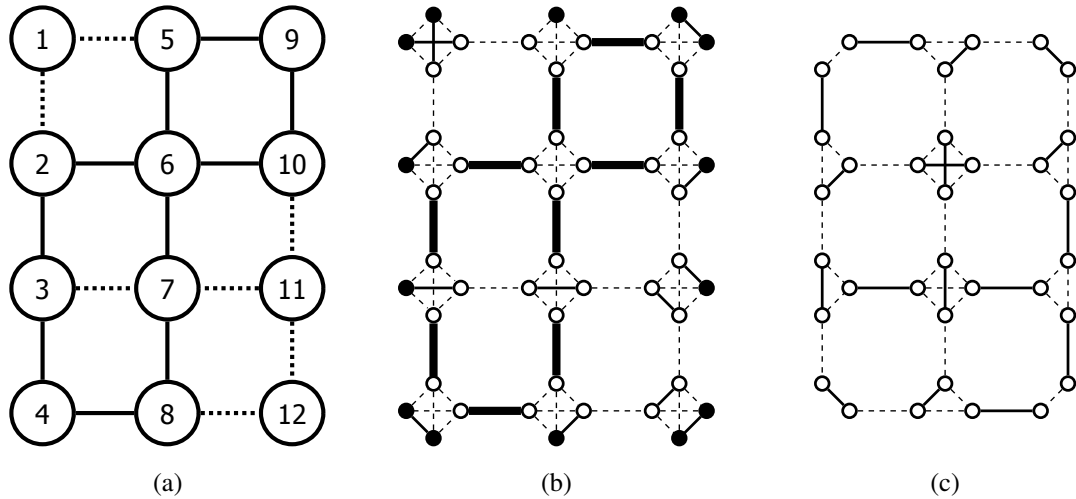


Figure 3.2: (a) An even subgraph (solid edges) of a rectangular lattice G_0 . (b) The corresponding dimer covering (solid edges) of the decorated graph G_d derived from G_0 with a direct mapping to the even subgraph in (a). Edges of the even subgraph are shown as bold dimers, while dummy nodes are shown as black disks. (c) The corresponding dimer covering (solid edges) of the decorated graph G_d with a complementary mapping to the even subgraph in (a).

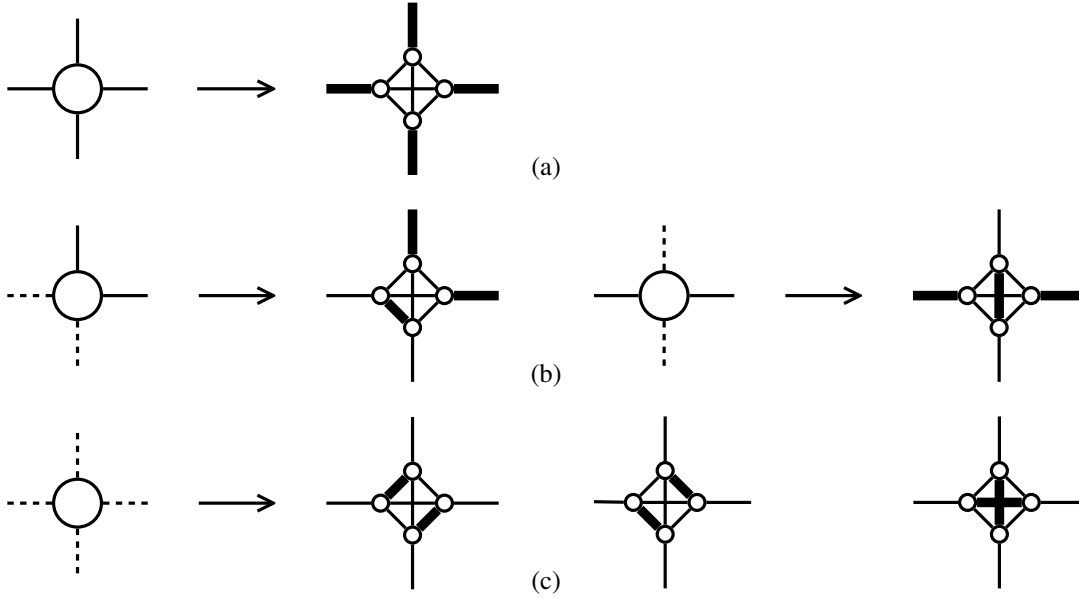


Figure 3.3: Correspondence between even subgraphs in G_0 (solid edges, left) and dimer configurations in Kasteleyn's decorated graph G_d (right) for nodes incident to (a) 4, (b) 2 and (c) 0 edges of an even subgraph. The correspondence is 1:1 for (a) and (b), but 1:3 for (c). Edges added to G_d are internal to the cliques, while the remaining edges in G_d are the original edges from G_0 .

Complementary Mapping

The direct mapping requires the addition of dummy nodes, which makes it difficult to generalize to arbitrary graphs. The alternative is to use a complementary mapping, where edges of an even subgraph in $G_0(\mathcal{V}_0, \mathcal{E}_0)$ correspond to unmatched edges of the perfect matching of $G_d(\mathcal{V}_d, \mathcal{E}_d)$. One possible G_d with a complementary mapping is shown in Figure 3.2(c). Fisher [31] was the first to use a complementary mapping, aiming to simplify Kasteleyn's construction by satisfying the following two conditions:

- All even subgraphs of G_0 are in 1:1 correspondence with dimer coverings of G_d .
- G_d is a planar graph, so it has a simple Pfaffian orientation described in Section 2.3.4.

Fisher [31] transforms $G_0(\mathcal{V}_0, \mathcal{E}_0)$ into a decorated graph $G_d(\mathcal{V}_d, \mathcal{E}_d)$ by replacing every node $i \in \mathcal{V}_0$ with a *Fisher city*. The construction is the following:

1. If $i \in \mathcal{V}_0$ has degree $d > 3$ then replace it with a chain of $(d - 2)$ vertices of degree 3, as shown in Figure 3.4(b); otherwise ($d \leq 3$) do nothing.
2. Replace every node with a d -clique, where d is the degree of the node, as shown in Figure 3.4(c).

Fisher's city contains all the *original* edges of G_0 , as well as added edges that are *internal* to the cliques. This construction plane triangulates G_0 , which we use in our construction (see Section 5.3.1). The complementary 1:1 correspondence is constructed as follows: with the *presence* of an even subgraph edge in G_0 , associate the *absence* of a dimer on the corresponding original edge of G_d . Similarly, with the *absence* of an even subgraph edge associate the *presence* of a dimer. Figure 3.5 shows an example

of the correspondence for a vertex with degree $d = 4$ (for vertices with $d < 4$, see [31]). Notice that G_d is planar and that the allowed dimer configurations on the internal bonds of G_d are always unique, in contrast to Kasteleyn's construction [60].

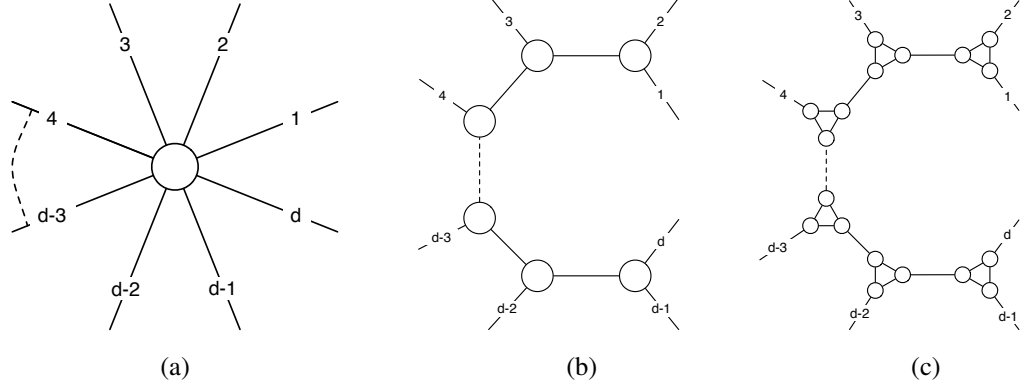


Figure 3.4: Fisher's construction of the decorated graph G_d (c) from the original graph G_0 (a). We show the intermediate step (b) for illustrative purposes only.

Construction of the Kasteleyn Matrix

We now describe how to construct the Kasteleyn matrix to solve the dimer problem on G_d . Let $w(i, j)$ be the weight of edge $(i, j) \in \mathcal{E}_d$ if it is an original edge, otherwise (it is an internal edge) set its weight to $w(i, j) := 0$. We now give $\mathcal{E}_d$ a Pfaffian orientation $\mathcal{E}'_d$ and use it to obtain a skew-symmetric Kasteleyn matrix $\mathbf{K}$, whose Pfaffian is equal to the Ising partition function of G [61]. $\mathbf{K}$ is defined as:

$$\mathbf{K}_{ij} := \begin{cases} +e^{w(i,j)} & \text{if } (i, j) \in \mathcal{E}'_d \\ -e^{w(i,j)} & \text{if } (j, i) \in \mathcal{E}'_d \\ 0 & \text{otherwise} \end{cases} \quad (3.9)$$

If G_d is constructed using Kasteleyn cities then the correspondence between even subgraphs in G_0 and perfect matchings in G_d is a mixture of 1:1 and 1:3 (see Figure 3.3). Furthermore, G_d is not planar, meaning that we cannot solve the dimer problem on G_d directly via the Pfaffian method (Section 2.3.4). Surprisingly however, the combination of non-planarity and lack of 1:1 correspondence for isolated nodes in G_0 does allow us to apply the Pfaffian method successfully. To obtain a Pfaffian orientation of G_d we give a clockwise-odd orientation to the faces of G_d that contain original edges. We then orient the remaining internal edges such that the 3 possible dimer configurations in Figure 3.3(c) are counted correctly as a single configuration (see [31] for more details). The oriented graph for a rectangular grid is shown in Figure 3.6(b).

3.2.3 Non-Planar Ising Models

Kasteleyn [59] also applied his technique to non-planar graphs, specifically a torus modeled as a rectangular lattice with periodic boundary conditions. The key problem with such graphs is that $|\mathbf{K}|$ attaches a

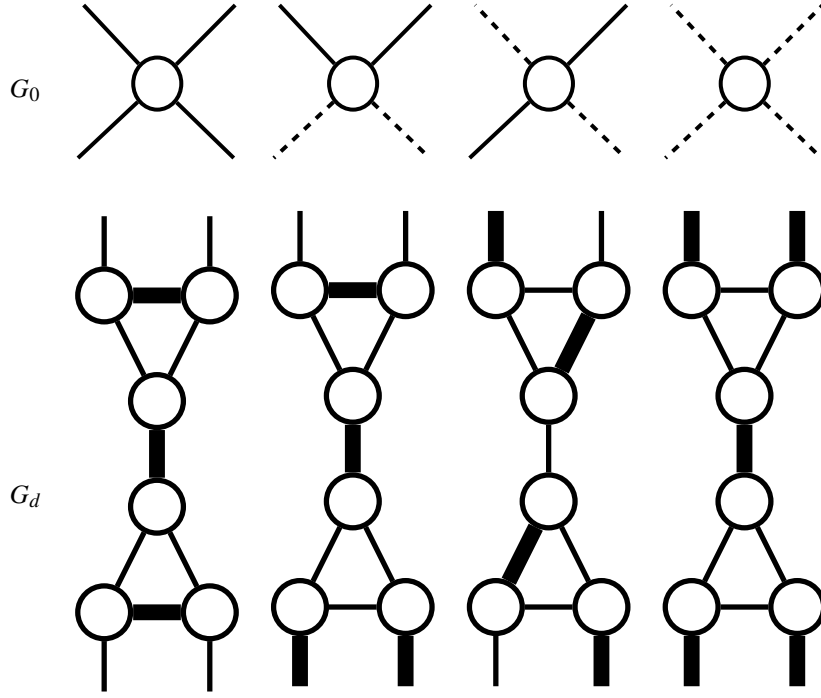


Figure 3.5: Correspondence between even subgraphs (solid edges) at a vertex of degree 4 in G_0 (top) and dimer configurations (bold edges) in Fisher's decorated graph G_d (bottom) (adapted from [31]).

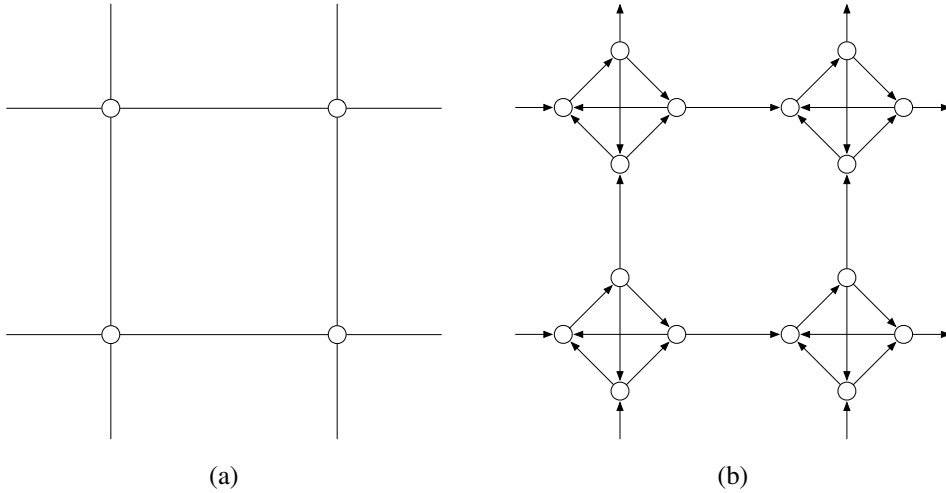


Figure 3.6: (a) A rectangular lattice G_0 . (b) Pfaffian orientation of the corresponding Kasteleyn's decorated graph G_d (adapted from [61]).

negative sign to even subgraphs that wind around the torus (*i.e.*, cross its periodic boundary, as the blue and red cycles do in Figure 7.2) an odd number of times. This means that the corresponding perfect matchings are counted incorrectly. Kasteleyn addressed this problem for a toroidal lattice by calculating

the partition function as

$$Z = \frac{1}{2} \left[-\text{Pf } \mathbf{K} + \text{Pf } h(\mathbf{K}) + \text{Pf } v(\mathbf{K}) + \text{Pf } h(v(\mathbf{K})) \right], \quad (3.10)$$

where $h(\cdot)$ and $v(\cdot)$ switch the sign of entries in $\mathbf{K}$, corresponding to edges crossing the horizontal and vertical periodic boundary, respectively [59, 102]. Kasteleyn realized that Equation 3.10 could be generalized to graphs of arbitrary genus g , but deemed this “practically useless” due to the need for evaluating 4^g determinants [61, p. 98]. This must be seen in light of the lack of computing power in the 1960s, and a wish to calculate partition functions for infinite lattices composed of non-planar units, for which g is unbounded.

Note that $h(\cdot)$ and $v(\cdot)$ preserve the clockwise odd orientation of any underlying graph, as Lemma 18 requires. This raises a troubling question: what distinguishes $\mathbf{K}$, with its negative sign in (3.10), from $h(\mathbf{K})$ or $v(\mathbf{K})$, which could have arisen from the construction of a different clockwise odd orientation?

Indeed (3.10) is necessary but not sufficient to correctly calculate the partition function for toroidal lattices. What is missing is a *lateral parity* condition. The missing condition holds tautologically for Kasteleyn’s particular construction of a toroidal lattice. A correct generalisation to arbitrary non-planar graphs was recently provided by Galluccio and Loeb [36]. Fast algorithms for the computation of the partition function in toroidal lattices and in arbitrary non-planar graphs (with bounded genus), were given in [107] and [37], respectively.

3.3 Previous Work on the Ground States of the Ising Model

Most previous methods compute the ground state of the Ising model by finding the minimum-weight perfect matching on a related dual graph. For a long time the computation of the ground state and the computation of the partition function were treated as two separate problems; Barahona [2] apparently was the first to use a single construction for both.

3.3.1 Work by Bieche et al

Bieche et al. [9] present an algorithm for the rectangular lattice; here we extend their method to general planar graphs. Consider a graph $G(\mathcal{V}, \mathcal{E})$ where each vertex i is assigned a spin $s_i = \pm 1$ and each edge $e = (i, j)$ is assigned a weight $w(e) := -J_{ij}s_i s_j$, where J_{ij} is a real number representing the intensity of the interaction. An edge e is said to be *satisfied* if it attains its minimal possible weight, i.e., when $w(e) = -|J_{ij}|$. Based on J_{ij} one can define the set of *frustrated cycles*:

Definition 25 A frustrated cycle $O \subseteq \mathcal{E}$ of a graph $G(\mathcal{V}, \mathcal{E})$ with edge weights w is a simple cycle (Definition 2) whose product of edge weights is negative, i.e., $\prod_{(i,j) \in O} w(i, j) < 0$.

In other words, a cycle is frustrated if it has no edges with zero weight and an odd number of edges with a negative weight. This means that there is no spin configuration that will satisfy all its edges. From this notion we can also define a *frustrated face* (called *frustrated plaquette* in [9]) to be a frustrated cycle that is also a face in a given embedding. It seems that the term “frustrated face” was introduced by Toulouse [129]

The method of Bieche et al. [9] computes the ground state by pairing up the frustrated faces of G , in such a way that the sum of edge weights along the paths connecting each pair is minimal. This method was first used by Vannimenus and Toulouse [133] who carried out the computation manually. Let $s_{ij} \in \{-1, 1\}$ be the *state* of edge $(i, j) \in \mathcal{E}$. We begin by initializing $s_{ij} := 1$ (agreement edge) if $J_{ij} \geq 0$ and $s_{ij} := -1$ (disagreement edge) otherwise. If there are no frustrated cycles then every edge will be satisfied and we have the ground state, as shown in Figure 3.7. If this is not the case then we must resolve all frustrated cycles.

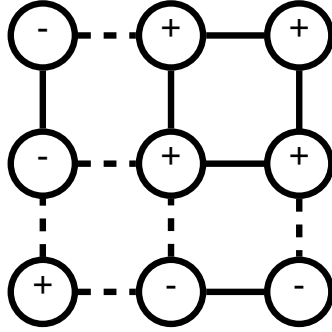


Figure 3.7: A rectangular lattice with no frustrated faces. Edges with non-negative interaction strength are solid, while those with negative interaction strength are dashed. The spin configuration of the ground state is shown for each site.

For convenience we describe the algorithm via the dual graph $G^*(\mathcal{F}, \mathcal{E}^*)$. To resolve a frustrated face f , we need to invert the state of one of its incident edges $e = (f, f')$, which changes $\mathcal{H}(s)$ by $2J_e s_e$. However by doing so, we also change the frustration status of face f' . If f' was already frustrated then we have resolved both f and f' in one shot. Otherwise we have to keep inverting edge states of f' until we find another frustrated face to match it up with. We are thus looking for a path of minimum energy between two frustrated faces. Let $\Pi(f_i, f_j) \subseteq \mathcal{E}^*$ be such a path for two frustrated faces f_i and f_j . The weight of this path is

$$w^*(\Pi(f_i, f_j)) := \sum_{e \in \Pi(f_i, f_j)} -|w(e)|. \quad (3.11)$$

Bieche et al. [9] solve this by finding the minimum perfect matching $\mathcal{M}$ (Definition 14) of the complete graph over frustrated faces $G_f(\mathcal{F}_f, \mathcal{E}_f)$. $\mathcal{F}_f$ is the set of frustrated faces in $\mathcal{F}$, and $\mathcal{E}_f = \mathcal{F}_f \times \mathcal{F}_f$; the weight of edge $(f_i, f_j) \in \mathcal{E}_f$ is $w^*(\Pi(f_i, f_j))$. Note that this perfect matching will always exist, since the number of frustrated faces will be even:

Lemma 26 *The number of frustrated faces in any planar Ising model is even.*

Proof Suppose we have a planar Ising model with $s_{ij} := 1 \forall (i, j) \in \mathcal{E}$. This model has 0 frustrated faces. Inverting the state of an edge negates that edge's weight, which changes the frustration status of exactly two faces, since in a planar graph each edge belongs to two faces. From the given edge state we can reach any other edge state via a sequence of edge state inversions. ■

To obtain the ground state, for each edge $(f_i, f_j) \in \mathcal{M}$ we invert the state of edges in G crossed by the path $\Pi(f_i, f_j)$. The energy of the ground state is given by

$$\mathcal{H}(s) = - \sum_{(i,j) \in \mathcal{E}} |J_{ij}| + 2w(\mathcal{M}), \quad (3.12)$$

where $w(\mathcal{M})$ is the weight of the perfect matching $\mathcal{M}$. For a complete example consider Figure 3.8. The edges whose state is inverted are those crossed by the bold dual edges in Figure 3.8.

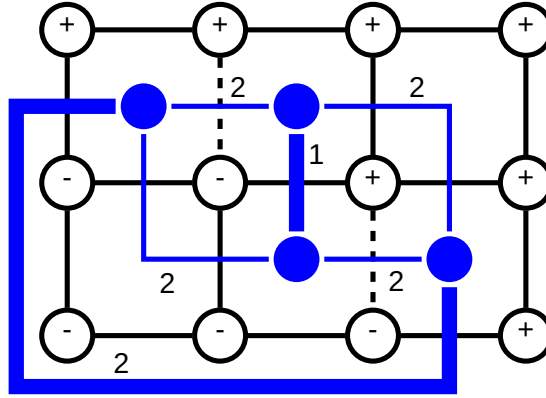


Figure 3.8: Finding the ground state of a planar graph with frustrated faces. Model graph G with white nodes, where dashed edges have disagreement cost -2 and solid edges have cost $+1$. Dual frustration graph G_f with blue nodes, with path weights indicated and the minimum-weight perfect matching $\mathcal{M}$ is shown in bold.

The approach of Bieche et al. [9] suffers from two major drawbacks. Firstly, the computation of all paths of shortest weight in G^* can be prohibitively expensive. Secondly, G_f can be expensive to store as it is a fully connected graph. Currently treatable rectangular lattices are limited to around 300×300 nodes.

3.3.2 Work by Barahona

Barahona [2] constructs the dual graph $G^*(\mathcal{F}, \mathcal{E}^*)$ whose edges have weight $w^*(e) := -|w(e)|$. Since there is a bijection between $\mathcal{E}$ and $\mathcal{E}^*$ we extend the notion of satisfiability: an edge $e^* \in \mathcal{E}^*$ is *satisfied* if its corresponding edge in $\mathcal{E}$ is satisfied, otherwise it is *unsatisfied*. A node $f_i \in \mathcal{F}$ is called *odd* if f_i represents a frustrated face, otherwise it is called *even*. Odd (even) nodes must be adjacent to an odd (even) number of unsatisfied edges.

Barahona [2] then constructs a graph $\tilde{G}$ such that there is a 1:1 correspondence between configurations of unsatisfied edges in G^* and perfect matchings in $\tilde{G}$. To obtain $\tilde{G}$ we first expand G^* as shown in Figure 3.4(b). The next stage is to transform all odd vertices as shown in Figure 3.9 (top) and all even vertices as shown in Figure 3.9 (bottom). All the original edges of G^* retain their weight, while the added edges are given zero weight.

Now each configuration of unsatisfied edges in G^* corresponds to a perfect matching of $\tilde{G}$, as shown in Figure 3.9. The edges of G^* that are in the minimum-weight perfect matching $\mathcal{M}$ of $\tilde{G}$ correspond to

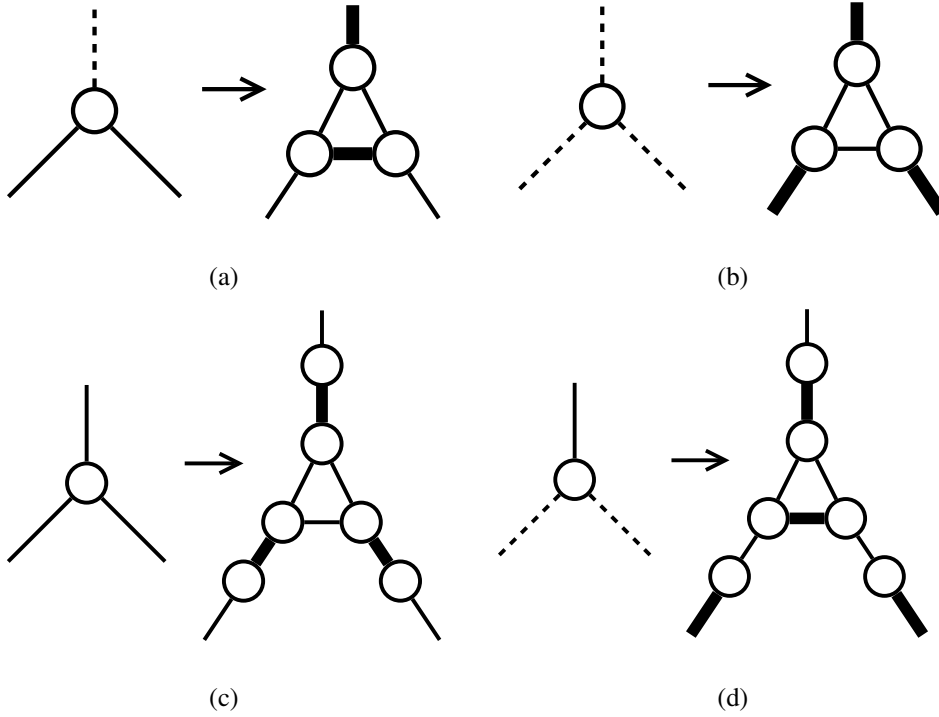


Figure 3.9: Correspondence between unsatisfied edges (dashed) in G^* and perfect matchings (bold) in $\tilde{G}$ for odd vertices (a and b) and even vertices (c and d) (adapted from [2]).

paths of minimum weight between frustrated faces. To obtain the ground state, we follow Bieche et al. [9] and invert the state of edges in G crossed by those paths.

The approach of Barahona [2] is preferable to that of Bieche et al. [9] for larger graphs G . This is because the size of $\tilde{G}$ grows more slowly than the size of G_f , and hence the computation of a minimum-weight perfect matching on $\tilde{G}$ is faster.

3.3.3 Work by Thomas and Middleton

Thomas and Middleton [126] use Kasteleyn cities (Definition 24) to decorate the dual graph G^* of the rectangular lattice G . In contrast to Barahona's construction, the topology of the resulting G_d^* does not depend on the weights of G . The original edges of G retain their weight in G_d^* , while newly introduced edges are given a weight of 0. The minimum-weight perfect matching $\mathcal{M}$ of G_d^* has a direct mapping (Section 3.2.2) to the even subgraph in G^* with the smallest total weight (see Figure 3.2). Edges of the even subgraph in G^* correspond to the set of disagreement edges in G (just rotated by 90°), *i.e.*, a minimum-weight cut of G . We can now use Algorithm 1 to obtain the ground state from this minimum-weight cut. In experiments, Thomas and Middleton [126] found this technique to be about 3 times faster than the method of Bieche et al. [9], with running times similar to those of the method of Barahona [2].

Review of Graphical Models

This chapter reviews probabilistic graphical models, which are necessary for understanding the contribution of this thesis. Here we predominantly focus on undirected graphical models. The expression “graphical model” can mean several things, but the meaning that we use here is the one used in the statistical machine learning community. In particular, it is a model defined by a graph whose nodes represent random variables and whose edges represent the possible dependencies between those random variables. In Section 4.3 we describe the current state-of-the-art inference methods for undirected graphical models.

4.1 Introduction

In recent years graphical models have become an invaluable tool in machine learning. Graphical models have been used with great success in a number of problem domains, ranging from bioinformatics and natural language processing to computer vision. In computer vision, graphical models have been applied to image segmentation and object identification, boundary detection, image restoration, texture modelling, graph matching, stereo reconstruction, digital photomontage, and video segmentation.

A graphical model defines a family of probability distributions that factorize according to an underlying model graph. The nodes of the graph correspond to random variables, while its edges represent statistical dependencies between these variables. Since the probability distribution can be over a large number of variables, the key idea is to represent it as a product over some local functions (*potentials*) each of which depends only on a few of these variables. This allows one to efficiently perform *inference*: compute various statistical quantities such as joint, marginal and conditional probability distributions, as well as maximum a posteriori states. The underlying graph can be directed or undirected. When it is directed, the model is often called a *Belief Network* or *Bayesian Network*, otherwise it is generally called a *Markov Random Field (MRF)*. In this thesis we are primarily interested in undirected graphical models.

Graphical models can be parameterised by computing the MRF’s potentials as inner product between parameters and features. Optimal parameters can be estimated from data (“learned”) according to a number of criteria, such as regularized maximum likelihood or maximum margin. Once the parameters have been learned the MRF can be used to make predictions on unseen data.

4.2 Undirected Graphical Models

An undirected graphical model or MRF is defined over an undirected graph $G(\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \{1, 2, \dots, n\}$ is the set of nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges in G . The MRF observes a set of real-valued *input* random variables X (*features*) with domain $\mathcal{X}$ and models a set of *output* random variables Y with domain $\mathcal{Y}$. $\mathcal{Y}$ can be either continuous or discrete, although in this thesis we will assume that it is always discrete, so $\mathcal{Y} = \{1, \dots, m\}$. In particular, to each node $i \in \mathcal{V}$ we associate an output random variable Y_i , that takes values $y_i \in \mathcal{Y}$.

In general, we aim to model the *joint probability distribution* $\mathbb{P}(Y = \mathbf{y}, X = \mathbf{x})$, although sometimes it is convenient to model the *conditional probability distribution* $\mathbb{P}(Y = \mathbf{y} | X = \mathbf{x})$, like in the case of *Conditional Random Fields* (CRFs)[79]. For simplicity, we assume that the inputs X are always given and so omit them in all subsequent formulae. Furthermore we will write the joint probability distribution $\mathbb{P}(Y = \mathbf{y})$ as $\mathbb{P}(\mathbf{y})$, and $\mathbb{P}(y_i)$ as a shorthand for $\mathbb{P}(Y_i = y_i)$. Let $\mathbf{Y}_A$ denote the vector of output random variables associated with the set of nodes $A \subseteq \mathcal{V}$ and let $\mathbf{y}_A$ be a particular realization of $\mathbf{Y}_A$. The *marginal probability distribution* for $\mathbf{Y}_A$ is computed as:

$$\mathbb{P}(\mathbf{y}_A) = \sum_{\mathbf{y}_{\mathcal{V} \setminus A}} \mathbb{P}(\mathbf{y}). \quad (4.1)$$

The *maximum a posteriori* (MAP) assignment $\mathbf{y}^*$ is the state that maximizes the joint probability distribution:

$$\mathbf{y}^* := \underset{\mathbf{y}}{\operatorname{argmax}} \mathbb{P}(\mathbf{y}). \quad (4.2)$$

4.2.1 Conditional Independence Property

Conditional independence is an important concept for probability distributions over multiple variables, and is the building block for efficient inference in graphical models. Let A, B and C be subsets of nodes in $\mathcal{V}$. A subset of random variables $\mathbf{Y}_A$ is said to be conditionally independent from the subset $\mathbf{Y}_C$ given $\mathbf{Y}_B$, iff given that $\mathbf{Y}_B$ is known the knowledge of $\mathbf{Y}_C$ does not add any information to the knowledge of $\mathbf{Y}_A$. More formally:

Definition 27 $\mathbf{Y}_A$ is conditionally independent of $\mathbf{Y}_C$ given $\mathbf{Y}_B$ iff $\mathbb{P}(\mathbf{y}_A | \mathbf{y}_B, \mathbf{y}_C) = \mathbb{P}(\mathbf{y}_A | \mathbf{y}_B) \forall \mathbf{y}_A, \mathbf{y}_B, \mathbf{y}_C$. We will write this as $(\mathbf{Y}_A \perp\!\!\!\perp \mathbf{Y}_C) | \mathbf{Y}_B$.

As an example, consider a football tournament. Given that we know the current league table, the knowledge of previous match outcomes does not help us in determining the current ranking of a team. So we have (team ranking $\perp\!\!\!\perp$ match outcomes) | league table. If $(\mathbf{Y}_A \perp\!\!\!\perp \mathbf{Y}_C) | \mathbf{Y}_B$ then we have $\mathbb{P}(\mathbf{y}_A, \mathbf{y}_C | \mathbf{y}_B) = \mathbb{P}(\mathbf{y}_A | \mathbf{y}_B) \mathbb{P}(\mathbf{y}_C | \mathbf{y}_B)$.

An important and elegant feature of graphical models is that conditional independence properties of the joint probability distribution can be read directly from the underlying graph. In directed graphical models these properties can be deduced by using the concept of d-separation [99]. In undirected graphical models, $(\mathbf{Y}_A \perp\!\!\!\perp \mathbf{Y}_C) | \mathbf{Y}_B$ is true iff every path from any node in A to any node in C passes through at least one node in B ; all such paths are then said to be *blocked* by B .

4.2.2 Factorization Property

Let $\mathcal{K}$ be the set of maximal cliques (Definition 1) of the model graph G . A *potential function* ψ_C of a clique $C \in \mathcal{K}$ is a function that associates a positive real number to every assignment of labels of nodes in C . The joint probability distribution can be written as a product of potential functions $\psi_C(\mathbf{y}_C)$ over the maximal cliques:

$$\mathbb{P}(\mathbf{y}) = \frac{1}{Z} \prod_{C \in \mathcal{K}} \psi_C(\mathbf{y}_C), \quad \text{where } Z := \sum_{\mathbf{y}} \prod_{C \in \mathcal{K}} \psi_C(\mathbf{y}_C) \quad (4.3)$$

is the *partition function* which ensures that $\mathbb{P}(\mathbf{y})$ is correctly normalized. The presence of this normalization constant is one of the major limitations of undirected graphical models, because its naive evaluation involves summing over $|\mathcal{Y}|^{|V|}$ states. The partition function is needed for maximum-likelihood parameter estimation, but not for computing the MAP states or maximum margin parameter estimation. The positivity of $\psi_C(\mathbf{y}_C)$ ensures that $\mathbb{P}(\mathbf{y})$ is non-negative. To make $\psi_C(\mathbf{y}_C)$ strictly positive it is often convenient to express it as an exponential function:

$$\psi_C(\mathbf{y}_C) := \exp(-E(\mathbf{y}_C)), \quad (4.4)$$

where $E(\mathbf{y}_C)$ is the *energy function* of the clique C . The energy function of the joint probability distribution can be written as $E(\mathbf{y}) = \sum_{C \in \mathcal{K}} E(\mathbf{y}_C)$, which allows us to conveniently rewrite Equation 4.3 as

$$\mathbb{P}(\mathbf{y}) = \frac{1}{Z} \exp\left(-\sum_{C \in \mathcal{K}} E(\mathbf{y}_C)\right) = \frac{1}{Z} e^{-E(\mathbf{y})}. \quad (4.5)$$

Using the fact that $\mathbb{P}(\mathbf{y}) \propto \exp(-E(\mathbf{y}))$, we can rewrite the MAP definition (4.2), so that it matches the definition (3.6) of a ground state of an Ising model:

$$\mathbf{y}^* = \underset{\mathbf{y}}{\operatorname{argmax}} e^{-E(\mathbf{y})} = \underset{\mathbf{y}}{\operatorname{argmin}} E(\mathbf{y}). \quad (4.6)$$

The true power of graphical models comes from the fact that the conditional independence property implies the factorization property and vice-versa, as stated by the *Hammersley-Clifford* theorem [7, 44]:

Theorem 28 (Hammersley-Clifford Theorem) *Any strictly positive probability distribution which is consistent with the conditional independence properties imposed by a graph G also factorizes with respect to G .*

This theorem tells us that any strictly positive probability distribution that respects the conditional independence properties of some graph G can be factorized as a product over functions ψ , which are local in that each only involves the nodes of one maximal clique of G .

4.3 Inference Algorithms

For a general undirected graphical model the inference task is NP-hard. However, efficient algorithms exist for certain types of models. The complexity of inference is governed by the following factors: graph structure, form of the energy function and the number of output labels. In this section we will present the most popular algorithms used for inference in undirected graphical models. Variable elimination (Section 4.3.1) is an exact algorithm for small model graphs. Loopy Belief Propagation (Section 4.3.3) is a commonly-used approximate inference method, which is exact for trees. Its cousin, the Junction Tree algorithm (Section 4.3.4), is an exact inference method for “thin” graphs or graphs with small *treewidth*. Graph cuts (Section 4.3.5) are an exact inference method for models with *submodular* energy functions (Definition 31) and binary labels. Tree Reweighting (Section 4.3.6) is an approximate inference method that often outperforms standard Loopy Belief Propagation and has better convergence guarantees.

4.3.1 Variable Elimination

Suppose we want to evaluate the marginal probability distribution $\mathbb{P}(y_i)$ on a graph with n nodes. We can find $\mathbb{P}(y_i)$ by *marginalizing out* all the random variables in the joint probability distribution $\mathbb{P}(\mathbf{y})$ that are not y_i :

$$\mathbb{P}(y_i) = \frac{1}{Z} \sum_{\mathbf{y} \setminus \{y_i\}} \mathbb{P}(\mathbf{y}) \quad (4.7)$$

$$= \frac{1}{Z} \sum_{y_1} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_n} \mathbb{P}(Y_1 = y_1, \dots, Y_n = y_n). \quad (4.8)$$

Naively, we could first evaluate the joint probability distribution for all possible configurations and then perform all the summations. If each variable has m states then we will need $O(m^n)$ space to store the joint probability distribution, followed by m^n summations.

We can obtain a much more efficient algorithm by exploiting the conditional independence properties of the graphical model. If we substitute Equation 4.3 for $\mathbb{P}(\mathbf{y})$, then we can reorder the summations and multiplications to minimize the total number of computations. The method is best demonstrated with an example. Consider the tree in Figure 4.1(a), for which we want to compute $\mathbb{P}(y_5)$. Based on the factorization properties we can write the computation as

$$\mathbb{P}(y_5) = \frac{1}{Z} \sum_{y_1} \sum_{y_2} \sum_{y_3} \sum_{y_4} \psi_{12}(y_{12}) \psi_{23}(y_{23}) \psi_{34}(y_{34}) \psi_{35}(y_{35}). \quad (4.9)$$

We notice that y_1 participates in just one term, so it makes sense to sum over y_1 first. After we have summed over (or *eliminated*) y_1 , we can sum over the remaining variables. Again we will choose to eliminate a variable that occurs in the least number of terms. We continue this process until all variables have been eliminated. We can now rewrite Equation 4.9:

$$\mathbb{P}(y_5) = \frac{1}{Z} \sum_{y_3} \psi_{35}(y_{35}) \sum_{y_4} \psi_{34}(y_{34}) \sum_{y_2} \psi_{23}(y_{23}) \sum_{y_1} \psi_{12}(y_{12}). \quad (4.10)$$

In the above we are exploiting the fact that multiplication is distributive over addition, so in

$$ab + ac = a(b + c), \quad (4.11)$$

the right-hand-side is preferable as it requires only 2 operations, compared to the 3 operations (2 multiplications and one addition) required by the left-hand-side. The resulting term of $\sum_{y_i} \psi_{ij}(y_{ij})$ is a function of j , which for convenience we will write as $m_{ij}(y_j)$. The index i refers to the variable being summed over, while the index j refers to other variables appearing in the summand. Note that for trees, there is never more than two variables in the summand, since trees have a maximum clique size of 2. We continue the derivation using the new notation:

$$\begin{aligned} \mathbb{P}(y_5) &= \frac{1}{Z} \sum_{y_3} \psi_{35}(y_{35}) \sum_{y_4} \psi_{34}(y_{34}) \sum_{y_2} \psi_{23}(y_{23}) m_{12}(y_2) \\ &= \frac{1}{Z} \sum_{y_3} \psi_{35}(y_{35}) \sum_{y_4} \psi_{34}(y_{34}) m_{23}(y_3) \\ &= \frac{1}{Z} \sum_{y_3} \psi_{35}(y_{35}) m_{23}(y_3) \sum_{y_4} \psi_{34}(y_{34}) \\ &= \frac{1}{Z} \sum_{y_3} \psi_{35}(y_{35}) m_{23}(y_3) m_{43}(y_3) \\ &= \frac{1}{Z} m_{35}(y_5). \end{aligned} \quad (4.12)$$

The final expression is a function of y_5 only, and thus is the desired marginal. This algorithm is usually called *variable elimination* as the variables are *eliminated* one by one.

4.3.2 Belief Propagation

In many problems we would like to compute more than a single marginal probability. For example in Figure 4.1(a) we might want to obtain both $\mathbb{P}(y_4)$ and $\mathbb{P}(y_5)$. We could compute each marginal separately using variable elimination, but would end up computing same $m_{ij}(y_j)$ twice, such as $m_{12}(y_2)$ and $m_{23}(y_3)$.

Belief Propagation (aka the *sum-product algorithm*) [99] is a method that efficiently computes all the possible $m_{ij}(y_j)$, thus allowing reuse of intermediate results for multiple computations. The intermediate term $m_{ij}(y_j)$ can be viewed as a “message” that is passed from node i to node j about what state node j should be in, written as:

$$m_{ij}(y_j) := \sum_{Y_i} \psi_{ij}(y_{ij}) \prod_{k \in N(i) \setminus j} m_{ki}(y_i), \quad (4.13)$$

where $N(i)$ is the set of neighbours of node i . Node i can send a message to a neighbouring node j once it has received all the incoming messages from its neighbours (except from node j). This set of incoming messages is computed as $\prod_{k \in N(i) \setminus j} m_{ki}(y_i)$ in Equation 4.13. Since leaf nodes have only one neighbour, they can start sending messages straight away. Belief propagation proceeds by propagating messages from the leaves to the root. The algorithm terminates once a message has been sent along each edge in both directions. It can be shown that belief propagation always converges to the optimum solution for

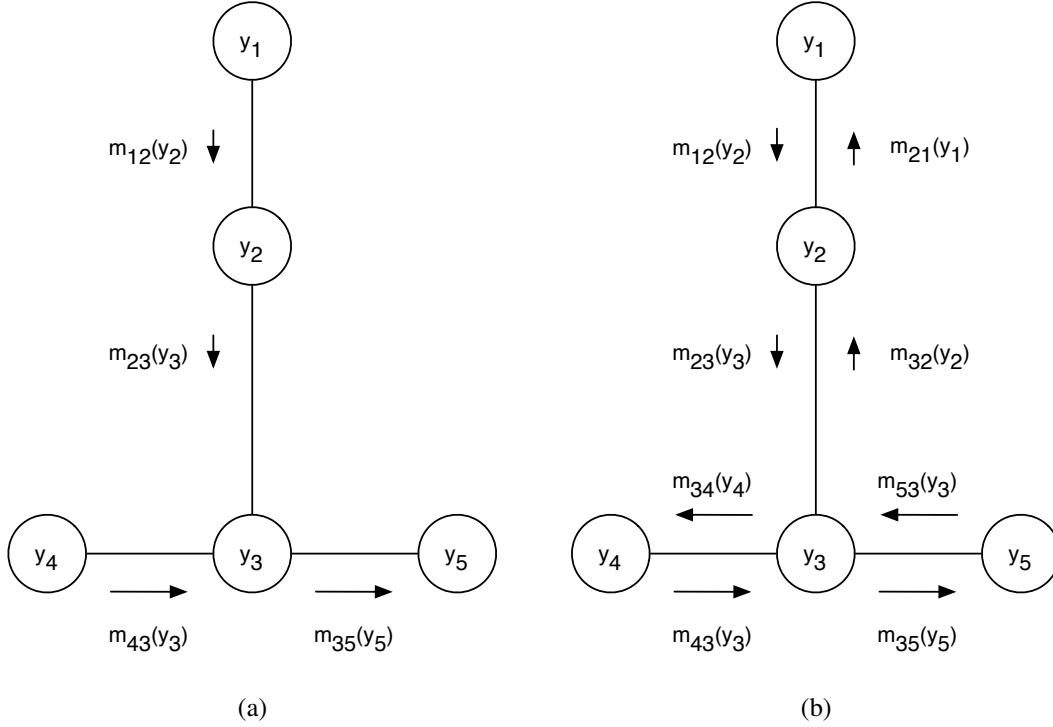


Figure 4.1: An undirected graphical model whose underlying graph is a tree. (a) Shows the intermediate messages that arise during variable elimination with elimination order: y_1, y_2, y_4, y_3 . (b) Shows the set of all messages computed by the belief propagation algorithm.

trees. Figure 4.1(b) shows the set of messages needed to compute every individual marginal probability. Ignoring normalization terms, the final marginal probability at a node i is given by the product of all its incoming messages:

$$\mathbb{P}(y_i) \propto \prod_{k \in N(i)} m_{ki}(y_i). \quad (4.14)$$

The sum-product algorithm can be easily modified to compute the probability of the MAP state. The idea is to use Equation 4.13, replacing every instance of $\sum$ operator with the max operator. This works, since maximization commutes with products, just as summation does, *i.e.*, $\max(ab, ac) = a \max(b, c)$. The resulting algorithm is called the *max-product algorithm*. In practice, products of many small probabilities can lead to numerical underflow problems. These problems can be avoided if one computes the logarithm of the joint probability distribution. Since the logarithm is a monotonic function, we can write $\operatorname{argmax}_y \log(\mathbb{P}(\mathbf{y})) = \operatorname{argmax}_y \mathbb{P}(\mathbf{y})$. This has the effect of replacing the products in the max-product algorithm with sums. This is called the *max-sum algorithm*.

4.3.3 Loopy Belief Propagation

It can be shown that belief propagation obtains the optimum solution for graphs containing a single loop. For general graphs, however, belief propagation is not guaranteed to converge to the optimum

solution. Loosely speaking, this is because general graphs may contain numerous loops and thus the same evidence can be passed around multiple times and mistaken for new evidence. Despite the lack of formal convergence guarantees, belief propagation is still widely used for general graphs, such as grids. In this case the method is called *Loopy Belief Propagation* [34, 137, 143].

From Section 4.3.2, we have seen that a node can pass messages once it has received messages from all its neighbours. Since there can be loops in the graph, we need to decide how to initiate the message passing algorithm. One possibility is to send an initial message given by the unit function across every edge in every direction. Now each node is in a position to start sending messages. We also have that $m_{ij}(y_j)$ can be computed multiple times for the same i and j . Let us assume that $m_{ij}(y_j)$ sent at time t replaces any previously sent $m_{ij}(y_j)$.

There are numerous ways to define a message passing schedule: *e.g.*, *flooding schedule* passes a message across every edge in both directions at every time step, while a *serial schedule* passes one message at a time. Kschischang et al. [75] describes one such serial schedule. Node i has a message *pending* to node j if node i has received any message since the last time it sent anything to node j . Only pending messages need to be transmitted, because all other messages would duplicate previously sent messages. The algorithm proceeds by sending all pending messages one by one. For trees this will terminate once a message has passed in each direction of each edge. For graphs with loops this might never terminate. Once the algorithm has terminated (due to convergence or otherwise), the final marginal of node i is computed as a product of the most recently received messages to node i . Note that this marginal is only an approximation of the true marginal.

4.3.4 Junction Tree Algorithm

Belief propagation can be generalised to general graphs, giving an exact inference method known as the *Junction Tree algorithm* [81]. The idea is to construct a tree-based data structure, called a *junction tree*, whose nodes are cliques of the original graph. Belief propagation can then be used to compute exact results on this tree. There are several inference algorithms available [83]: Hugin, Shenoy-Shafer and Lauritzen-Spiegelhalter. However, all these algorithms perform exact inference and only differ in computational speed. Below we will describe the Shenoy-Shafer algorithm [114, 115, 116]. If the original graph is directed then we must first make it undirected via a method called *moralization*. The first step in constructing the junction tree involves transforming the model graph $G(\mathcal{V}, \mathcal{E})$ into a *chordal* graph $G_c(\mathcal{V}, \mathcal{E}_c)$:

Definition 29 A graph is *chordal* if each of its cycles of four or more nodes has a chord. A chord in a cycle is an edge joining two nodes that are not adjacent in that cycle.

We can construct G_c by adding edges to G . An example of a chordal graph is shown in Figure 5.6(a). We can now construct the junction tree:

Definition 30 The junction tree $\mathcal{T}(\mathcal{K}, \mathcal{E}')$ of an undirected chordal graph G_c is a tree, whose every node $i \in \mathcal{K}$ corresponds to a maximal clique of G_c . The edges $\mathcal{E}'$ satisfy the running intersection property: for any edge $(i, k) = (C_i, C_k) \in \mathcal{E}'$, the variables in the intersection $i \cap k$ are contained in every node of the tree on the unique path between i and k .

The junction tree is maximal in the sense that the number of variables shared by its two connected nodes is maximal. Each maximal clique in G_c is associated with exactly one node in $\mathcal{T}$. We set the potential of a clique C to the product of potentials of its sub-cliques:

$$\psi_C(\mathbf{y}_C) := \prod_{C_i \subset C} \psi_{C_i}(\mathbf{y}_{C_i}). \quad (4.15)$$

Instead of sending messages between nodes (4.13), we now send messages between cliques. A message from clique C_i to C_j is given as

$$m_{C_i C_j}(\mathbf{y}_{C_{ij}}) := \sum_{C_i \setminus C_{ij}} \psi_{C_i}(\mathbf{y}_{C_i}) \prod_{C_k \in N(C_i) \setminus C_j} m_{C_k C_i}(\mathbf{y}_{C_{ki}}), \quad (4.16)$$

where $C_{ij} := C_i \cap C_j$ and $N(C_i)$ are the neighbours of clique C_i in the junction tree. Similar to Equation 4.14, we can compute the marginal probability of a clique C_i as a product of all its incoming messages:

$$\mathbb{P}(\mathbf{y}_{C_i}) \propto \prod_{C_k \in N(C_i)} m_{C_k C_i}(\mathbf{y}_{C_{ki}}). \quad (4.17)$$

Finally, we can obtain the marginal probability for an individual node y_i in clique C :

$$\mathbb{P}(y_i) = \sum_{C \ni i} \mathbb{P}(\mathbf{y}_C). \quad (4.18)$$

Once again the problem of computing the probability of the MAP state can be solved by replacing $\sum$ with $\max$. Since the junction tree algorithm works with joint probability distributions of nodes in each clique, its computational complexity is exponential in the number of nodes in its largest clique. This number is closely related to the *treewidth* of a graph G , which is defined as one less than the size of the maximal clique in any chordal graph containing G . Thus the complexity of the junction tree algorithm is exponential in the treewidth of its underlying graph. The treewidth of a tree is 1, while the treewidth of an $n \times n$ grid is n .

4.3.5 Graph Cuts

The widespread use of *graph cuts* to find lowest-energy configurations, particularly in computer vision, was pioneered by Greig et al. [41]. Given an undirected graphical model with graph $G(\mathcal{V}, \mathcal{E})$, the algorithm aims to minimize the following real-valued energy function

$$E(\mathbf{y}) := \sum_{i \in \mathcal{V}} E_i(y_i) + \sum_{(i,j) \in \mathcal{E}} E_{ij}(y_i, y_j), \quad (4.19)$$

where $\mathbf{y} = \{y_1, \dots, y_{|\mathcal{V}|}\}$ is the set of node labels, $E_i(y_i)$ is the cost of assigning node i to label y_i , and $E_{ij}(y_i, y_j)$ is the cost of assigning the adjacent nodes i and j to the labels y_i and y_j , respectively. In general this problem is NP-hard, but is tractable for binary-labeled models with a *submodular* energy function E :

Definition 31 A function E of the form (4.19) is submodular iff each term E_{ij} satisfies the inequality:

$$E_{ij}(0, 0) + E_{ij}(1, 1) \leq E_{ij}(0, 1) + E_{ij}(1, 0).$$

For the purpose of the following discussion, we assume that the above conditions hold. The graph cut algorithm constructs a directed graph $G'(\mathcal{V}', \mathcal{E}')$ such that the weight of its minimum s-t cut corresponds to the global minimum of E [70].

Definition 32 An s-t cut $C(S, T)$ of a directed graph $G'(\mathcal{V}', \mathcal{E}')$ is a partition of vertices $\mathcal{V}'$ into two disjoint sets S and T , such that $s \in S$ and $t \in T$. The weight of the s-t cut $w(C(S, T))$ is the sum of weights of boundary edges $(p, q) \in \mathcal{E}'$, where $p \in S$ and $q \in T$.

The minimum s-t cut is an s-t cut with the smallest weight. Every s-t cut partitions the nodes into two sets S and T . Those nodes that are in S are labeled 0, while the remaining nodes are labeled 1. Due to the theorem of Ford and Fulkerson [32], the minimum s-t cut is equivalent to the maximum flow from s to t . By construction, all edges of $\mathcal{E}'$ are non-negative, thus the maximum flow of G' can be computed in polynomial time, and hence also its minimum s-t cut. G' is constructed from G as follows:

1. Add 2 extra terminal vertices s (source) and t (sink). Thus we have $\mathcal{V}' := \{s, t, 1, \dots, |\mathcal{V}|\}$.
2. For each node cost $E_i(y_i)$ we add an edge from node i to one of the terminal nodes. If $E_i(1) > E_i(0)$ then we add the edge (s, i) with the weight $E_i(1) - E_i(0)$. Otherwise, we add the edge (i, t) with the weight $E_i(0) - E_i(1)$ (Figure 4.2(a)).
3. For each edge cost $E_{ij}(y_i, y_j)$ we add an edge (i, j) with weight $E_{ij}(0, 1) + E_{ij}(1, 0) - E_{ij}(0, 0) - E_{ij}(1, 1) \geq 0$. If $E_{ij}(1, 0) > E_{ij}(0, 0)$ then we add a weight of $E_{ij}(1, 0) - E_{ij}(0, 0)$ to edge (s, i) , otherwise we add a weight of $E_{ij}(0, 0) - E_{ij}(1, 0)$ to edge (i, t) . If $E_{ij}(1, 0) > E_{ij}(1, 1)$ then we add a weight of $E_{ij}(1, 0) - E_{ij}(1, 1)$ to edge (j, t) , otherwise we add a weight of $E_{ij}(1, 1) - E_{ij}(1, 0)$ to edge (s, j) . Figure 4.2(b) shows the graph for $E_{ij}(y_i, y_j)$ for the case when $E_{ij}(1, 0) > E_{ij}(0, 0)$ and $E_{ij}(1, 0) > E_{ij}(1, 1)$.

A more elaborate construction can give partial answers for non-submodular energy functions, *i.e.*, graphs with some negative edge weights [43, 68, 105]. Furthermore, a sequence of *expansion moves* (energy minimizations in binary graphs) can efficiently yield an approximate answer for graphs with discrete but non-binary node labels [12].

4.3.6 Tree Reweighting

Given an undirected graphical model with graph $G(\mathcal{V}, \mathcal{E})$, Wainwright et al. [136] compute the MAP energy by maximizing a concave lower bound derived from a convex combination of tree-structured distributions. Let us assume that every $E(y_C)$ decomposes into a product of parameters θ_C and *sufficient statistics* $\phi_C(y_C)$. We can now write $E(y, \theta) = \langle \theta, \phi(y) \rangle$, where θ is the set of all parameters and $\phi(y)$ is the set of all sufficient statistics. The energy of the optimal configuration y^* can be written as $E(y^*, \theta) = \min_y E(y, \theta)$.

Let $\mathcal{T}$ be the set of spanning trees of G , such that each edge appears in at least one tree. Let $T \in \mathcal{T}$ be one of the spanning trees and $\theta(T)$ be its set of parameters. Let $\tilde{\theta}$ be a concatenation of all such parameter sets $\theta(T)$. We associate a weight $p(T)$ with every tree T , such that $p(T) \geq 0$ and

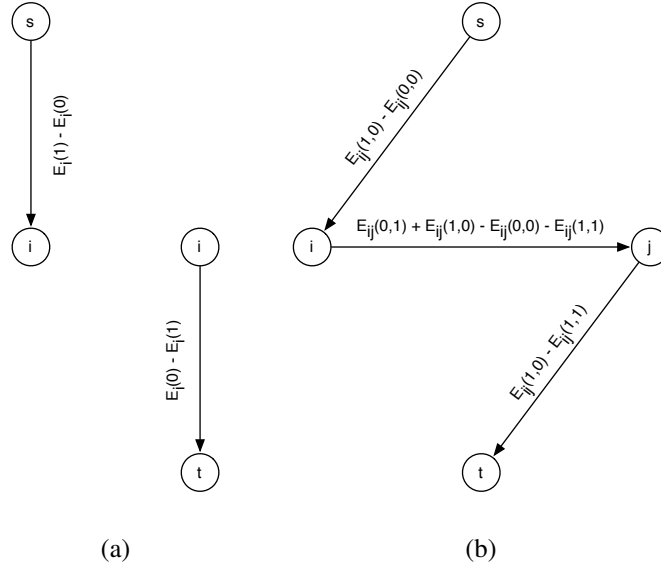


Figure 4.2: Graph cut construction of G' for node i (a) and edge (i, j) (b).

$\sum_{T \in \mathcal{T}} p(T) = 1$. If we set $\theta(T)$ such that $\sum_T p(T)\theta(T) = \theta$ then applying Jensen's inequality yields the lower bound

$$E(\mathbf{y}^*, \theta) \geq \sum_{T \in \mathcal{T}} p(T) E(\mathbf{y}^*, \theta(T)). \quad (4.20)$$

This means that we can compute a lower bound on $E(\mathbf{y}^*, \theta)$ via a convex sum of ground energies of individual trees, which can be found exactly. It turns out that equality in Equation 4.20 holds if the collection of trees share a common optimum. This is known as the *tree agreement* property. Wainwright et al. [136] aim to find the tightest lower bound of the form (4.20). The problem is first posed as a linear programming (LP) relaxation in the primal:

$$\begin{aligned} \max_{\theta} \quad & \sum_{T \in \mathcal{T}} p(T) E(\mathbf{y}^*, \theta(T)) \\ \text{such that} \quad & \sum_{T \in \mathcal{T}} p(T) \theta(T) = \theta. \end{aligned} \quad (4.21)$$

Specialized versions of this LP relaxation were formulated earlier in [16, 73, 108]. For a general distribution of trees, the primal problem (4.21) entails maximizing a sum of functions over all spanning trees of the graph, which can be a prohibitively large collection. Although this LP relaxation can be solved in polynomial time using interior-point methods, due to their large memory requirements the current state of the art software can only handle instances of a few hundred variables. For example, Meltzer et al. [90] report that the largest grid they could solve with such a method is only 39×39 . Luckily the primal can be solved via its Lagrangian dual. This involves iterating over a relatively simple *local consistency polytope* $LOCAL(G)$. In particular, $LOCAL(G)$ can be characterized by $O(m|\mathcal{V}| +$

$m^2|\mathcal{E}|$) constraints, where m is the number of possible label assignments per node. A nice consequence of this duality is the fact that the optimal solution does not depend on the choice of trees and their probabilities, provided that each edge appears in $\mathcal{T}$.

For solving the dual problem on $LOCAL(G)$ Wainwright et al. [136] propose two max-product tree-reweighted message passing algorithms (TRW): edge-based updates (TRW-E) and tree-based updates (TRW-T). TRW-T iterates between two phases: a) running max-product for all trees and b) performing averaging operations for all nodes and edges. TRW-E is similar, except that in phase a) it performs one parallel message update for all trees. The algorithms reduce to the ordinary max-product algorithm when applied to trees. Kolmogorov [66] shows that TRW-T and TRW-E can sometimes increase the lower bound and so do not guarantee convergence. He introduces a sequential update algorithm TRW-S which guarantees that the value of the lower bound does not decrease and converges to a solution that satisfies the *weak tree agreement* criterion (WTA) [66]. Although his experiments suggest that this can produce high-energy solutions for non-submodular energy functions. Kolmogorov [66] also introduces BP-S — ordinary max-product belief propagation algorithm with the same sequential schedule of updating messages as TRW-S.

Kolmogorov and Wainwright [69] show that tree-reweighted message passing is guaranteed to obtain the global optimum for submodular energy functions with binary variables.

4.3.7 Work by Globerson and Jaakkola

Globerson and Jaakkola [38] follow TRW’s approach and compute a lower bound of the partition function by decomposing the model graph into disjoint planar subgraphs. They adopt and simplify the classical constructions of Kasteleyn [59, 60, 61] and Fisher [30, 31] to perform exact computation of the partition function (and marginals) for each planar subgraph. The model graph is first *plane triangulated* (Definition 40) and then used to construct the dual graph G^* . Since G is plane triangulated, all vertices in G^* have degree 3, which is convenient for the next step of constructing the terminal graph G^{T*} of G^* . The original edges of G^* (and G) retain their weight in G^{T*} , while the introduced edges are given a weight of $e^0 = 1$. As shown by Fisher [31], there is now a 1:1 correspondence between perfect matchings in G^{T*} and configurations of even subgraphs in G . Now the partition function of G can be computed as a sum over the weights of perfect matchings in G^{T*} (Section 2.3.4). G^{T*} is given a Pfaffian orientation, from which the Kasteleyn matrix is constructed. The partition function of the model graph is then computed via the determinant of the Kasteleyn matrix. If $\mathcal{R}$ is the set of disjoint planar subgraphs of G then the lower bound of the log-partition function is given as

$$\log Z(\theta) \leq \sum_{R \in \mathcal{R}} p(R) \log Z(\theta(R)). \quad (4.22)$$

Our work is based on [38], although it simplifies it in a number of important ways. Our method does not require the construction of the expanded dual graph as everything is done in the model graph. Also, we break the construction of the Kasteleyn matrix into two phases, the first of which is invariant with respect to the model’s disagreement costs. Further details are discussed in the next chapter.

Planar Ising Graphical Models:

Inference

In this chapter we introduce the Ising graphical model. This is an undirected graphical model that obeys the constraints of the Ising model, *i.e.*, a binary-labeled energy function that is a sum of edge disagreement costs. Here we focus on the *planar* Ising graphical model, while in Chapter 7 we will look at its extensions to *non-planar* graphs. For the planar model we give polynomial-time algorithms for the exact computation of MAP (ground) states, log partition function and marginal edge probabilities. Our approach provides an interesting alternative to the well-known graph cut paradigm in that it does not impose any submodularity constraints. Instead, we require planarity to establish a correspondence with perfect matchings in an expanded dual graph. We describe a unified framework, while delegating complex but well-understood subproblems (planar embedding, maximum-weight perfect matching) to established algorithms, for which efficient implementations are freely available.

In Section 5.1 we show that an Ising model with an additional bias node can express any arbitrary binary energy function. In Section 5.1.2 we explain the planarity requirement and how it is affected by this additional bias node. In Section 5.1.4 we show that inference in a general planar graph can be broken down into inference in each of its biconnected components. Section 5.2 describes the expanded dual graph (Section 5.2.1), and how it is used for computing the MAP state⁴ of the model graph. In Section 5.2.2 we give the precise complementary connection between graph cuts of the model graph and perfect matchings of the expanded dual. We explain the role of planarity (*resp.* triangulation) in making that relation a surjection (*resp.* bijection). In Section 5.3 we deal with the calculation of the partition function and marginal probabilities. By using a particular indexing scheme (Section 5.3.3) we construct a correspondence between the edges of the model graph and those of its expanded dual. This is simpler than earlier work, as it eliminates the need to construct the expanded dual graph. In Section 5.3.3 we introduce the “proto-Kasteleyn” matrix $\mathbf{H}$, which can be efficiently stored as a prefactored bit matrix and reused in the parameter estimation loop. We show how to prefactor Kasteleyn matrices (Section 5.3.4), which reduces storage and speeds up computation of marginal probabilities. In Chapter 6 we apply all of the above algorithms to real examples of machine learning problems.

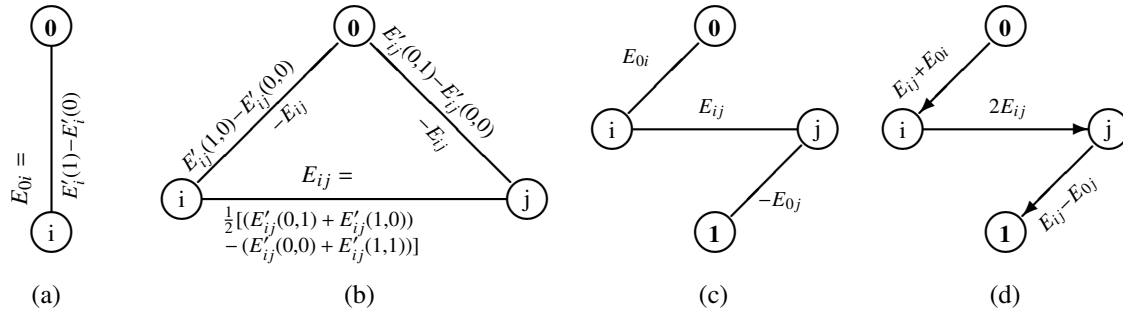


Figure 5.1: Constructive proof of Theorem 33: conversion of general energy (5.3) into Ising energy (5.1) via addition of a bias node. (a) shows the change in node energy E'_i . (b) shows the change in edge energy E'_{ij} . (c) our construction of an equivalent submodular model for $E_{ij} > 0$, $E_{0i} > 0$ and $E_{0j} < 0$. (d) equivalent directed model of Kolmogorov and Zabih [70, Figure 2d].

5.1 Introduction

The *Ising Graphical Model* is a binary graphical model defined on an undirected graph $G(\mathcal{V}, \mathcal{E})$ whose energy function takes the form $E : \{0, 1\}^{|\mathcal{E}|} \rightarrow \mathbb{R}$ with

$$E(\mathbf{y}) := \sum_{(i,j) \in \mathcal{E}} \llbracket y_i \neq y_j \rrbracket E_{ij}, \quad (5.1)$$

where $\llbracket \cdot \rrbracket$ denotes the indicator function, *i.e.*, the *disagreement cost* E_{ij} is incurred only in those states where y_i and y_j disagree. The joint probability distribution of an Ising graphical model is the same as that of a MRF (4.5):

$$\mathbb{P}(\mathbf{y}) = \frac{1}{Z} e^{-E(\mathbf{y})}, \quad \text{where } Z := \sum_{\mathbf{y}} e^{-E(\mathbf{y})} \quad (5.2)$$

is the *partition function*. Recall that the energy function (4.19) of a general graphical model has the form

$$E'(\mathbf{y}) := \sum_{i \in \mathcal{V}} E'_i(y_i) + \sum_{(i,j) \in \mathcal{E}} E'_{ij}(y_i, y_j). \quad (5.3)$$

Compared to this general energy function, (5.1) with binary node labels imposes two additional restrictions: zero node energies, and edge energies in the form of disagreement costs. At first glance these constraints look severe. For example, such systems must obey the *label symmetry* $E(\mathbf{y}) = E(\neg \mathbf{y})$, where $\neg$ denotes Boolean negation (ones' complement). However, Globerson and Jaakkola [38] suggested that adding a single node makes the Ising model (5.1) as expressive as the general model (5.3) for binary variables. Here we provide a formal proof:

Theorem 33 *Every energy function of the form (5.3) over n binary variables is equivalent to an Ising energy function of the form (5.1) over $n + 1$ variables, with the additional variable held constant.*

Proof We will prove this theorem by construction. Two energy functions are equivalent if they differ only by a constant. Without loss of generality, we add an extra *bias* variable y_0 and hold it constant at $y_0 := 0$. Given an energy function of the form (5.3), we construct an Ising model with disagreement costs as follows:

1. For each node energy function $E'_i(y_i)$, add a disagreement cost of $E_{0i} := E'_i(1) - E'_i(0)$, as shown in Figure 5.1(a). E_{0i} refers to the disagreement cost of the edge between node i and the bias node. Note that in both states of y_i , the energy of the resulting Ising model is shifted relative to $E'_i(y_i)$ by the same constant amount, namely $E'_i(0)$:

y_i	General energy (5.3)	Ising energy (5.1)
0	$E'_i(0)$	$0 = E'_i(0) - E'_i(0)$
1	$E'_i(1)$	$E_{0i} = E'_i(1) - E'_i(0)$

2. For each edge energy function $E'_{ij}(y_i, y_j)$, add the three disagreement cost terms

$$\begin{aligned}
 E_{ij} &:= \frac{1}{2}(E'_{ij}(0, 1) + E'_{ij}(1, 0) - E'_{ij}(0, 0) - E'_{ij}(1, 1)) \\
 E_{0i} &:= E'_{ij}(1, 0) - E'_{ij}(0, 0) - E_{ij} \\
 E_{0j} &:= E'_{ij}(0, 1) - E'_{ij}(0, 0) - E_{ij},
 \end{aligned} \tag{5.4}$$

as shown in Figure 5.1(b). Note that for all states of y_i and y_j , the energy of the resulting Ising model is shifted relative to $E'_{ij}(y_i)$ by the same constant amount, namely $E'_{ij}(0, 0)$:

y_i	y_j	General energy (5.3)	Ising energy (5.1)
0	0	$E'_{ij}(0, 0)$	$0 = E'_{ij}(0, 0) - E'_{ij}(0, 0)$
0	1	$E'_{ij}(0, 1)$	$E_{0j} + E_{ij} = E'_{ij}(0, 1) - E'_{ij}(0, 0)$
1	0	$E'_{ij}(1, 0)$	$E_{0i} + E_{ij} = E'_{ij}(1, 0) - E'_{ij}(0, 0)$
1	1	$E'_{ij}(1, 1)$	$E_{0i} + E_{0j} = E'_{ij}(1, 1) - E'_{ij}(0, 0)$

Summing the above terms gives us the *bias* of node i , i.e., its disagreement cost with the bias node

$$E_{0i} = E'_i(1) - E'_i(0) + \sum_{j:(i,j) \in \mathcal{E}} E'_{ij}(1, 0) - E'_{ij}(0, 0) - E_{ij}, \tag{5.5}$$

where the sum iterates over all nodes j connected to nodes i . This construction defines an Ising model whose energy in every configuration $\mathbf{y}$ is shifted, relative to that of the general model we started with, by the same constant amount, namely $E'(\mathbf{0})$:

$$\begin{aligned}
 E([\mathbf{y}, y_0]) &= E'(\mathbf{y}) - \sum_{i \in \mathcal{V}} E'_i(0) - \sum_{(i,j) \in \mathcal{E}} E'_{ij}(0, 0) \\
 &= E'(\mathbf{y}) - E'(\mathbf{0}),
 \end{aligned} \tag{5.6}$$

where $[\mathbf{y}, y_0]$ denotes y_0 appended to the state $\mathbf{y}$. The energy functions of the two models are therefore equivalent. ■

Note how in the above construction, the label symmetry $E(\mathbf{y}) = E(\neg \mathbf{y})$ of the Ising energy function (5.1) is conveniently broken by the introduction of a bias node, through the convention that $y_0 := 0$.

5.1.1 Energy Minimization via Cuts

In Section 4.3.5 we have seen that the minimum s-t cut of a directed graph can be used to compute the MAP state for a binary graphical model with a submodular energy function. We will now show that the MAP state of an Ising graphical model induces the minimum cut of its model graph:

Definition 34 *The cut C of a binary undirected graphical model $G(\mathcal{V}, \mathcal{E})$ induced by state $\mathbf{y} \in \{0, 1\}^n$ is the set $C(\mathbf{y}) = \{(i, j) \in \mathcal{E} : y_i \neq y_j\}$; its weight $w(C(\mathbf{y}))$ is the sum of the weights of its edges.*

Any given state $\mathbf{y}$ partitions the nodes of a binary graphical model into two sets: those labeled ‘0’, and those labeled ‘1’. The corresponding *graph cut* is the set of edges crossing the partition. Since only these edges contribute disagreement costs to the energy of an Ising graphical model (5.1), we have $w(C(\mathbf{y})) = E(\mathbf{y}) \forall \mathbf{y}$. Therefore the lowest-energy state of an Ising graphical model induces its minimum-weight cut. Conversely, the ground state of an Ising graphical model can be determined from its minimum-weight cut via a graph traversal, such as our depth-first-search shown in Algorithm 1.⁴ Algorithm 1 labels nodes as it encounters them, and checks for consistency on subsequent encounters. The traversal starts by labeling the bias node with its known state $y_0 = 0$.

We note that there is a bijection between states and their induced cuts. In particular, each state $\mathbf{y}$ induces a cut $C(\mathbf{y})$ and from each cut C we can derive (up to label symmetry) the corresponding node state $\mathbf{y}$. Thus we have that for all $\mathbf{y}$ the following holds: $\mathbb{P}(C(\mathbf{y})) = \mathbb{P}(\mathbf{y})$. Finally, the label symmetry can be broken by conditioning on any node k :

$$\mathbb{P}(C(\mathbf{y})) = \mathbb{P}(\mathbf{y}|y_k). \quad (5.7)$$

Algorithm 1 Find State from Corresponding Cut

Input: embedded model graph $G(\mathcal{V}, \mathcal{E})$, cut $C(\mathbf{y}) \subseteq \mathcal{E}$

1. $\forall i \in \{0, 1, 2, \dots, n\} : y_i = \text{unknown}$
2. $\text{dfs_state}(0, 0)$

Output: state vector $\mathbf{y}$

```

procedure dfs_state( $i \in \{0, 1, 2, \dots, n\}, s \in \{0, 1\}$ )
  if  $y_i := \text{unknown}$  then
    1.  $y_i := s$ 
    2.  $\forall (i, j) \in \mathcal{E}_i$ : (edges incident to node  $i$ )
      if  $(i, j) \in C$  then
        dfs_state( $j, \neg s$ )
      else dfs_state( $j, s$ )
  else if  $y_i \neq s$  then throw error

```

⁴The recursion used in Algorithm 1 is for illustrative purposes only, since in practice such an implementation would rapidly run out of stack space. Our implementation of Algorithm 1 uses an iterative breadth-first-search.

Minimum-weight cuts can be computed in polynomial time in graphs whose edge weights are all non-negative. We can construct an energy function equivalent to (5.1):

- Introduce one more bias node, with the constraint $y_{n+1} := 1$.
- Replace each negatively weighted bias edge $E_{0i} < 0$ by an edge to the new node $n + 1$ with the positive weight $E_{i,n+1} := -E_{0i} > 0$ (Figure 5.1(c)).

This still leaves us with the requirement that all non-bias edges be non-negative. This *submodularity* constraint (Definition 31) implies that agreement between nodes must be locally preferable to disagreement — a severe limitation.

Our construction (Figure 5.1(c)) differs from that of Kolmogorov and Zabih [70] (Figure 5.1(d)) in that we only use undirected edges. It may thus be possible to use edge direction for other purposes, such as representing a directed graphical model.

5.1.2 Planarity Constraint

Unlike graph cut methods, the *exact* inference algorithms we will describe do not depend on submodularity. Instead they require that the model graph is planar (see Definition 6 and Figure 5.2(b)), and that a planar embedding is provided (Definition 10).

In certain domains (*e.g.*, when working with geographic information) a plane drawing of the graph may be available, from which the corresponding embedding is readily determined. Where it is not, we employ the algorithm of Boyer and Myrvold [11]. Given a connected graph G , this linear-time algorithm produces either a planar embedding for G or a proof that G is non-planar. Source code for this step is freely available [11, 139].

In Section 5.1 we have mapped the energy of a general binary graphical model (5.3) to the energy of a Ising graphical model (5.1) via the addition of a bias node. Now that we require that the Ising graphical model is planar, what does that imply for the original general model? If all nodes of the graph are to be connected to the bias node without violating planarity, the graph has to be *outerplanar*, *i.e.*, have a planar embedding in which all its nodes lie on the external face — a very severe restriction (see Figure 5.2(a)).

The situation improves, however, if we do not insist that all nodes are connected to the bias. If only a subset $\mathcal{B} \subset \mathcal{V}$ of nodes have non-zero bias (5.5), then the graph only needs to be $\mathcal{B}$ -outerplanar, *i.e.*, have a planar embedding in which all nodes in $\mathcal{B}$ lie on the same (external) face. Model selection may thus entail the step of picking the face of a suitably embedded planar Ising graphical model whose nodes will be connected to the bias node. In image processing, for instance, where it is common to operate on a square grid of pixels, we can permit bias for all nodes on the perimeter of the grid, which borders the external face. We will refer to this as *perimeter bias*.

For each node i , we can assign a particular weight that measures how “important” it is to include that node’s bias (5.5). This weight can be the magnitude or square of E_{0i} . In general, a planar embedding which maximizes a weighted sum over the nodes bordering a given face can be found in linear time [42]; by setting node weights to some measure of their bias, such as the magnitude or square of E_{0i} (5.5), we can thus efficiently obtain the planar Ising model closest (in that measure) to any given planar binary graphical model.

In contrast to submodularity, $\mathcal{B}$ -outerplanarity is a structural constraint. This has the advantage that once a model obeying the constraint is selected, inference (*e.g.*, parameter estimation) can proceed via unconstrained methods (*e.g.*, optimization).

It must be noted that all our exact algorithms can be extended to work exactly for non-planar graphs as well. In that case they take time exponential in the genus of the embedding though still polynomial in the size of the graph (Equation 3.10). Finally, in Chapter 7 we develop efficient approximate inference algorithms for non-planar graphs, such as the one shown in Figure 5.2(c).

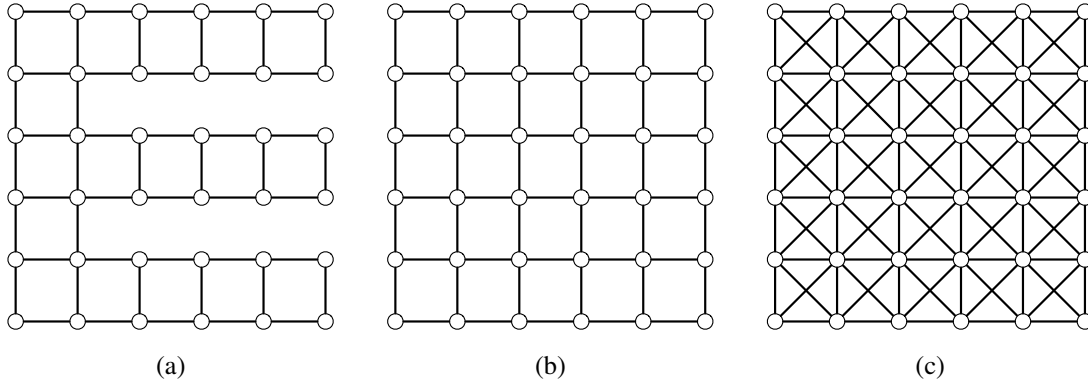


Figure 5.2: (a) outerplanar graph. (b) planar graph. (c) non-planar graph.

5.1.3 Connectivity

All algorithms in this thesis require that the model graph G is *connected* (Definition 4). Where this is not the case, we can simply determine the connected components (Definition 3) of G in linear time [47], then invoke the algorithm in question separately on each of them. Since each component is unconditionally independent of all others (as they have no edges between them), the results can be trivially combined:

- G is planar iff all of its connected components are planar. Any concatenation of a planar embedding for each connected component is a planar embedding of G .
- Any concatenation of a ground state for each connected component of G is a ground state of G .
- The edge marginal probabilities of G are the concatenation of edge marginal probabilities of its connected components.
- The log partition function of G is the sum of the log partition functions of its connected components.

5.1.4 Biconnectivity

Although the algorithms in this thesis do not require the model graph G to be biconnected (Definition 5), simpler and more efficient alternatives are applicable when this is the case.

As Figure 5.3 illustrates, any planar graph G can be decomposed into a tree of edge-disjoint biconnected components which overlap in the articulation vertices they share (Definitions 3, 5). This

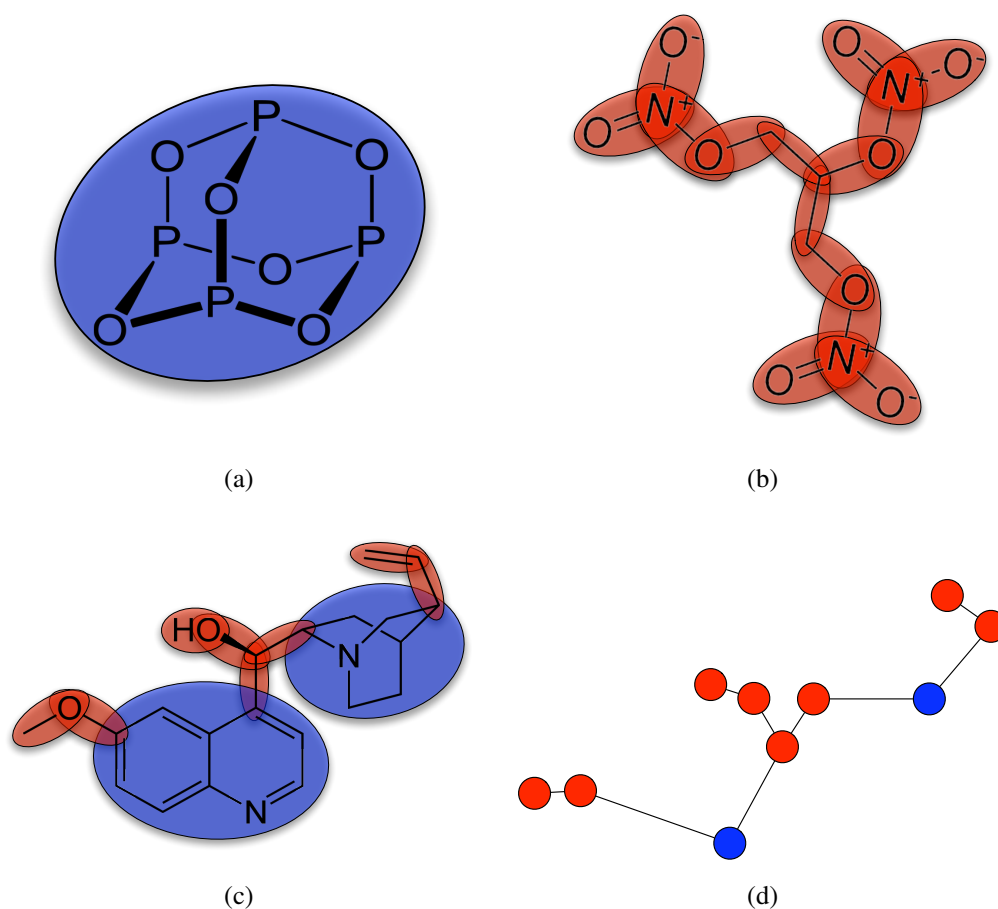


Figure 5.3: Skeletal chemical structure of (a) phosphorus trioxide (b) nitroglycerine and (c) quinine (courtesy of [Wikipedia](#)). Biconnected components are shown with shaded ovals. Trivial biconnected components are red, while nontrivial biconnected components are blue. Phosphorous trioxide is biconnected (*i.e.*, all one component). Nitroglycerine is a tree, *i.e.*, has only trivial biconnected components. Quinine is a general graph, *i.e.*, has both trivial and nontrivial biconnected components. (d) shows the decomposition tree of quinine.

decomposition can be performed in linear time [47]. We call this tree of biconnected components the *decomposition tree* (Figure 5.3(d)).⁵ We show the following key result:

Theorem 35 *Let $\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_n$ be the edge sets corresponding to n biconnected components of a graph $G(\mathcal{V}, \mathcal{E})$. The probability of a cut (Definition 34) induced by the states of a Markov Random Field (5.2) on G factors into that of its biconnected components:*

$$\mathbb{P}(C(\mathbf{y})) = \mathbb{P}(\mathbf{y}) = \prod_{k=1}^n \mathbb{P}(C(\mathbf{y}) \cap \mathcal{E}_k). \quad (5.8)$$

Proof If $G(\mathcal{V}, \mathcal{E})$ is biconnected, $n = 1$ and $\mathcal{E}_1 = \mathcal{E}$ then Theorem 35 is trivially true. Otherwise split G into a biconnected component $G_1(\mathcal{V}_1, \mathcal{E}_1)$ which is a leaf of the decomposition tree, and the remainder $G'(\mathcal{V}', \mathcal{E}')$. It is always possible to find such a split, by splitting at the single articulation vertex i connecting G_1 to G' . To summarize:

$$\begin{aligned} \mathcal{V}_1 \cup \mathcal{V}' &= \mathcal{V}, \quad \mathcal{V}_1 \cap \mathcal{V}' = \{i\}, \\ \mathcal{E}_1 \cup \mathcal{E}' &= \mathcal{E}, \quad \mathcal{E}_1 \cap \mathcal{E}' = \emptyset, \quad G_1 \text{ is biconnected.} \end{aligned} \quad (5.9)$$

Let $\mathbf{y}_1$ and $\mathbf{y}'$ be the states in $\mathbf{y}$ restricted to vertices in $\mathcal{V}_1$ and $\mathcal{V}'$, respectively. By definition, $\mathbf{y}_1$ is independent of $\mathbf{y}'$ when both are conditioned on the state y_i of the articulation vertex i connecting them. By using the result in Equation 5.7 we obtain

$$\begin{aligned} \mathbb{P}(C(\mathbf{y})) &= \mathbb{P}(\mathbf{y}|y_i) = \mathbb{P}(\mathbf{y}_1, \mathbf{y}'|y_i) \\ &= \mathbb{P}(\mathbf{y}_1|y_i) \mathbb{P}(\mathbf{y}'|y_i) = \mathbb{P}(C(\mathbf{y}) \cap \mathcal{E}_1) \mathbb{P}(C(\mathbf{y}) \cap \mathcal{E}'). \end{aligned} \quad (5.10)$$

Recursively applying this argument to G' yields Theorem 35. ■

Even though the node states of two biconnected components do correlate through their common articulation vertex, Theorem 35 shows that their edge states are independent when conditioned on that articulation vertex. By decomposing graphs into their biconnected components, we can perform efficient inference in graphs that are not biconnected themselves.

Ising Trees

Any undirected graph G can be decomposed into biconnected components. These can be either *trivial* or *nontrivial*:

Definition 36 *A biconnected component is trivial if it does not contain any cycles, otherwise it is non-trivial. A trivial biconnected component consists of a single edge and the two vertices it connects.*

A *tree* T does not contain any cycles, hence consists entirely of trivial biconnected components (Figure 5.3(b)). Theorem 35 then implies that each edge can be considered individually, making inference in an Ising tree $T(\mathcal{V}, \mathcal{E})$ simple:

⁵This should not be confused with the junction tree (Definition 30).

- T is planar; any embedding of T is a planar embedding.
- The minimum-weight cut of T is the set $\mathcal{E}^- = \{(i, j) \in \mathcal{E} : E_{ij} < 0\}$. We can use Algorithm 1 to obtain the corresponding ground state.
- The marginal probability of any edge (i, j) of T is

$$\mathbb{P}((i, j) \in C) = \frac{\mathbb{P}(y_i \neq y_j)}{\mathbb{P}(y_i = y_j) + \mathbb{P}(y_i \neq y_j)} = \frac{e^{-E_{ij}}}{e^{-0} + e^{-E_{ij}}} = \frac{1}{1 + e^{-E_{ij}}}. \quad (5.11)$$

- The log partition function of T is

$$\ln Z = \ln \prod_{(i,j) \in \mathcal{E}} (e^{-0} + e^{-E_{ij}}) = \sum_{(i,j) \in \mathcal{E}} \ln(1 + e^{-E_{ij}}). \quad (5.12)$$

General Case

The most efficient way to employ the inference algorithms (Sections 5.2 and 5.3) on graphs G that are neither biconnected nor trees (e.g., Figure 5.3(c)) is to apply them to each nontrivial biconnected component of G in turn. We can then use Theorem 35 to combine the results, along with the simple solutions given in Section 5.1.4 above for trivial biconnected components, into a result for the full graph. Letting $\mathcal{E}^T \subseteq \mathcal{E}$ denote the set of edges that belong to trivial biconnected components of G , we have:

- A planar embedding of G is obtained by arbitrarily combining the planar embeddings for each of its nontrivial biconnected components and the edges in $\mathcal{E}^T$. The edges in $\mathcal{E}^T$ can be embedded arbitrarily.
- A minimum-weight cut of G is the union between the edges in $\mathcal{E}^- \cap \mathcal{E}^T$ and a minimum-weight cut for each of G 's nontrivial biconnected components. We can use Algorithm 1 to obtain the corresponding ground state.
- The marginal probability of an edge $(i, j) \in \mathcal{E}$ is $(1 + e^{-E_{ij}})^{-1}$ if $(i, j) \in \mathcal{E}^T$, or computed (by the method of Section 5.3) for the biconnected component it belongs to.
- The log partition function of G is the sum of the log partition functions of its nontrivial biconnected components, plus $\sum_{(i,j) \in \mathcal{E}^T} \ln(1 + e^{-E_{ij}})$ for its trivial biconnected components.

For the rest of this Chapter, we assume that G is connected and planar, and that a plane embedding is provided. We do not require that G is biconnected, though when this is not the case, it is generally more efficient to decompose G into biconnected components as discussed above.

5.2 Computing Ground States

Frustration (Definition 25) is the key problem in the computation of ground states. A weighted graph is said to be *frustrated* if it contains at least one frustrated cycle, otherwise it is *unfrustrated*. Note that trees can never be frustrated because by their definition they do not contain any cycles.

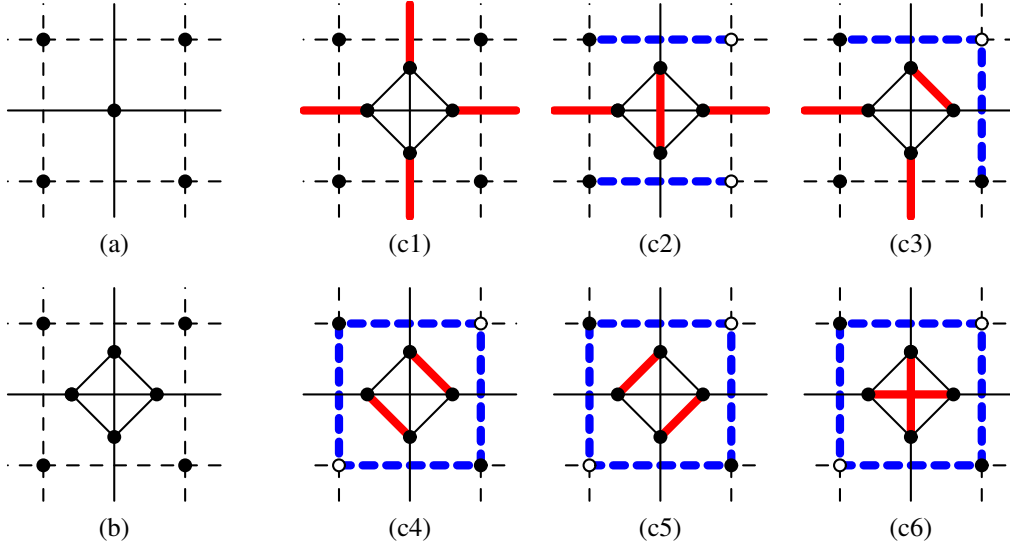


Figure 5.4: Square face of a plane graph (dashed lines) with its ordinary (a) *resp.* expanded (b) dual graph (solid lines). (c1)-(c6): Binary node states (open and filled circles) of the graph, induced cuts (bold blue dashes), and complementary perfect matchings (bold red lines) of the expanded dual.

The lowest-energy (ground) state $\mathbf{y}^* := \operatorname{argmin}_{\mathbf{y}} E(\mathbf{y})$ of an *unfrustrated* Ising graphical model is found via essentially the same method as in a tree (Section 5.1.4): paint nodes as you traverse the graph, flipping the binary color of your paintbrush whenever you traverse an edge with a negative disagreement cost (as done by Algorithm 1 when invoked on the cut $C = \{(i, j) \in \mathcal{E} : E_{ij} < 0\}$). This cannot lead to a contradiction because by Definition 25 you will flip the brush color an even number of times along any cycle in the graph, hence always end a cycle on the same color you started it with.

The presence of frustration unfortunately makes the computation of ground states much harder — in fact, it is known to be NP-hard in general [2]. In this section we show that the ground state of a *planar* Ising graphical model can be computed exactly in polynomial time via maximum-weight perfect matching on the expanded dual of its embedded model graph.

A relationship between the states of a planar Ising model and perfect matchings (or *dimer coverings*) was first established by Kasteleyn [59, 60, 61] and Fisher [30, 31] (Section 3.2). Perfect matchings in dual graph constructs were used by Bieche et al. [9] (Section 3.3.1) and Barahona [2] (Section 3.3.2) to compute ground states of the Ising model. We generalize a simpler construction for triangulated graphs due to Globerson and Jaakkola [38] (Section 4.3.7). For rectangular lattices our approach reduces to the construction of Thomas and Middleton [126] (Section 3.3.3), though their algorithm to compute ground states is somewhat less straightforward. Pardella and Liers [98] apply this construction to very large square lattices (up to 3000×3000), and find it to be more efficient than earlier methods [2, 9].

5.2.1 Expanded Dual Graph

The dual graph for a square face of a plane model graph is shown with solid edges in Figure 5.4(a), and in Figure 5.5(b) for an entire graph. Each edge of the dual graph G^* crosses exactly one edge of the original graph. Due to this one-to-one relationship we will consider the dual to have the *same* set

of edges $\mathcal{E}$ (with the same energies) as the original graph G . Since the nodes in the dual are different, however, we will (with some abuse of notation) use a single index for dual edges and their weights, corresponding to the index of the original edge in some arbitrary ordering of $\mathcal{E}$. Thus if (i, j) is the k^{th} edge in the (ordered) $\mathcal{E}$, its dual will have weight $E_k := E_{ij}$. We now construct the *expanded dual* graph:

Definition 37 G_E^* is the expanded dual graph of the model graph G . It is constructed from the dual graph G^* (Definition 12) by replacing each of its nodes with a q -clique, where q is the degree of the node.

The expanded dual graph for a dual node with degree $q = 4$ is shown in Figure 5.4(b), and in Figure 5.5(c) for an entire graph. The additional edges internal to each q -clique are given zero energy so as to leave the model's energy unaffected. For large q the introduction of these $q(q-1)/2$ internal edges slows down subsequent computations (solid line in Figure 5.8(a)). This can be avoided by subdividing the offending q -gonal face of the model graph with chords (which are also given zero energy) before constructing the dual. Our implementation performs best when “octangulating” the graph, *i.e.*, splitting octagons off all faces with $q > 13$. In our experiments, we found this to be more efficient than a full triangulation (Figure 5.8(a)).

It can be seen that the expanded dual has $2|\mathcal{E}|$ vertices, two for each edge in the original graph. We therefore give the two vertices connected by the dual of the k^{th} edge in $\mathcal{E}$ the indices $2k-1$ and $2k$ (see Section 5.3.3 and Figure 5.7). This consistent indexing scheme allows us to run the inference algorithms described in the remainder of this thesis without explicitly constructing the expanded dual graph data structure.

5.2.2 Complementary Perfect Matchings

Figures 5.5(d) and 5.5(e) show two perfect matchings (in bold) of the nodes of the expanded dual of an Ising graphical model. We prove that there is a complementary relationship between such perfect matchings and graph cuts in the original Ising graphical model, characterized by the following two theorems. The reader may find it helpful to refer to Figure 5.5 while going through the proofs.

Theorem 38 For every cut C of an embedded graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ there exists at least one perfect matching $\mathcal{M}$ of its expanded dual complementary to C , *i.e.*, $\mathcal{E} \setminus \mathcal{M} = C$. If G is triangulated (Definition 40) then there is exactly one such $\mathcal{M}$.

Proof By construction, the set of edges $\mathcal{E}$ constitutes a perfect matching of the expanded dual. Any subset of $\mathcal{E}$ therefore is a (possibly partial) matching of the expanded dual. The complement $\mathcal{M}' := \mathcal{E} \setminus C$ of a cut of G is a subset of $\mathcal{E}$ and thus a matching of the expanded dual; it obeys $\mathcal{E} \setminus \mathcal{M}' = \mathcal{E} \setminus (\mathcal{E} \setminus C) = C$. The nodes that $\mathcal{M}'$ leaves *unmatched* in the expanded dual (bold red in Figures 5.5(d) and 5.5(e)) are those neighboring the edges of the cut C .

By definition, C intersects any cycle of G , and therefore also the perimeters of G 's faces $\mathcal{F}$, in an even number of edges. In each clique of the expanded dual, $\mathcal{M}'$ thus leaves an even number of nodes unmatched. It can therefore be completed into a perfect matching $\mathcal{M} \supseteq \mathcal{M}'$ using only edges interior to the cliques to pair up unmatched nodes. These edges have no counterpart in the original graph:

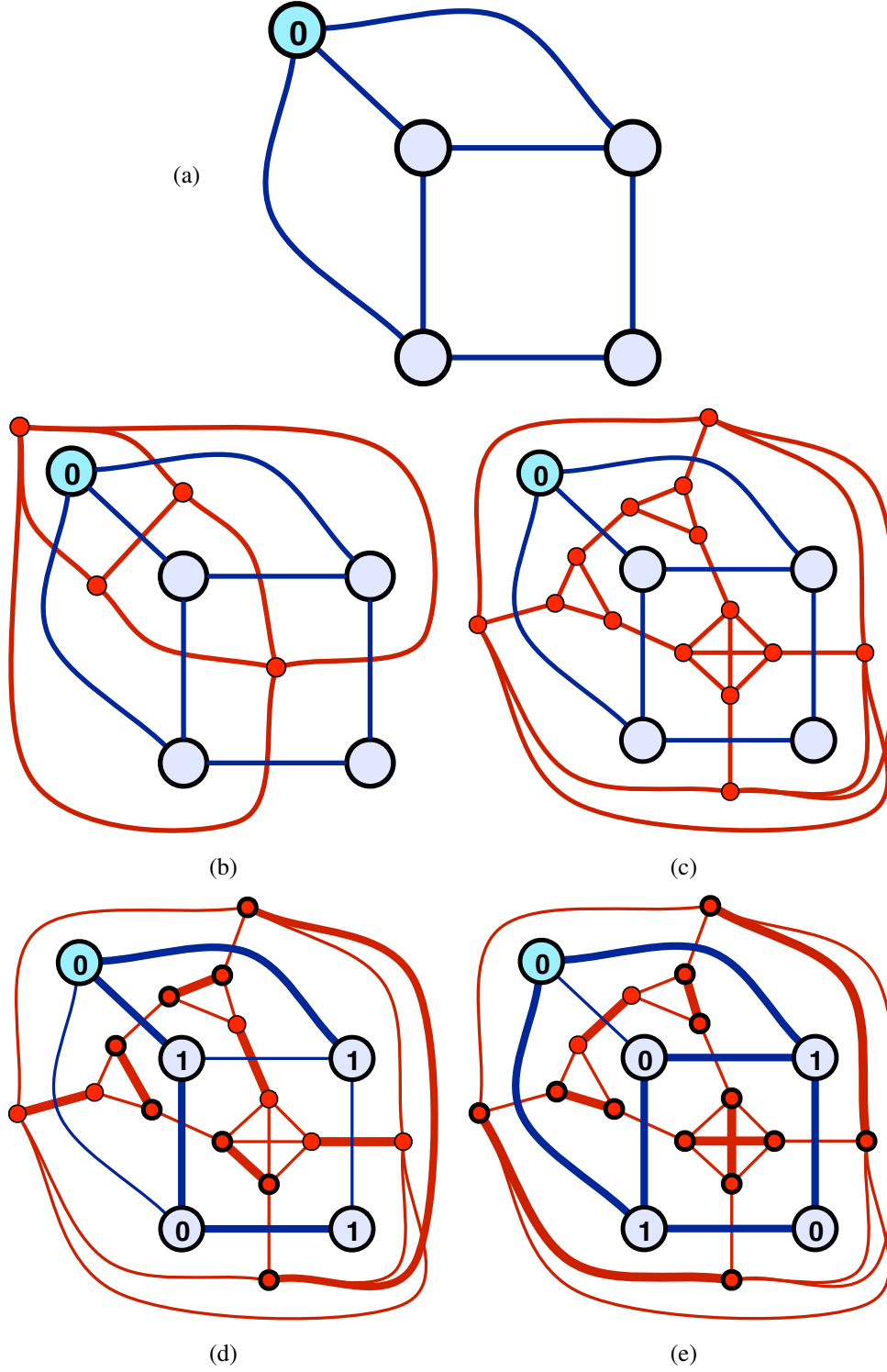


Figure 5.5: (a) Planar Ising graphical model with graph G . G has five binary nodes, including a bias node (light blue) fixed to label '0'. (b) The dual graph G^* (small red nodes). (c) The expanded dual graph G_E^* . (d) and (e) show two different states of G . The graph cut induced by the given state and its complementary perfect matching of the expanded dual are shown as bold blue and bold red edges, respectively. The nodes left unmatched by the complement $M' := E \setminus C$ of the cut in the expanded dual are in bold red.

$(\mathcal{M} \setminus \mathcal{M}') \cap \mathcal{E} = \emptyset$. We thus have

$$\mathcal{E} \setminus \mathcal{M} = \mathcal{E} \setminus [(\mathcal{M} \setminus \mathcal{M}') \cup \mathcal{M}'] = [\mathcal{E} \setminus (\mathcal{M} \setminus \mathcal{M}')] \setminus \mathcal{M}' = \mathcal{E} \setminus \mathcal{M}' = \mathcal{C}. \quad (5.13)$$

In a K_3 clique of the expanded dual, $\mathcal{M}'$ will leave two nodes unmatched or none at all. In either case there is only one way to complete the matching, by adding one edge *resp.* none at all. By construction, if G is triangulated all cliques in its expanded dual are K_3 cliques, and so $\mathcal{M}$ is unique. ■

Theorem 38 shows that there exists a surjection from perfect matchings in G_E^* to cuts in G . Furthermore, since we have given edges interior to the cliques of the expanded dual zero energy, every perfect matching $\mathcal{M}$ complementary to a cut $\mathcal{C}$ of our Ising graphical model (5.1) obeys the relation

$$w(\mathcal{M}) + w(\mathcal{C}) = w(\mathcal{M}') + w(\mathcal{C}) = \sum_{(i,j) \in \mathcal{E}} E_{ij} = \text{constant}, \quad (5.14)$$

where $w(\mathcal{M})$ and $w(\mathcal{C})$ are the weights of the perfect matching $\mathcal{M}$ and the cut $\mathcal{C}$, respectively. This complementary relationship means that instead of a minimum-weight cut in a graph we can look for a maximum-weight perfect matching in its expanded dual. But will that matching always be complementary to a cut? We now prove the following theorem, which shows that this is true for *plane* graphs:

Theorem 39 *Every perfect matching $\mathcal{M}$ of the expanded dual of a plane graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ is complementary to a cut $\mathcal{C}$ of G , i.e., $\mathcal{E} \setminus \mathcal{M} = \mathcal{C}$.*

Proof By definition, $\mathcal{E} \setminus \mathcal{M}$ is a cut of G iff it intersects every cycle $\mathcal{O} \subseteq \mathcal{E}$ of G an even number of times. This can be shown by induction over the faces of the embedding:

Base case — let $\mathcal{O} \subseteq \mathcal{E}$ be the perimeter of a face of the embedding, and consider the corresponding clique of the expanded dual: $\mathcal{M}$ matches an even number of nodes in the clique via interior edges. All other nodes must be matched by edges crossing $\mathcal{O}$. The complement of the matching in G thus intersects $\mathcal{O}$ an even number of times:

$$|(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}| \equiv 0 \pmod{2}. \quad (5.15)$$

Induction — let $\mathcal{O}_i, \mathcal{O}_j \subseteq \mathcal{E}$ be cycles in G obeying (5.15). We define the *symmetric difference* of two cycles $\mathcal{O}_i$ and $\mathcal{O}_j$ as $\mathcal{O}_i \Delta \mathcal{O}_j := (\mathcal{O}_i \cup \mathcal{O}_j) \setminus (\mathcal{O}_i \cap \mathcal{O}_j)$. We now have

$$\begin{aligned} |(\mathcal{E} \setminus \mathcal{M}) \cap (\mathcal{O}_i \Delta \mathcal{O}_j)| &= |[(\mathcal{E} \setminus \mathcal{M}) \cap (\mathcal{O}_i \cup \mathcal{O}_j)] \setminus (\mathcal{O}_i \cap \mathcal{O}_j)| \\ &= |[(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}_i] \cup [(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}_j]| - |(\mathcal{E} \setminus \mathcal{M}) \cap (\mathcal{O}_i \cap \mathcal{O}_j)| \\ &= |(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}_i| + |(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}_j| - 2|(\mathcal{E} \setminus \mathcal{M}) \cap \mathcal{O}_i \cap \mathcal{O}_j| \\ &\equiv 0 + 0 - 2n \equiv 0 \pmod{2}, \quad \forall n \in \mathbb{N}. \end{aligned} \quad (5.16)$$

We see that property (5.15) is preserved under composition of cycles via symmetric differences, and thus holds for all cycles that can be composed from face perimeters of the embedding of G .

All cycles in a *plane* graph G are *contractible* on its embedding surface (a plane or a sphere) because that surface has zero genus, and is therefore simply connected. Intuitively this means that all such

cycles can be “contracted” to a single point, while remaining on the surface of their embedding. All contractible cycles of G can be constructed by composition of face perimeters via symmetric differences, thus intersect $\mathcal{E} \setminus \mathcal{M}$ an even number of times. Therefore $\mathcal{E} \setminus \mathcal{M}$ is a cut. ■

This is where planarity matters: surfaces of non-zero genus are not simply connected, and thus non-plane graphs may contain non-contractible cycles (*e.g.*, cycles enclosing the hole of a torus, as shown in Figure 7.2). In the presence of frustration, our construction does not guarantee that the complement $\mathcal{E} \setminus \mathcal{M}$ of a perfect matching of the expanded dual contains an even number of edges along such cycles. For planar graphs, however, the above theorems allow us to leverage known polynomial-time algorithms for perfect matchings into inference methods for Ising graphical models. This approach also works for non-planar Ising graphical models that do not contain any frustrated non-contractible cycles.

We note that if all cliques of the expanded dual are of even size, there is also a direct (non-complementary) surjection from perfect matchings to cuts in the original graph (see Section 3.2.2). In contrast to our complementary map in Theorem 38, the direct surjection requires the addition of dummy vertices into the expanded dual for faces of G with odd perimeter, so as to make the corresponding cliques even [60, 84].

5.2.3 Computing the Ground State

The blossom-shrinking algorithm [23, 24] is a sophisticated method to efficiently compute the maximum-weight perfect matching of a graph. It can be implemented to run in as little as $O(|\mathcal{E}| |\mathcal{V}| \log |\mathcal{V}|)$ time [89]. A detailed description of blossom-shrinking is provided in Section 7.1. In our experiments we used both Blossom IV [17] and the more recent Blossom V code [67]. We can now efficiently compute the lowest-energy state of a planar Ising graphical model as follows:

1. Find a planar embedding of the model graph G (Section 2.2.2).
2. Construct the expanded dual of G (Section 5.2.1).
3. Run the blossom-shrinking algorithm on G_E^* to compute its maximum-weight perfect matching. Its complement in the original model graph is the minimum-weight graph cut C (Section 5.2.2).
4. Identify the state which induces this cut C via a graph traversal (Algorithm 1).

5.3 Computing the Partition Function and Marginal Probabilities

As it involves a summation over exponentially many states $\mathbf{y}$, calculating the partition function (5.2) is generally intractable. For planar graphs, however, the generating function for perfect matchings can be calculated in polynomial time via the determinant of a skew-symmetric matrix [30, 31, 59, 60, 61], which we call the *Kasteleyn matrix* $\mathbf{K}$ (Section 2.3.3). Due to the close relationship with graph cuts (Section 5.2.2) we can apply this method to calculate Z in (5.2). We first convert a planar embedding of the Ising model graph into a Boolean “proto-Kasteleyn” matrix $\mathbf{H}$:

1. *Plane triangulate* (Section 5.3.1) the embedded graph so as to make the relationship between cuts and complementary perfect matchings a bijection (Section 5.2.2).

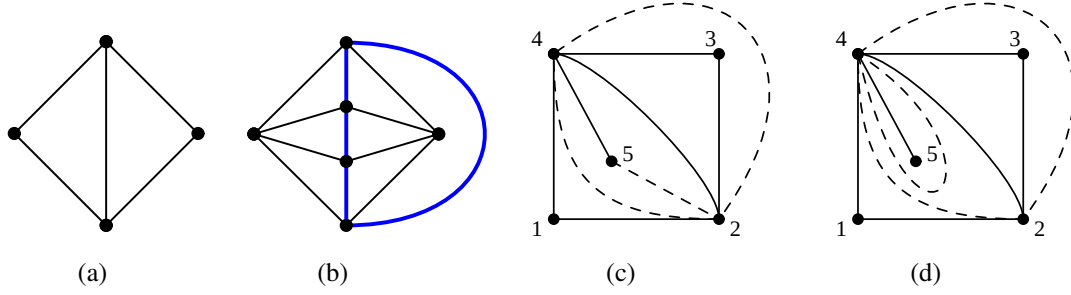


Figure 5.6: (a) A chordal graph whose external face is not triangulated. (b) a plane triangulated graph that has a cycle of length 4 (bold blue) without a chord. (c) proper and (d) improper plane triangulations (dashed) of the plane graph from Figure 2.2(d).

2. *Orient* the edges of the graph such that the in-degree of every node is odd (Section 5.3.2).
3. *Construct* the Boolean proto-Kasteleyn matrix $\mathbf{H}$ from the oriented graph (Section 5.3.3).
4. *Prefactor* the triangulation edges (added in Step 1) out of $\mathbf{H}$ (Section 5.3.4).

Our Step 2 simplifies equivalent operations in previous constructions [31, 38, 60, 61]. Step 3 differs in that it only sets unit (*i.e.*, +1) entries in a Boolean matrix. Step 4 can dramatically reduce the size of $\mathbf{H}$ for compact storage (as a bit matrix) and faster subsequent computations (Figure 5.8).

Edge disagreement costs do not enter into $\mathbf{H}$. They are only taken into account in a second phase, when the full Kasteleyn matrix $\mathbf{K}$ is constructed from $\mathbf{H}$ (Section 5.3.3). We can then factor $\mathbf{K}$ (Section 5.3.4) and compute the partition function from its determinant (Section 5.3.4 and [30, 59]). This factorisation can also be used to invert $\mathbf{K}$, which is necessary for obtaining the *marginal probabilities* of disagreement on the edges of the model graph (Section 5.3.4).

In what follows, we elaborate in turn on the graph operations of plane triangulation (Section 5.3.1), odd edge orientation (Section 5.3.2), construction (Section 5.3.3) and factoring (Section 5.3.4) of the Kasteleyn matrix $\mathbf{K}$ *resp.* $\mathbf{H}$.

5.3.1 Plane Triangulation

We begin by *plane triangulating* the model graph:

Definition 40 *An embedded graph is plane triangulated iff it is biconnected (Definition 5) and each of its faces (including the external face) is a triangle.*

Note that plane triangulation is *not* equivalent to making a graph *chordal* (Definition 29), though the latter process is sometimes also called “triangulation”. For instance, the graph in Figure 5.6(a) is chordal but not plane triangulated because the external face is not triangular, while that in Figure 5.6(b) is plane triangulated but not chordal because it contains a cycle of length 4 (blue) that has no chord.

We can plane triangulate an embedded graph in linear time by traversing all of its faces and inserting chords as necessary as we go along (Algorithm 2). This may create multiple edges between the same two vertices, such as edge (2, 4) in Figure 5.6(c). Care must be taken when encountering “dangling”

nodes, *i.e.*, nodes in trivial biconnected components that are leaves of the decomposition tree. Such nodes could cause the insertion of a self-loop (edge (4, 5) in Figure 5.6(d)). Our Algorithm 2 detects and biconnects such components, as Definition 40 requires. From now on, we will refer to the edges added by Algorithm 2 as the *triangulation edges*.

The `insert_chord( $i, j, k$ )` subroutine of Algorithm 2 updates $\mathcal{E}$, π_i and π_k so as to insert the new edge (i, k) in its proper place in the rotation system. In order to leave the distribution (5.2) modeled by the graph unchanged, the new edge is given zero energy. Repeated traversals of the same face in Algorithm 2 can be avoided by the use of “done” flags, which have been omitted here for the sake of clarity.

Note that plane triangulation is not strictly necessary for the computation of partition function or marginal probabilities. Plane triangulation ensures that the graph becomes biconnected, so an edge cannot border the same face of the model graph on both sides. Thus the expanded dual is a *proper graph*, *i.e.*, has no multiple edges and no self loops.

In previous work [31, 38], plane triangulation came at a computational price: the edges added during plane triangulation can make factoring and inversion of $\mathbf{K}$ (Section 5.3.4) significantly (up to 20 times) more expensive. We avoid this cost by removing the triangulation edges before constructing $\mathbf{K}$ (Section 5.3.4). This is legal, because we are not interested in the marginal probability of triangulation edges.

Algorithm 2 Plane Triangulation

Input: plane graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ with $|\mathcal{V}| \geq 3$
 $\forall i \in \mathcal{V}$:
 $\forall (i, j) \in \mathcal{E}_i$: (edges incident to node i)
 1. $(j, k) := \pi_j(j, i)$
 2. $(k, l) := \pi_k(k, j)$
 3. **while** $l \neq i$ **or** $\pi_l(l, k) \neq (l, j)$:
 (a) **if** $i = k$ **then** (avoid self-loop)
 $i := j, j := k, k := l$
 $(k, l) := \pi_k(k, j)$
 (b) `insert_chord( $i, j, k$ )`
 (c) $i := k, j := l$
 (d) $(j, k) := \pi_j(j, i)$
 (e) $(k, l) := \pi_k(k, j)$

Output: plane triangulated graph $G(\mathcal{V}, \mathcal{E}, \Pi)$

procedure `insert_chord( $i, j, k \in \mathcal{V}$ )` (insert (k, i) between (k, j) and $\pi_k(k, j)$)
 1. $\mathcal{E} := \mathcal{E} \cup \{(i, k)\}$
 Update π_k
 2. $\pi_k(k, i) := \pi_k(k, j)$
 3. $\pi_k(k, j) := (k, i)$
 Update π_i
 4. $\pi_i(\pi_i^{-1}(i, j)) := (i, k)$ ($\pi_i^{-1}(i, j)$ is the counter-clockwise edge from (i, j))
 5. $\pi_i(i, k) := (i, j)$
 6. $E_{ik} := 0$

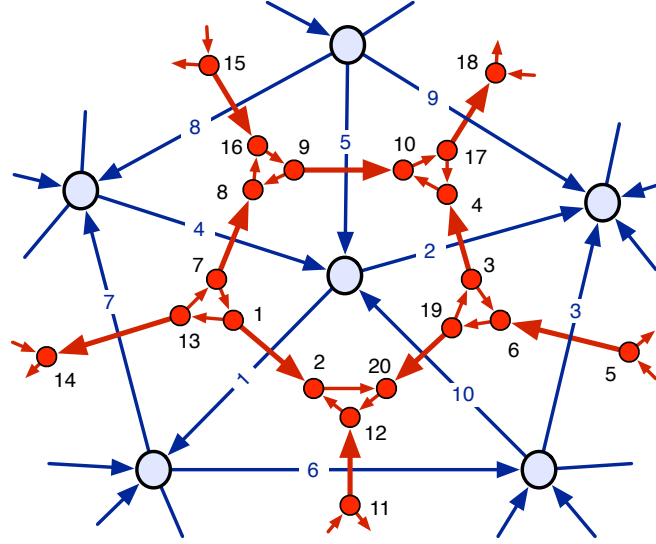


Figure 5.7: Clockwise odd orientation (Section 5.3.2) and indexing scheme (Section 5.3.3) for the expanded dual graph G_E^* (red, small nodes) of the model graph G (large nodes).

5.3.2 Odd Edge Orientation

To calculate the generating function for perfect matchings, the expanded dual graph G_E^* must be given a *clockwise odd orientation* (Section 2.3.4, Lemma 18). Here we will describe an equivalent orientation for the model graph G .

We will now explain the orientation process by referring to Figure 5.7. By giving all interior edges of the K_3 cliques of the expanded dual graph G_E^* a clockwise orientation (small red arrows), we ensure that

- The interior faces of the K_3 cliques have a clockwise odd orientation.
- All interior edges of the K_3 cliques are oriented *counter*-clockwise wrt. all faces exterior to the K_3 cliques, hence do not affect the latter's clockwise odd orientation status.

It remains to consider the orientation of edges external to the K_3 cliques (large red arrows in Figure 5.7). What does a clockwise odd orientation of these edges correspond to in the original model graph G ? To map these edges back into the model graph, *rotate* them clockwise by 90° degrees. A face with a clockwise odd orientation of its perimeter in G_E^* thus maps to a vertex with an odd *in-degree*, i.e., a vertex with an odd number of incoming edges. This facilitates a drastic simplification of this step in our construction:

Lemma 41 *To establish a clockwise odd orientation of the expanded dual graph G_E^* , orient the edges of the model graph G such that all vertices, except possibly one, have an odd in-degree.*

Our Algorithm 3 achieves this time linear in $|\mathcal{E}|$ by orienting edges appropriately upon return from a depth-first traversal of the graph. In contrast to earlier constructions [31, 38, 60], it does not require following orbits around faces, and in fact does not refer to an embedding Π or dual graph G^* at all.

Algorithm 3 Construct Odd Edge Orientation

Input: undirected graph $G(\mathcal{V}, \mathcal{E})$

1. $\forall v \in \mathcal{V} : v.\text{visited} := \text{false}$
2. pick arbitrary edge $(r, s) \in \mathcal{E}$
3. $\mathcal{E}' := \{(r, s)\}$
4. $\text{make_odd}(r, s)$

Output: orientation $\mathcal{E}'$ of $G(\mathcal{V}, \mathcal{E})$:
 $\text{in-degree}(v) \equiv 1 \pmod{2}, \forall v \in \mathcal{V} \setminus \{s\}$

function $\text{make_odd}: (u, v \in \mathcal{V}) \rightarrow \{\text{true}, \text{false}\}$

1. $\mathcal{E} := \mathcal{E} \setminus \{(u, v)\}$
2. **if** $v.\text{visited} = \text{true}$ **then return** true
3. $v.\text{visited} := \text{true}$
4. $\text{odd} := \text{false}$
5. $\forall \{v, w\} \in \mathcal{E}$:
 - if** $\text{make_odd}(v, w) = \text{true}$ **then**
 - (a) $\mathcal{E}' := \mathcal{E}' \cup \{(w, v)\}$
 - (b) $\text{odd} := \neg \text{odd}$
 - else** $\mathcal{E}' := \mathcal{E}' \cup \{(v, w)\}$
6. **return** odd

In Algorithm 3 any vertex can be chosen to be the exceptional vertex s (Lemma 41), since the choice of the external face of a plane drawing is arbitrary. The external face is an artifact of the drawing, not an intrinsic property of the embedding. A planar graph embedded on a sphere has no external face (see Figure 2.7).

5.3.3 Constructing the Kasteleyn Matrix

The *Kasteleyn matrix* $\mathbf{K}$ is a skew-symmetric, $2|\mathcal{E}| \times 2|\mathcal{E}|$ matrix constructed from the model graph of the Ising graphical model. Its determinant is the square of the partition function. Our construction improves upon the work of Globerson and Jaakkola [38] in a number of ways:

- We employ an indexing scheme (Section 5.3.3) that removes any need to refer to the expanded dual graph G_E^* , which we never explicitly construct.
- We break the construction of the Kasteleyn matrix into two phases, the first of which is invariant with respect to the model's disagreement costs. This has several advantages as discussed in Section 5.3.3.
- We make the “proto-Kasteleyn” matrix $\mathbf{H}$ computed in the first phase very compact by *prefactoring* out the triangulation edges (see Section 5.3.4) and storing it as a bit matrix. Empirically this makes $\mathbf{H}$ several orders of magnitude smaller than $\mathbf{K}$.

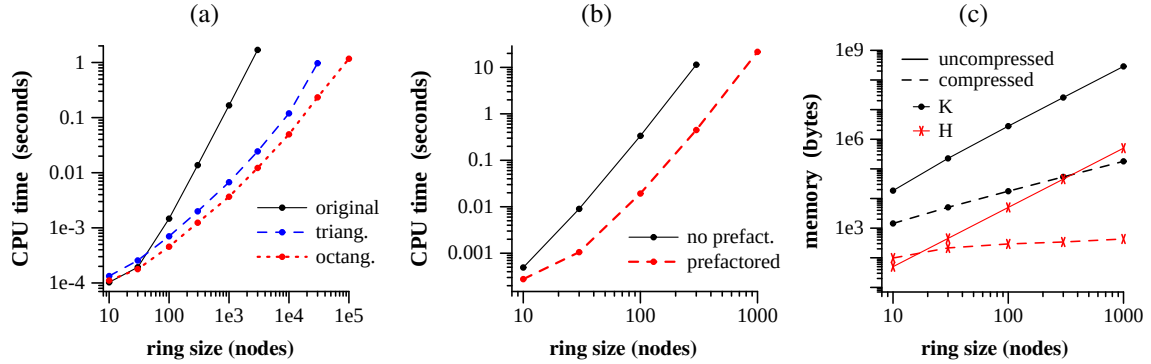


Figure 5.8: Cost of our inference methods on a ring graph (*i.e.*, single cycle), plotted against ring size. (a) and (b): CPU time on Apple MacBook with 2.2 GHz Intel Core2 Duo processor, averaged over 100 repetitions. (a) MAP state calculated via Blossom IV [17] on original, triangulated, and octangulated ring. (b) marginal edge probabilities with vs. without prefactoring. (c) size of K (double precision, no prefactoring) vs. prefactored bit matrix H , in uncompressed (solid lines) vs. compressed form (dashed lines), using compressed row storage for K and bzip2 compression for H .

Indexing Scheme

The expanded dual G_E^* has $2|\mathcal{E}|$ vertices, one lying to either side of every edge of the model graph G . We associate each model edge with an index k , so if $e_k = (i, j) \in \mathcal{E}$ then $\pi_i(e_k) = \pi_i(i, j)$ (Definition 7). When viewing the model edge e_k along its direction in $\mathcal{E}'$, we label the dual node to its right $2k-1$ and that to its left $2k$. For example in Figure 5.7 the node to the left of e_3 is 6, while the one to its right is 5. One benefit of this scheme is that quantities relating to the edges of the model graph (as opposed to internal edges of the cliques of the expanded dual) will always be found on the superdiagonal of K .

Two-Phase Construction

In the first phase we process the *structure* of the model graph into a Boolean “proto-Kasteleyn” matrix H which does not yet include disagreement costs. H only has positive entries, and only those corresponding to edges with zero disagreement cost, *i.e.*, those added during plane triangulation or those internal to the cliques of the expanded dual. All such entries have the weight $e^0 = 1$, making H a Boolean matrix, which can be stored compactly as a bit matrix. Using the indexing scheme from Section 5.3.3, our Algorithm 4 constructs H in linear time, cycling once through the edges incident upon each vertex of the model graph.

In the second phase, we use H to construct the conventional, real-valued Kasteleyn matrix K by adding the exponentiated disagreement costs E_k , $\forall k = \{1, 2, \dots, |\mathcal{E}|\}$ along the superdiagonal and skew-symmetrizing:

1. $K := H$
2. $K_{2k-1, 2k} := K_{2k-1, 2k} + e^{E_k} \quad \forall k \in \{1, 2, \dots, |\mathcal{E}|\}$
3. $K := K - K^\top$

This two-phase construction of K has a number of advantages over classical constructions [30, 38, 59]:

- When working with a large number of isomorphic graphs (as we do in Chapter 6), the corresponding proto-Kasteleyn matrix is identical for all of them, hence needs to be constructed just once.
- During maximum likelihood parameter estimation, the partition function and/or marginals have to be recomputed many times for the same graph, with disagreement costs varying due to the ongoing adaptation of the model parameters. $\mathbf{H}$ remains valid when disagreement costs change, so we can compute it just once upfront and then reuse it in the parameter estimation loop.
- $\mathbf{H}$ can be stored very compactly as a prefactored bit matrix. As Figure 5.8(c) shows, the uncompressed $\mathbf{H}$ can be several orders of magnitude smaller than the corresponding Kasteleyn matrix $\mathbf{K}$. Since G_E^* is cubic (*i.e.*, every node has degree 3), the Kasteleyn matrix $\mathbf{K}$ is sparse with each row and column having exactly 3 non-zero entries. Compressed row storage [1] of $\mathbf{K}$ is efficient, but applying the bzip2 compressor (from <http://www.bzip.org/>) to the prefactored bit matrix $\mathbf{H}$ yields by far the most compact storage format we have found. Such memory efficiency becomes very important when working with large data sets of non-isomorphic graphs.

Algorithm 4 Construct proto-Kasteleyn Bit Matrix $\mathbf{H}$

Input: oriented, embedded and triangulated graph $G(\mathcal{V}, \mathcal{E}', \Pi)$

1. $\mathbf{H} := \mathbf{0} \in \{0, 1\}^{2|\mathcal{E}'| \times 2|\mathcal{E}'|}$
2. $\forall v \in \mathcal{V}$:
 - (a) $e_s :=$ any edge incident on v
 - (b) **if** e_s points to v **then** $\alpha := 2s$
 else $\alpha := 2s - 1$
 - (c) $e_r := \pi_v(e_s)$
 - (d) **do**
 - if** e_r points to v **then**
 - $H_{2r-1, \alpha} := 1$
 - $\alpha := 2r$
 - if** e_r was created by plane triangulation (Alg. 2) **then**
 - $H_{2r-1, 2r} := 1$
 - else**
 - $H_{2r, \alpha} := 1$
 - $\alpha := 2r - 1$
 - $e_r := \pi_v(e_r)$
 - while** $e_r \neq \pi_v(e_s)$

Output: proto-Kasteleyn bit matrix $\mathbf{H}$

5.3.4 Factoring Kasteleyn Matrices

Standard approaches such as LU-factorization can be used to factor the Kasteleyn matrix $\mathbf{K}$, but they do not exploit its skew symmetry and may encounter numerical difficulties. The preferred decompositions for even-sized skew-symmetric matrices are $LDL^\top$ (*i.e.*, root-free Cholesky-style) factorizations

operating on 2×2 matrix blocks [46, Chapter 11]. Here we develop such a decomposition which can be used to factor $\mathbf{K}$ as well as to prefactor $\mathbf{H}$ (see below). We begin by writing the Kasteleyn matrix as

$$\mathbf{K} = \left[\begin{array}{cc|c} 0 & c & \mathbf{a}^\top \\ -c & 0 & \mathbf{b}^\top \\ \hline -\mathbf{a} & -\mathbf{b} & \mathbf{C} \end{array} \right], \quad (5.17)$$

for some scalar c , vectors $\mathbf{a}$ and $\mathbf{b}$, and a matrix $\mathbf{C}$ which is either empty or again of the same form. We factor (5.17) into [see 5, 14]

$$\mathbf{K} = \left[\begin{array}{cc|c} 0 & -1 & \mathbf{0}^\top \\ 1 & 0 & \mathbf{0}^\top \\ \hline \mathbf{a}/c & \mathbf{b}/c & \mathbf{I} \end{array} \right] \left[\begin{array}{cc|c} 0 & c & \mathbf{0}^\top \\ -c & 0 & \mathbf{0}^\top \\ \hline \mathbf{0} & \mathbf{0} & \mathbf{C}' \end{array} \right] \left[\begin{array}{cc|c} 0 & 1 & \mathbf{a}^\top/c \\ -1 & 0 & \mathbf{b}^\top/c \\ \hline \mathbf{0} & \mathbf{0} & \mathbf{I} \end{array} \right], \quad (5.18)$$

where $\mathbf{0}$ is the zero matrix, $\mathbf{I}$ is the identity matrix and $\mathbf{C}'$ is the Schur complement given as

$$\mathbf{C}' := \mathbf{C} + (\mathbf{b}\mathbf{a}^\top - \mathbf{a}\mathbf{b}^\top)/c. \quad (5.19)$$

Iterated application of (5.18) to the Schur complement ultimately yields $\mathbf{K} = \mathbf{R}^\top \mathbf{J} \mathbf{R}$, where

$$\mathbf{R} := \left[\begin{array}{cc|cc|c} 0 & 1 & & & \mathbf{a}_1^\top/c_1 \\ -1 & 0 & & & \mathbf{b}_1^\top/c_1 \\ \hline 0 & 0 & 0 & 1 & \mathbf{a}_2^\top/c_2 \\ & 0 & -1 & 0 & \mathbf{b}_2^\top/c_2 \\ \hline \vdots & & \ddots & & \vdots \\ & & & 0 & 0 & 0 & 1 \\ 0 & \dots & 0 & -1 & 0 \end{array} \right] \quad \text{and} \quad \mathbf{J} := \left[\begin{array}{cc|cc|cc|c} 0 & c_1 & 0 & \dots & 0 \\ -c_1 & 0 & 0 & 0 & & & \\ \hline 0 & 0 & 0 & c_2 & \ddots & & \vdots \\ & 0 & -c_2 & 0 & & 0 & \\ \hline \vdots & & \ddots & & \ddots & 0 & 0 \\ & & & 0 & 0 & 0 & c_{|\mathcal{E}|} \\ 0 & \dots & 0 & -c_{|\mathcal{E}|} & 0 \end{array} \right]. \quad (5.20)$$

To prevent small *pivots* c_i from causing numerical instability, pivoting is required. In our experience, partial pivoting suffices here since Kasteleyn matrices are sparse and have at least two entries of unit magnitude (*i.e.*, suitable pivots) in each row and column.

Partition Function

The partition function for perfect matchings is $\sqrt{|\mathbf{K}|}$ [30, 59]. Our factoring gives $|\mathbf{R}| = 1$ and $|\mathbf{J}| = \prod_i c_i^2$, so we have

$$\sqrt{|\mathbf{K}|} = \sqrt{|\mathbf{R}^\top| |\mathbf{J}| |\mathbf{R}|} = \sqrt{|\mathbf{J}|} = \prod_{i=1}^{|\mathcal{E}|} |c_i|. \quad (5.21)$$

Calculation of the product in (5.21) is prone to numerical overflow. This can be avoided by working with logarithms. Using the complementary relationship (5.14) with graph cuts in planar Ising graphical

models, we show that the log partition function for the latter is

$$\begin{aligned} \ln Z &:= \ln \sum_{\mathbf{y}} \exp \left(- \sum_{e_k \in C(\mathbf{y})} E_k \right) = \ln \sum_{\mathbf{y}} \exp \left(- \left(\sum_{e_k \in \mathcal{E}} E_k - \sum_{e_k \notin C(\mathbf{y})} E_k \right) \right) \\ &= \ln \left(\exp \left(- \sum_{e_k \in \mathcal{E}} E_k \right) \sum_{\mathbf{y}} \exp \left(\sum_{e_k \notin C(\mathbf{y})} E_k \right) \right) = \ln \sqrt{|\mathbf{K}|} - \sum_{e_k \in \mathcal{E}} E_k = \sum_{i=1}^{|\mathcal{E}|} (\ln |c_i| - E_i). \end{aligned} \quad (5.22)$$

Marginal Probabilities

The *marginal probability* of disagreement along an edge equals the negative gradient of the log partition function (5.22) with respect to the disagreement costs. Computing this involves the inverse of $\mathbf{K}$. If C is some cut of the model graph, we compute the marginal probability as follows:

$$\begin{aligned} \mathbb{P}(e_k \in C) &= \sum_{\mathbf{y}: e_k \in C(\mathbf{y})} \mathbb{P}(\mathbf{y}) = \frac{1}{Z} \sum_{\mathbf{y}: e_k \in C(\mathbf{y})} e^{-E(\mathbf{y})} = -\frac{1}{Z} \frac{\partial Z}{\partial E_k} = -\frac{\partial \ln Z}{\partial E_k} \\ &= 1 - \frac{1}{2|\mathbf{K}|} \frac{\partial |\mathbf{K}|}{\partial E_k} = 1 - \frac{1}{2} \text{tr} \left(\mathbf{K}^{-1} \frac{\partial \mathbf{K}}{\partial E_k} \right) = 1 + \mathbf{K}_{2k-1,2k}^{-1} \mathbf{K}_{2k-1,2k}, \end{aligned} \quad (5.23)$$

where tr denotes the matrix trace, and we have used the fact that $\mathbf{K}^{-1}$ is also skew-symmetric. To invert $\mathbf{K}$, observe from (5.20) that $\mathbf{R}$ and $\mathbf{J}$ are essentially triangular *resp.* diagonal (simply swap rows $2k-1$ and $2k$, $k = 1, 2, \dots, |\mathcal{E}|$), and thus easily inverted. Then use $\mathbf{K}^{-1} = \mathbf{R}^{-1} \mathbf{J}^{-1} \mathbf{R}^{-\top}$ to obtain

$$\begin{aligned} \mathbf{K}_{2k-1,2k}^{-1} &= \sum_{i=1}^{2|\mathcal{E}|} \sum_{j=1}^{2|\mathcal{E}|} \mathbf{R}_{2k-1,i}^{-1} \mathbf{J}_{i,j}^{-1} \mathbf{R}_{j,2k}^{-\top} = \frac{-1}{c_k} + \sum_{i=k+1}^{|\mathcal{E}|} d_{ik}, \\ \text{where } d_{ik} &= \frac{\mathbf{R}_{2k-1,2i}^{-1} \mathbf{R}_{2k,2i-1}^{-1} - \mathbf{R}_{2k-1,2i-1}^{-1} \mathbf{R}_{2k,2i}^{-1}}{c_i}. \end{aligned} \quad (5.24)$$

Note in the above $\mathbf{A}_{i,k}^{-1} = [\mathbf{A}^{-1}]_{i,k}$ so inversion is performed first.

Prefactoring

Consider the rows and columns of $\mathbf{K}$ corresponding to an edge added during plane triangulation (Section 5.3.1). Reorder $\mathbf{K}$ to bring those rows and columns to the top left, so that they form the $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ of (5.17). Since the disagreement cost of a triangulation edge is zero, we now have a unity pivot: $c = e^0 = 1$. This has two advantageous consequences:

Size reduction: The unity pivot does not affect the value of the partition function. Since we are not interested in the marginal probability of triangulation edges (which after all are not part of the original model), we do not need $\mathbf{a}$ or $\mathbf{b}$ either, once we have computed the Schur complement (5.19). We can therefore discard the first two rows and first two columns of $\mathbf{K}$ after factoring (5.17). Factoring out all triangulation edges in this fashion reduces the size of $\mathbf{K}$ (*resp.* $\mathbf{R}$ and $\mathbf{J}$) to range only over the edges of the original model graph. This reduces storage requirements and speeds up subsequent computation of the inverse (Figure 5.8(b)).

Boolean closure: The unity pivot eliminates the division from the Schur complement (5.19). In fact we show below that applying (5.18) to prefactor $\mathbf{H} - \mathbf{H}^\top$ yields a Schur complement that can be expressed as $\mathbf{H}' - \mathbf{H}'^\top$, where $\mathbf{H}'$ is again a Boolean matrix. This closure property allows us to simply prefactor triangulation edges directly out of $\mathbf{H}$ without explicitly constructing $\mathbf{K}$.

Specifically, let $\mathbf{K} = \mathbf{H} - \mathbf{H}^\top$ for a proto-Kasteleyn matrix $\mathbf{H}$ with elements in $\{0, 1\}$. Without loss of generality, assume that $\mathbf{H}$ and its transpose are *disjoint*, i.e., have no non-zero elements in common: $\mathbf{H} \odot \mathbf{H}^\top = \mathbf{0}$, where $\odot$ denotes Hadamard (element-wise) multiplication. Algorithm 4 respects this condition; violations would cancel in the construction of $\mathbf{K}$ anyway. Expressing $\mathbf{H}$ as

$$\mathbf{H} = \left[\begin{array}{cc|c} 0 & 1 & \mathbf{a}_1^\top \\ 0 & 0 & \mathbf{b}_1^\top \\ \hline \mathbf{a}_2 & \mathbf{b}_2 & \mathbf{C}_1 \end{array} \right], \quad (5.25)$$

we can write $\mathbf{K} = \mathbf{H} - \mathbf{H}^\top$ as (5.17) with $\mathbf{a} = \mathbf{a}_1 - \mathbf{a}_2$, $\mathbf{b} = \mathbf{b}_1 - \mathbf{b}_2$, $c = 1$, and $\mathbf{C} = \mathbf{C}_1 - \mathbf{C}_1^\top$. The Schur complement (5.19) then becomes

$$\begin{aligned} \mathbf{C}' &= \mathbf{C}_1 - \mathbf{C}_1^\top + (\mathbf{b}_1 - \mathbf{b}_2)(\mathbf{a}_1 - \mathbf{a}_2)^\top - (\mathbf{a}_1 - \mathbf{a}_2)(\mathbf{b}_1 - \mathbf{b}_2)^\top \\ &= (\mathbf{C}_1 + \mathbf{b}_1\mathbf{a}_1^\top + \mathbf{b}_2\mathbf{a}_2^\top + \mathbf{a}_1\mathbf{b}_2^\top + \mathbf{a}_2\mathbf{b}_1^\top) - (\mathbf{C}_1^\top + \mathbf{a}_1\mathbf{b}_1^\top + \mathbf{a}_2\mathbf{b}_2^\top + \mathbf{b}_2\mathbf{a}_1^\top + \mathbf{b}_1\mathbf{a}_2^\top) \\ &= \mathbf{H}' - \mathbf{H}'^\top, \end{aligned} \quad (5.26)$$

where

$$\mathbf{H}' = \mathbf{C}_1 + \mathbf{b}_1\mathbf{a}_1^\top + \mathbf{b}_2\mathbf{a}_2^\top + \mathbf{a}_1\mathbf{b}_2^\top + \mathbf{a}_2\mathbf{b}_1^\top. \quad (5.27)$$

It remains to show that $\mathbf{H}'$ is a Boolean matrix, ie all its elements are in $\{0, 1\}$. By definition of $\mathbf{H}$ (5.25), all elements of $\mathbf{C}_1, \mathbf{a}_1, \mathbf{a}_2, \mathbf{b}_1, \mathbf{b}_2$ are in $\{0, 1\}$, and by closure of multiplication in $\{0, 1\}$, so are their products. Thus an element of $\mathbf{H}'$ will be in $\{0, 1\}$ iff it is non-zero in at most one term on the right-hand side of (5.27), or equivalently iff all pairs formed from the five terms in question are disjoint. We show that this is the case:

- Because $\mathbf{H} \odot \mathbf{H}^\top = \mathbf{0}$, we know that neither $\mathbf{a}_1$ and $\mathbf{a}_2$ nor $\mathbf{b}_1$ and $\mathbf{b}_2$ can have any non-zero elements in common, so $\mathbf{b}_1\mathbf{a}_1^\top$ and $\mathbf{b}_2\mathbf{a}_2^\top$ are disjoint, as are $\mathbf{a}_1\mathbf{b}_2^\top$ and $\mathbf{a}_2\mathbf{b}_1^\top$.
- By construction of $\mathbf{H}$ (Algorithm 4), $\mathbf{a}_1$ and $\mathbf{b}_1$ (resp. $\mathbf{a}_2$ and $\mathbf{b}_2$) can only have an element in common if the dual nodes on both sides of the corresponding edge are members of the same clique. This cannot happen because we explicitly ensure that the graph becomes biconnected during plane triangulation (Section 5.3.1), so that an edge cannot border the same face of the model graph on both sides. Thus all four outer products in (5.27) are pairwise disjoint.
- Finally, each outer product in (5.27) is disjoint from $\mathbf{C}_1$ as long as the edges being factored out do not form a cut of the model graph (i.e., cycle of the dual). We are prefactoring only edges added during triangulation. These edges will form a cut only if the model graph was disconnected prior to triangulation. However this cannot happen, because we deal with (and eliminate) this possibility during earlier preprocessing (Section 5.1.3).

In summary, all five terms on the right-hand side of (5.27) are pairwise disjoint. Thus the Schur complement $\mathbf{H}'$ is a Boolean matrix as well, and can be computed from $\mathbf{H}$ (5.25) very efficiently by replacing the additions and multiplications in (5.27) with bitwise OR and AND operations, respectively. As long as further triangulation edges remain in $\mathbf{H}'$, we then set $\mathbf{H} := \mathbf{H}'$ and iteratively apply (5.25) and (5.27) so as to prefactor them out as well.

5.4 Conclusion

This chapter described an alternative algorithmic framework for efficient exact inference in binary graphical models, which replaces the submodularity constraint of graph cut methods with a planarity constraint. In contrast to previous approaches, our construction uses the complementary mapping (Section 3.2.2) between cuts in the model graph G (even subgraphs in G^*) and perfect matchings in its expanded dual graph G_E^* . The existence of two distinct tractable frameworks for inference in binary graphical models implies a more powerful hybrid. Consider a graph each of whose biconnected components is either planar or submodular. In its entirety, this graph may be neither planar nor submodular, yet efficient exact inference in it is clearly possible by applying the appropriate framework to each component, then combining the results (Section 5.1.4). Can this hybrid approach be extended to cover less obvious situations?

Although we have only dealt with binary-labeled models, the Ising graphical model can also implement α -expansion moves and α - β -swaps [12], allowing it to deal with multi-labeled models.

Our exact algorithms can all be extended to non-planar graphs, at a cost exponential in the genus of the embedding. These extensions may prove of great practical value for graphs that are “almost” planar. Examples of such graphs include road networks (where edge crossings arise from overpasses without on-ramps) and graphs describing the tertiary structure of proteins [135].

Planar Ising Graphical Models:

Experiments

Unlike graph cut methods, the algorithms described in Chapter 5 allow us to perform penalized maximum-likelihood (Section 6.1.1) as well as maximum-margin (Section 6.1.2) parameter estimation in our Ising graphical model. For prediction we can employ marginal posterior probabilities as well as MAP states.

We demonstrate the suitability of our approach to CRF parameter estimation on 3 tasks whose underlying model graph is planar. We begin with two computer vision problems: the synthetic binary image denoising task of Kumar and Hebert [76, 77] (Section 6.2), and boundary detection in noisy masks from the GrabCut Ground Truth image segmentation database [104] (Section 6.3). Finally, we apply Ising CRFs to estimate territory in the game of *go* by employing graph abstraction and parametrisation techniques (Section 6.4).

6.1 Parameter Estimation

We chose to represent the Ising graphical model as a planar Conditional Random Field (CRF). We compute the disagreement costs in Equation 5.1 as $E_k := \theta^T \mathbf{x}_k$, *i.e.*, as inner products between local features (sufficient statistics) $\mathbf{x}_k$ of the modeled data at each edge e_k , and corresponding parameters θ of the model. In a computer vision task, for example, $\mathbf{x}_k$ might be the absolute intensity difference between two adjacent pixels. We use a Markov Random Field (5.2) to model the *conditional probability distribution* $\mathbb{P}(\mathbf{y}|\mathbf{x}, \theta)$, where $\mathbf{x}$ denotes the union of all local features.

6.1.1 Maximum Likelihood

Penalized maximum-likelihood (ML) CRF parameter estimation seeks to minimize *wrt.* θ the L_2 -regularized negative log-likelihood of a given target labeling $\tilde{\mathbf{y}}$.⁶ Using (5.2) this negative log-likelihood

⁶For notational clarity we suppress here the fact that we are usually modeling a *collection* of data items. The objective function for such a set is simply the sum of objectives for each individual item in it.

for the Ising graphical model becomes

$$\begin{aligned} L_{\text{ML}}(\boldsymbol{\theta}) &:= \frac{1}{2} \lambda \|\boldsymbol{\theta}\|^2 - \ln \mathbb{P}(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) \\ &= \frac{1}{2} \lambda \|\boldsymbol{\theta}\|^2 + E(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) + \ln Z(\boldsymbol{\theta}|\mathbf{x}), \end{aligned} \quad (6.1)$$

where λ is the regularization parameter, $E(\mathbf{y})$ is the energy function of the Ising graphical model (5.1) and $\mathbb{P}(\mathbf{y})$ is its joint probability distribution (5.2). This is a smooth, convex, non-negative objective that can be optimized via gradient methods such as LBFGS, either in conventional batch mode [86, 95] or online [113]. The gradient of (6.1) with respect to the parameters $\boldsymbol{\theta}$ is given by

$$\frac{\partial}{\partial \boldsymbol{\theta}} L_{\text{ML}}(\boldsymbol{\theta}) = \lambda \boldsymbol{\theta} + \sum_{e_k \in \mathcal{E}} (\mathbb{I}[e_k \in C(\tilde{\mathbf{y}})] - \mathbb{P}(e_k \in C(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}))) \mathbf{x}_k, \quad (6.2)$$

where $C(\tilde{\mathbf{y}})$ is the cut induced by the target state $\tilde{\mathbf{y}}$, and $\mathbb{P}(e_k \in C(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}))$ is the marginal probability of e_k being contained in a cut, given $\mathbf{x}$ and $\boldsymbol{\theta}$. We compute the latter via the inverse of the Kasteleyn matrix (5.23).

6.1.2 Maximum Margin

For maximum-margin (MM) parameter estimation [123] we instead minimize

$$\begin{aligned} L_{\text{MM}}(\boldsymbol{\theta}) &:= \frac{1}{2} \lambda \|\boldsymbol{\theta}\|^2 + E(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) - \min_{\mathbf{y}} M(\mathbf{y}|\tilde{\mathbf{y}}, \mathbf{x}, \boldsymbol{\theta}) \\ &= \frac{1}{2} \lambda \|\boldsymbol{\theta}\|^2 + E(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) - E(\hat{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) + d(\hat{\mathbf{y}}|\tilde{\mathbf{y}}), \end{aligned} \quad (6.3)$$

where $\hat{\mathbf{y}} := \operatorname{argmin}_{\mathbf{y}} M(\mathbf{y}|\tilde{\mathbf{y}}, \mathbf{x}, \boldsymbol{\theta})$ is the worst margin violator, *i.e.*, the state that minimizes, relative to a given target state $\tilde{\mathbf{y}}$ the *margin energy*

$$M(\mathbf{y}|\tilde{\mathbf{y}}) := E(\mathbf{y}) - d(\mathbf{y}|\tilde{\mathbf{y}}), \quad (6.4)$$

where $d(\cdot|\cdot)$ is some measure of divergence in state space. We choose d to be a weighted Hamming distance between induced cuts:

$$d(\mathbf{y}|\tilde{\mathbf{y}}) := \sum_{e_k \in \mathcal{E}} \left[\mathbb{I}[e_k \in C(\mathbf{y})] \neq \mathbb{I}[e_k \in C(\tilde{\mathbf{y}})] \right] v_k, \quad (6.5)$$

where $v_k > 0$ are constant weighting factors (in the simplest case: all ones) on the edges of G . It is now easy to verify that the margin energy (6.4) is implemented (up to a shift that depends only on $\tilde{\mathbf{y}}$) by an isomorphic Ising graphical model with disagreement costs

$$\bar{E}_k := E_k + (2 \mathbb{I}[e_k \in C(\tilde{\mathbf{y}})] - 1) v_k. \quad (6.6)$$

Thus we can efficiently find the worst margin violator $\hat{\mathbf{y}}$ by computing the ground state (Section 5.2.3) of this isomorphic Ising graphical model. Thomas and Middleton [126] employ a similar approach to obtain the ground state from a given state $\tilde{\mathbf{y}}$ by setting up an isomorphic Ising model with disagreement

costs $\bar{E}_k := E_k(1 - 2 \llbracket e_k \in C(\tilde{\mathbf{y}}) \rrbracket)$. Using (6.6), the maximum-margin objective (6.3) can be expressed as

$$L_{\text{MM}}(\boldsymbol{\theta}) := \frac{1}{2} \lambda \|\boldsymbol{\theta}\|^2 + \bar{E}(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) - \bar{E}(\hat{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}), \quad (6.7)$$

where $\bar{E}(\hat{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta})$ is the energy of the worst margin violator. This objective is convex but non-smooth; its gradient is

$$\frac{\partial}{\partial \boldsymbol{\theta}} L_{\text{MM}}(\boldsymbol{\theta}) = \lambda \boldsymbol{\theta} + \sum_{e_k \in \mathcal{E}} (\llbracket e_k \in C(\tilde{\mathbf{y}}) \rrbracket - \llbracket e_k \in C(\hat{\mathbf{y}}) \rrbracket) \mathbf{x}_k. \quad (6.8)$$

In other words, local features $\mathbf{x}_k$ are multiplied by one of $\{-1, 0, 1\}$, depending on the membership of edge e_k in the cuts induced by $\tilde{\mathbf{y}}$ and $\hat{\mathbf{y}}$, respectively. We can minimize (6.7) via bundle methods, such as the bundle trust (BT) algorithm [109], making use of the convenient lower bound: $L_{\text{MM}}(\boldsymbol{\theta}) \geq 0$, $\forall \boldsymbol{\theta}$. This lower bound holds because in Equation 6.7

$$\begin{aligned} \bar{E}(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) &= \bar{E}(\tilde{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}) - d(\tilde{\mathbf{y}}|\tilde{\mathbf{y}}) \\ &= M(\tilde{\mathbf{y}}|\tilde{\mathbf{y}}, \mathbf{x}, \boldsymbol{\theta}) \\ &\geq \min_{\mathbf{y}} M(\mathbf{y}|\tilde{\mathbf{y}}, \mathbf{x}, \boldsymbol{\theta}) = \bar{E}(\hat{\mathbf{y}}|\mathbf{x}, \boldsymbol{\theta}). \end{aligned} \quad (6.9)$$

6.2 Synthetic Binary Image Denoising

Kumar and Hebert [76, 77] developed an image denoising benchmark problem for binary-labeled CRFs based on four hand-drawn 64×64 pixel images (Figure 6.1, top row). We created 50 instances of each image corrupted with pink noise, produced by convolving a white noise image (all pixels i.i.d. uniformly random) with a Gaussian density of one pixel standard deviation.⁷ Original and pink noise images were linearly mixed using signal-to-noise (S/N) amplitude ratios of 1:n, where $n \in \mathbb{N}$. Figure 6.1 shows samples of the resulting noisy instances for S/N ratios of 1:5 (row 2) and 1:6 (row 4).

We then employed a grid Ising CRF to denoise the images, with edge disagreement energies set to $E_{ij} := [1, |x_i - x_j|]^\top \boldsymbol{\theta}$, where x_i is the pixel intensity at node i . The perimeter of the grid was connected to a bias node with constant pixel intensity $x_0 := 0$ and fixed label $y_0 := 0$. Edges to the bias had their own parameters, yielding a CRF with four parameters and up to (for a 64×64 grid) 4097 nodes and 8316 edges.

The CRFs were trained by maximum margin (MM) and maximum likelihood (ML) parameter estimation (Section 6.1) on the 50 noisy instances derived from the first image (Figure 6.1, left column) only.⁸ We assessed the quality of the obtained parameters by determining (via the method of Section 5.2.3) the *maximum a posteriori* (MAP) states

$$\mathbf{y}^* := \operatorname{argmax}_{\mathbf{y}} \mathbb{P}(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) = \operatorname{argmin}_{\mathbf{y}} E(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \quad (6.10)$$

of the trained CRF for all 150 noisy instances of the other three images. Interpreting these MAP states

⁷See http://en.wikipedia.org/wiki/Pink_noise and http://en.wikipedia.org/wiki/White_noise for more details.

⁸The regularizer λ was set to 2.

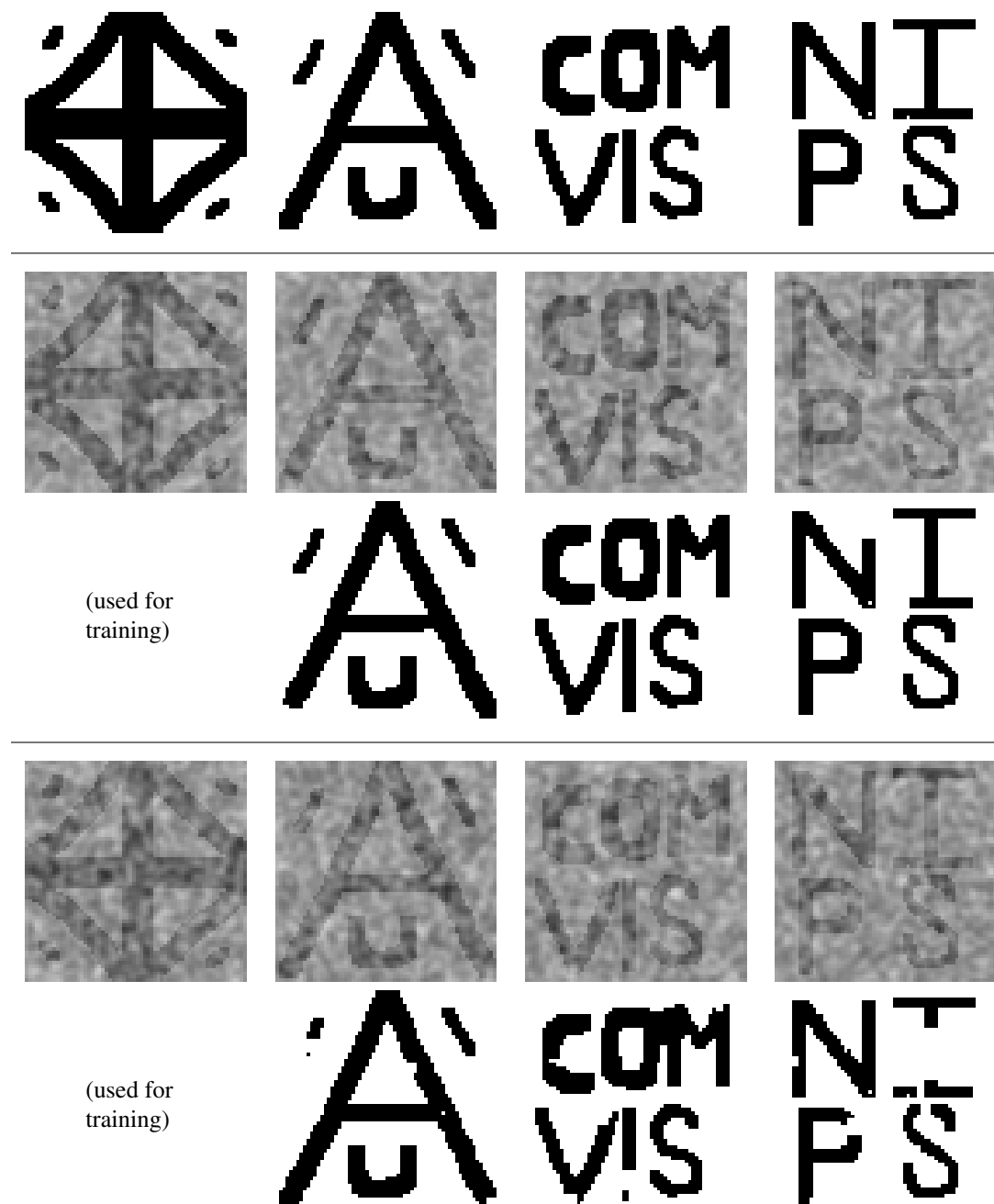


Figure 6.1: Denoising of binary images by maximum-margin training of grid Ising CRFs. Row 1: original images [76, 77]. Rows 2 and 3: examples of images mixed with pink noise in a 1:5 ratio and their reconstruction via MAP state of 64×64 Ising grid CRF. Rows 4 and 5: examples of images mixed with pink noise in a 1:6 ratio and their reconstruction via MAP state of 64×64 Ising grid CRF. Only the left-most image was used for training (via max-margin CRF parameter estimation, $\lambda = 2$); the other 3 images were used for reconstruction.

Train Method	Patch Size	Train Time (s)	Edge Error (%)	Node Error (%)
MM	64×64	490.4	1.15	2.10
	32×32	174.7	1.16	2.15
	16×16	91.4	1.12	1.98
	8×8	78.1	1.09	1.83
ML		5468.2	1.11	1.92

Table 6.1: Performance comparison of parameter estimation methods on the denoising task with image reconstruction via MAP on the full (64×64) model and images. Patch Size is the size of patches used during training. Train Time is the time required to train on 50 images. The minimum in each result column is boldfaced.

as attempted reconstructions of the original images, we then calculated the reconstruction error rates for both nodes and edges of the model.

All experiments in Section 6.2 used Blossom IV [17] and were carried out on a Linux PC with 3 GB RAM and dual Intel Pentium 4 processors running at 3.6 GHz, each with 2 MB of level 2 cache.

6.2.1 Noise Level

We first explore the limit of the ability of a full-size (64×64) MM-trained Ising grid CRF to reconstruct the test images as the noise level increases. Rows 3 and 5 of Figure 6.1 show sample reconstructions obtained from the noisy instances shown in rows 2 and 4, respectively. At low noise levels ($n < 5$) we obtain perfect reconstructions of the original images. At an S/N ratio of 1:5 the first subtle errors do creep in (Figure 6.1, row 3), though less than 0.5% of the nodes and 0.3% of the edges are predicted incorrectly. At the 1:6 S/N ratio, these figures increase to 2.1% for nodes and 1.15% for edges, and the errors become far more noticeable (Figure 6.1, row 5). For higher noise levels ($n > 6$) the reconstructions rapidly deteriorate as the noise finally overwhelms the signal. It must be noted that at these noise levels our human visual system is also unable to accurately reconstruct the images.

6.2.2 Parameter Estimation

Next we compared MM and ML parameter estimation at the S/N ratio of 1:6 (Figure 6.1, row 4), where reconstruction begins to break down and any differences in performance should be evident. To make ML training computationally feasible, we subdivided each training image into $64 \times 8 \times 8$ patches, then trained an 8×8 grid CRF on those patches. For MM training we used the full (64×64) images and model, as well as 32×32 , 16×16 , and 8×8 patches, so as to assess how this subdivision impacts the quality of the model. Testing always employed the MAP state of the full model on the full images (*i.e.*, 64×64).

Table 6.1 reports the edge and node errors obtained under each experimental condition. To assess statistical significance, we performed binomial pairwise comparison tests at a 95% confidence level against the null hypothesis that each of the two algorithms being compared has an equal (50%) chance of outperforming the other on a given test image.

We found no statistically significant difference here between 8×8 CRFs trained by MM *vs.* ML. However, ML training took 70 times as long to achieve this, so MM training is much more preferable on computational grounds.

Counter to our expectations, the node and edge errors suggest that MM training actually works better on small (8×8 and 16×16) image patches. We believe that this is because small patches have a relatively larger perimeter, leading to better training of the bias edges. Pairwise comparison tests, however, only found the node error for the 32×32 patch-trained CRF to be significantly worse than for the smaller patches; all other differences were below the significance threshold. We can confidently state that subdividing the images into small patches did not hurt performance, and yielded much shorter training times.

The reduced performance of the 32×32 model could be an artifact of the training image we are using (first image in Figure 6.1, row 4). The 32×32 patches are generated by cutting the image in half both vertically and horizontally. The resulting four 32×32 patches each have a black region on two edges of their boundaries. Since the model connects the entire perimeter to a bias node, the strong presence of these black regions weakens the learned boundary parameters, which reduces the prediction accuracy.

6.2.3 Reconstruction

Fox and Nicholls [33] argued that the MAP state does not summarize well the information in the posterior probability distribution of an Ising model of noisy binary images, and proposed reconstruction via the *marginal posterior mode* (MPM) instead. For binary labels, the MPM is simply obtained by thresholding the marginal posterior node probabilities: $y_i := \llbracket \mathbb{P}(y_i = 1 \mid \mathbf{x}, \boldsymbol{\theta}) > 0.5 \rrbracket$. However, in our Ising graphical model we have marginal posterior probabilities for edges (Section 5.3.4), and infer node states from graph cuts (Algorithm 1). Here implementing the MPM runs into a difficulty, since the edge set

$$\{e_k \in \mathcal{E} : \mathbb{P}(e_k \in C(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})) > 0.5\} \quad (6.11)$$

may not be a cut of the model graph, hence may not unambiguously induce a node state. What we really need is the cut closest (in some given sense) to (6.11). For this purpose we formulate the state $\mathbf{y}^+$ with the *maximal minimum marginal posterior* (M³P):

$$\mathbf{y}^+ := \operatorname{argmax}_{\mathbf{y}'} \min_{e_k \in \mathcal{E}} \begin{cases} \mathbb{P}(e_k \in C(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})) & \text{if } e_k \in C(\mathbf{y}'), \\ 1 - \mathbb{P}(e_k \in C(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})) & \text{otherwise.} \end{cases} \quad (6.12)$$

In other words, the M³P state (6.12) is induced by the cut whose edges (and those of its complement) have the largest minimum marginal probability. We can efficiently compute $\mathbf{y}^+$ as follows:

1. Find the maximum-weight spanning tree $T(\mathcal{V}, \mathcal{E}^+)$ of the model graph $G(\mathcal{V}, \mathcal{E})$ with edge weights $|\mathbb{P}(e_k \in C(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})) - 0.5|$. This can be done in $O(|\mathcal{E}| \log |\mathcal{E}|)$ time.
2. Run Algorithm 1 on $T(\mathcal{V}, \mathcal{E}^+)$ to find $\mathbf{y}^+$ as the state induced by the edge set (6.11).

Since $T(\mathcal{V}, \mathcal{E}^+)$ is a tree, it contains no cycles, so Algorithm 1 will unambiguously identify the M³P state.

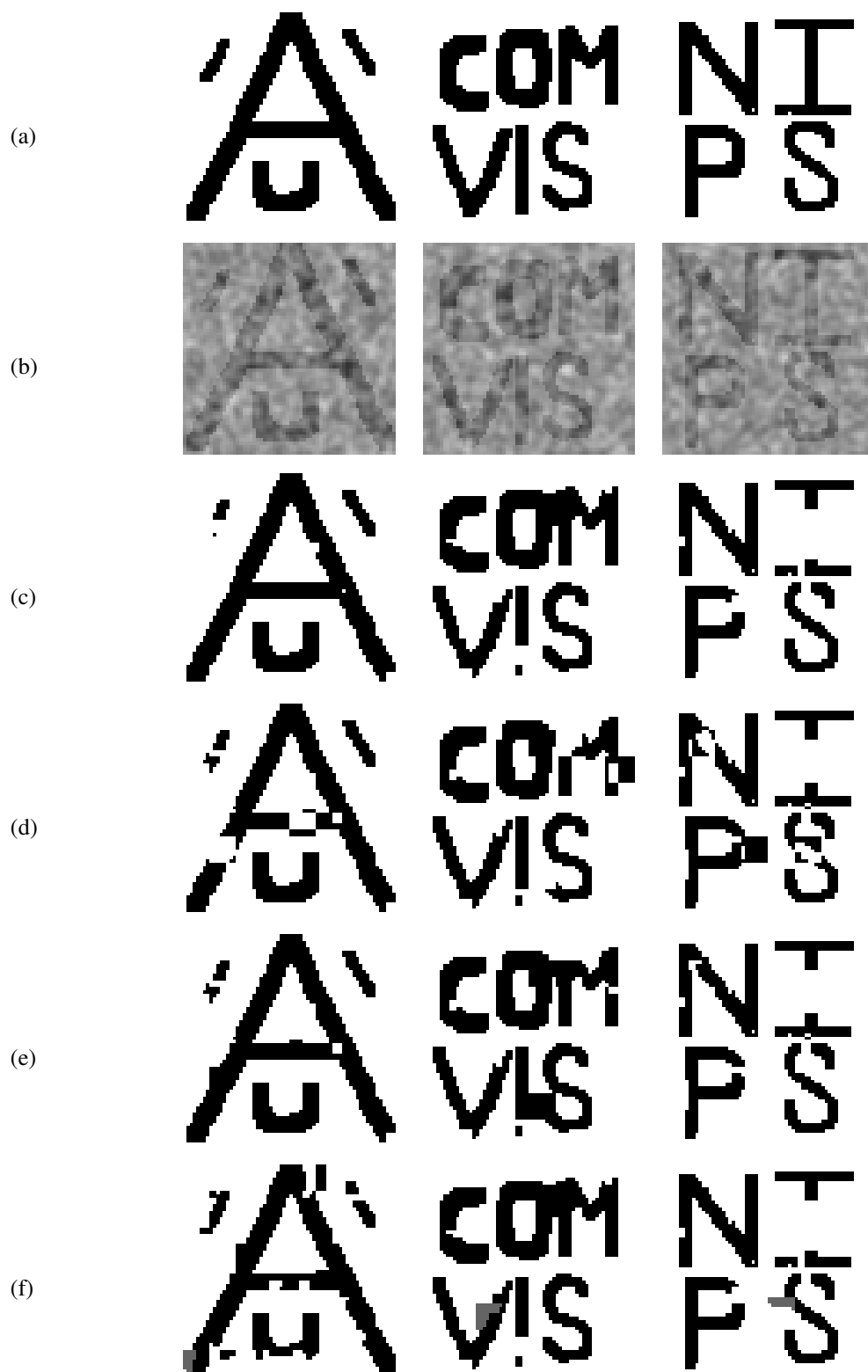


Figure 6.2: Image reconstructions on the denoising task with the Ising CRF trained via MM on 8×8 patches. (a) original images [76, 77]. (b) examples of images mixed with pink noise in a 1:6 ratio. (c) examples of MAP reconstruction of the full 64×64 image. (d) examples MAP reconstruction of 8×8 patches. (e) examples M³P reconstruction of 8×8 patches. (f) examples QPBO reconstruction of 8×8 patches, where gray regions could not be reconstructed due to unlabeled nodes.

Test Method	Patch Size	Test Time (s)	Edge Error (%)	Unl. Edges (%)	Node Error (%)	Unl. Nodes (%)
MAP	8×8	3.3	1.96	—	3.31	—
M ³ P		397.5	1.95	—	3.32	—
QPBO		475.3	2.70	1.99	3.75	1.94
QPBOP		450.4	1.96	< 0.01	3.31	< 0.01

Test Method	Patch Size	Test Time (s)	Edge Error (%)	Unl. Edges (%)	Node Error (%)	Unl. Nodes (%)
MAP	16×16	3.7	1.37	—	2.33	—
QPBO		154.7	3.66	5.18	4.21	5.09
QPBOP		159.5	1.41	0.08	2.35	0.08
MAP	32×32	4.2	1.14	—	1.89	—
QPBO		51.1	21.20	41.52	21.40	41.44
QPBOP		988.4	7.12	12.37	7.77	12.35
MAP	64×64	5.2	1.09	—	1.83	—
QPBO		24.7	49.83	99.66	49.83	99.63
QPBOP		24031.5	43.94	87.66	44.60	87.51

Table 6.2: Comparison of reconstruction methods on the denoising task, using the parameters of an MM-trained 8×8 CRF. Patch Size is the size of patches used during testing. Test Time is the time required to reconstruct all 150 images. Unlabeled nodes and edges count as half errors. Unlabeled percentages are not applicable to MAP and M³P, since these methods provide a full labeling. The minimum in each result column is boldfaced.

Algorithm Comparison

Table 6.2 lists the reconstruction errors achieved on the denoising task using the parameters of the MM-trained 8×8 CRF which gave the best performance in Section 6.2.2. Note that we obtained comparable results for ML training on 8×8 patches and MM training on the full images.

We were able to compare MAP vs. M³P states only when reconstructing 8×8 patches (Table 6.2 top). Since the M³P state (6.12) requires calculation of the edge marginals, it was not feasible to compute it for larger patches. To highlight the difficulty of this task, we also compared to quadratic pseudo-boolean optimization (QPBO) and its variant QPBOP (with probing) [43, 68, 105]. These methods extend the classical graph cut algorithm (Section 4.3.5) by providing a *part* of an optimal node state for graphical models with non-submodular energy functions. Columns 5 and 7 of Table 6.2 show the percentage of unlabeled edges and nodes, respectively. To be fair on QPBO and QPBOP we count unlabeled nodes and edges as half errors.

The binomial pairwise comparison test at a 95% confidence level reveals no statistically significant difference between MAP and M³P states. Thus in our framework we are not able to reproduce the advantage that Fox and Nicholls [33] report for MPM. Furthermore, the MAP state on the entire image clearly outperforms any reconstruction from 8×8 patches in terms of both edge and node error (Table 6.2 and Figures 6.2(c) and 6.2(d)). The impressive scalability of the MAP state computation via blossom-shrinking (Section 5.2.3) thus in practice overrules any possible theoretical advantage that reconstructions via marginal probabilities may have.

The binomial pairwise comparison test at a 95% confidence level also reveals no statistically significant difference between MAP and QPBOP states. QPBOP manages to label nearly all the nodes on this task, thus its reconstruction is close to optimal. However, QPBO cannot label 1.94% of nodes (Figure 6.2(f)) making it significantly worse than QPBOP and MAP.

Scalability

When reconstructing on patches larger than 8×8 , the MAP state significantly outperforms QPBO and QPBOP in both edge and node error (Table 6.2 bottom). As the patch size increases, the reconstruction quality of QPBO and QPBOP rapidly decreases, since the percentage of unlabeled edges and nodes becomes high.

MAP is orders of magnitude faster than all of the control methods. Its advantage is most evident when reconstructing the full images. Interestingly, QPBO becomes relatively faster as the patch size increases. This is because in larger patches QPBO has less chance of resolving unlabeled nodes, therefore it “gives up” more quickly.

6.3 Boundary Detection

We designed a boundary detection task based on images from the GrabCut Ground Truth image segmentation database [104]. We took 100×100 pixel subregions of images that depicted a segmentation boundary, and corrupted the segmentation mask with pink noise, produced by convolving a white noise image (all pixels i.i.d. uniformly random) with a Gaussian density with one pixel standard deviation.

We then employed a planar Ising graphical model to recover the original boundary. We used a 100×100 square grid with one additional edge fixed to a high energy, encoding prior knowledge that the bottom left and top right corners of the image depict different regions. We set the energy of the other edges to $E_{ij} := [1, |x_i - x_j|]^T \theta$, where x_i is the pixel intensity at node i . We train unsupervised on the noisy mask (Figure 6.3(b)), then use the trained CRF’s MAP state for prediction (Figure 6.3(d)).⁹ We did not employ a bias node for this task, and simply set the regularization constant to $\lambda = 1$. All experiments were carried out on an Apple MacBook laptop with 2.2 GHz Intel Core2 Duo processor and 4 GB RAM.

Note that this is a large model with 10 000 nodes and 19 801 edges. Due to the sparsity of the associated Kasteleyn matrix, we can nonetheless compute its log partition function in less than half a CPU second, using the SuperLU sparse matrix factorisation package [19]. However it then takes over 200 CPU seconds to solve for the marginals, making optimization of the ML objective (6.1) computationally unattractive. By contrast, we can compute the ground state of this model with Blossom V [67] via the algorithm described in Section 5.2 in less than 100 CPU milliseconds. We can therefore efficiently minimize the MM objective (6.3).

We estimated the smallest S/N ratio of the form 1:n for which we obtained a good segmentation. Depending on the image, this occurred for $n = 7$ or 8. Figure 6.3 (right column) shows that at these noise levels our approach is capable of recovering the original segmentation boundary quite well, with

⁹For presentation purposes we show the segmented image, obtained by masking the original image with our binary segmentation.

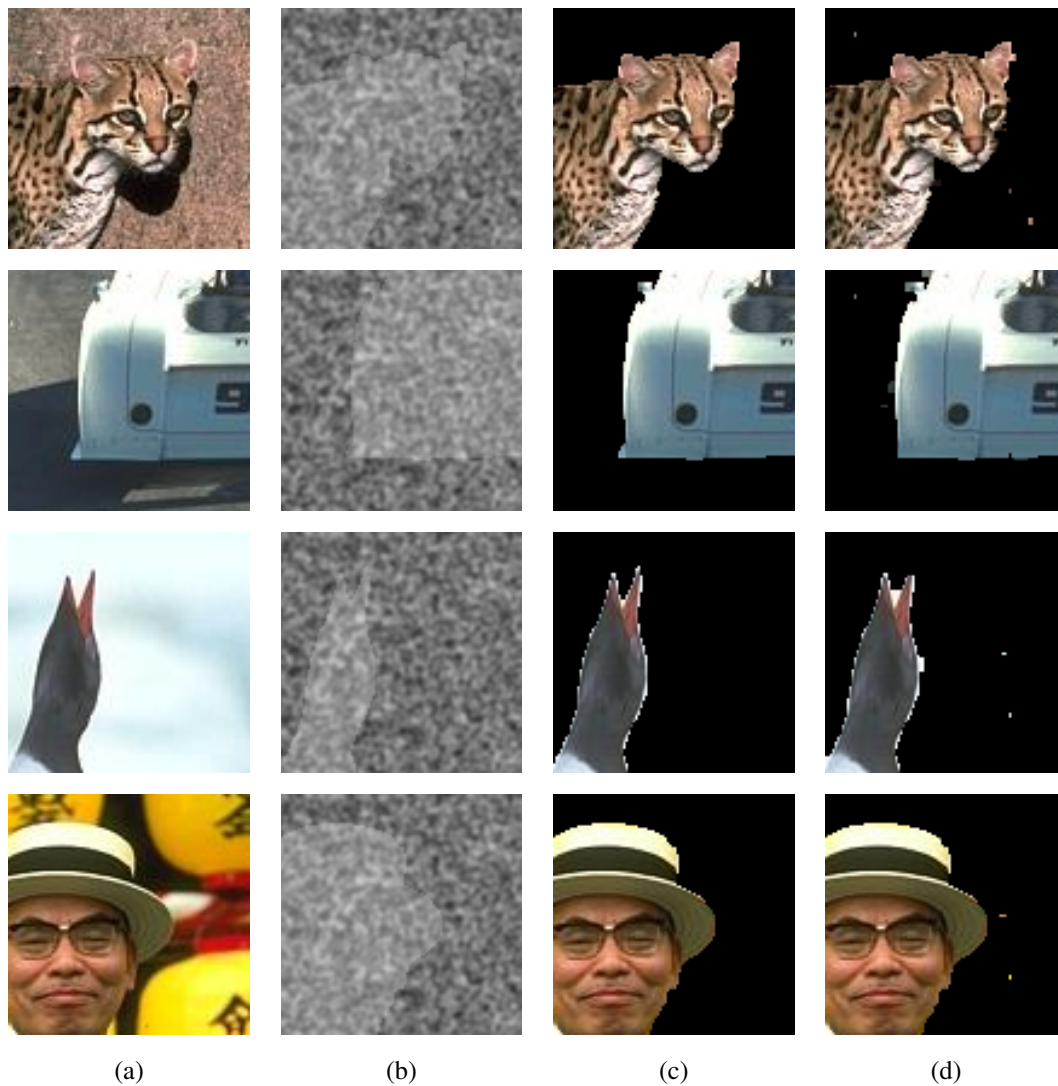


Figure 6.3: Boundary detection by maximum-margin training of grid Ising CRFs. (a) original image. (b) noisy mask for S/N ratio of 1:8 (rows 1 and 2) *resp.* 1:7 (rows 3 and 4). (c) ground truth segmentation. (d) MAP segmentation obtained from the Ising grid CRF trained on the noisy mask.

less than 1% of nodes mislabeled. For S/N ratios of 1:9 and lower the system is unable to locate the boundary; for S/N ratios of 1:6 and higher we obtain perfect reconstruction. Again this corresponds closely to our human ability to visually locate the segmentation boundary accurately.

In further scalability tests we were able to use Blossom V to calculate the MAP state for a random 1500×1500 Ising grid with perimeter bias (a graph with 2.25 million nodes and 4.5 million edges) in 200 CPU seconds. It is worth mentioning that the approach of Pardella and Liers [98] is also able to compute the MAP state of such problem instances. Despite the similarity, there are a few key differences between the two approaches:

- Our method is a generalisation that can handle arbitrary planar graphs.
- Our construction uses the complementary mapping (Section 3.2.2) between even subgraphs in G_0 and perfect matchings in the decorated graph G_d , instead of the direct mapping used in [98].
- We use Blossom V instead of Blossom IV. Although both implementations have the same worst-case asymptotic time complexity, Blossom V is usually considered faster in practice [67].

We were also able to use Blossom V to compute the log partition function for a 600×600 image (a graph with 0.36 million nodes and 0.72 million edges) in 100 CPU seconds. A vector of edge marginals for the latter, however, would take over 100 CPU hours to compute.

6.4 Territory Prediction in Go

We apply planar Ising CRFs to territory prediction in the game of Go. We propose a two-stage graph reduction of the position on the Go board grid, with the first stage used for parameter estimation and the second for inference. We use our polynomial-time algorithms for both model parameter estimation and computation of the MAP state for territory prediction.

6.4.1 The Game of Go in Machine Learning

Go, known as *wei-qi* in China and *baduk* in Korea, is a board game that originated in China over 4000 years ago. It is played on a 19×19 grid, though smaller boards are commonly used by beginners and computers. Two players (black and white) alternate in placing stones on the intersections of the grid. Once a stone is placed it cannot be moved, but it can be removed (*captured* or *killed*) if all its empty neighbours (*liberties*) are occupied by opponent stones. Neighboring stones of the same color form a contiguous *block*. Note that a block can contain a single stone. Stones in a block are surrounded and captured as one. Players aim to create *alive blocks*, *i.e.*, blocks that cannot be captured. The area occupied by these alive blocks counts as their final *territory*. The winner of the game is the player who controls the most territory at the end.

Computer Go

From these simple rules emerges a game of greater subtlety and strategic complexity than Chess. It is not surprising then that there is great interest in creating computer programs that can play *go*. Unlike

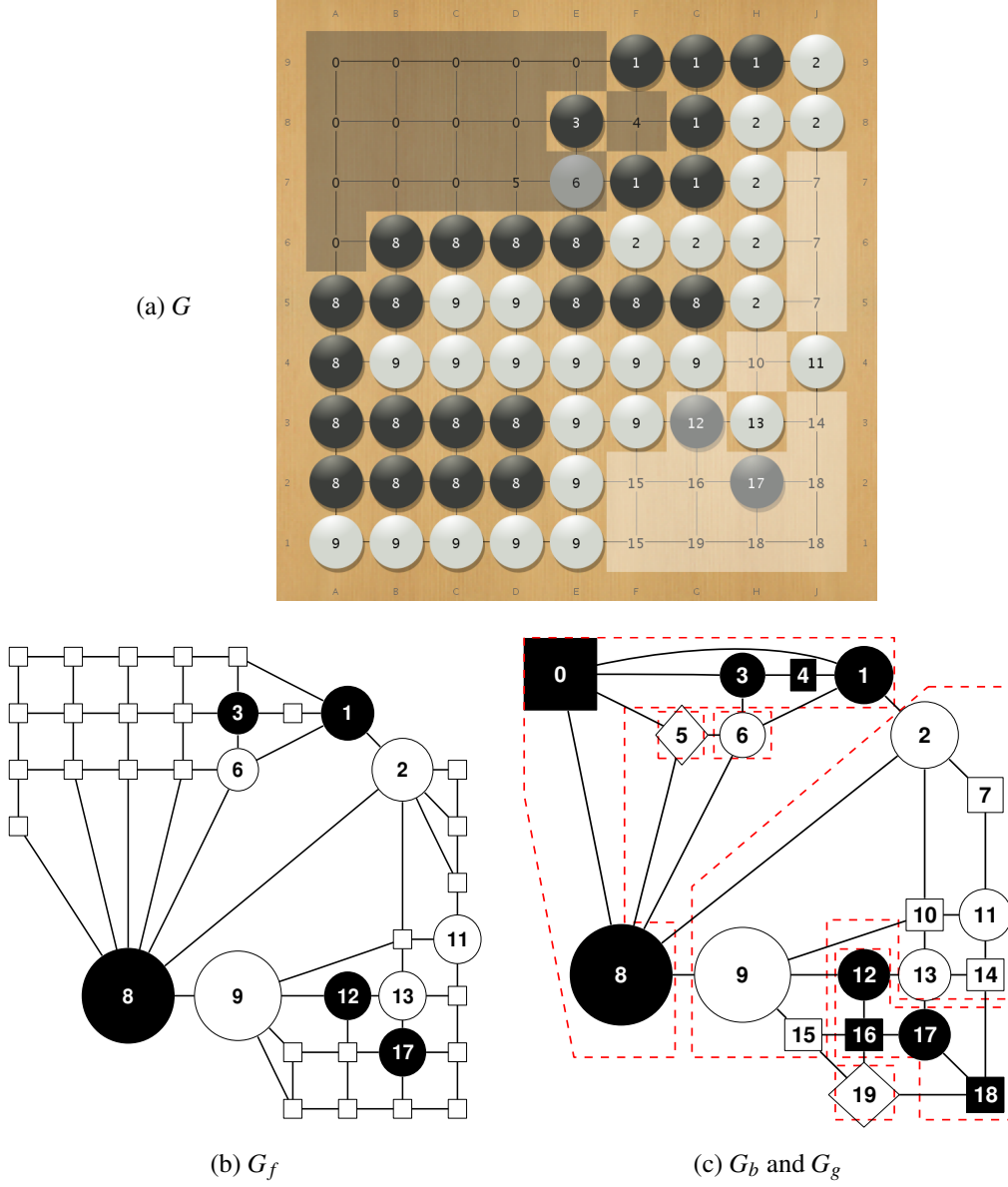


Figure 6.4: (a) A typical 9×9 endgame board position corresponding to a grid graph $G(\mathcal{V}, \mathcal{E})$. The player's final territory is his alive stones plus the area shaded with his color. Stones ⑥, ⑫ and ⑰ are dead; all other stones are alive. (b) Corresponding common fate graph $G_f(\mathcal{V}_f, \mathcal{E}_f)$. (c) Corresponding block graph $G_b(\mathcal{V}_b, \mathcal{E}_b)$. $\circ$ represent stones, $\square$ represent surounds, and $\diamond$ are neutral. Node size reflects the number of points in the block. Dashed red lines indicate nodes of the group graph $G_g(\mathcal{V}_g, \mathcal{E}_g)$.

Chess, however, *go* is not amenable to conventional game tree search. This is because its branching factor of about 200 does not permit search tree expansion to a depth anywhere near that employed by human players, who routinely look ahead dozens of moves. Static position evaluation is an even greater challenge, since (again in contrast to Chess) individual stones carry little information. It is the details of the shape and arrangement of entire *groups* of blocks of stones that determine their fate. At the same time, that fate can depend on the precise location of a faraway individual stone, so it does not suffice to simply combine the results of local searches.

Due to these difficulties, *go* programs of the past took an intensive knowledge-based approach, combining many local searches with pattern matching on configurations of stones. The patterns were typically handcrafted or harvested from search techniques. Recently, Monte-Carlo game-tree searches have shown great promise on the small (9×9) *go* board, and considerable progress has been made on 19×19 boards. These Monte Carlo *go* programs critically depend on very fast evaluation of their leaf nodes.

Go is the second-most researched board game after Chess [91]. Despite all this activity, *go* remains a paradigmatic unsolved AI problem, with the best computer programs playing at the level of a strong human amateur.

Machine Learning and Go

Go is a very interesting domain for machine learning: it is a finite, deterministic, discrete, zero-sum, perfect-information game, and thus represents the closest we can come to a fully observable real-world domain. Yet the domain is extremely complex—even its subproblems (*e.g.*, ladders and endgame) are known to be in PSPACE [18, 140]. It is generally acknowledged that the traditional knowledge engineering paradigm to computer *go* is at a dead end, and future advances are expected to come from machine learning and Monte Carlo techniques. Millions of recorded games between highly proficient human players, including professionals, are available on the Internet. Now the challenge is to develop techniques that effectively use all this data.

Territory Prediction

An important subproblem in *go* is *territory prediction*: given a board position, determine which player *controls* each intersection. A player controls an empty intersection if his opponent cannot place stones at that intersection without them being eventually captured¹⁰ (shaded empty intersections in Figure 6.4(a)). A player controls a stone if that stone cannot be captured, making it *alive*¹⁰ (unshaded stones in Figure 6.4(a)); otherwise it is *dead*¹⁰ (shaded stones in Figure 6.4(a)).

Since the game is won by whoever controls more territory at the end, territory prediction amounts to static evaluation and is therefore inherently difficult. In fact, a game of *go* ends as soon as both players agree on their territory prediction—so by definition they disagree on it throughout the game.

Even the most sophisticated exact methods cannot exhaustively solve board positions that contain more than 18 enclosed empty points (this is the case in 41% of the games in our 9×9 training set), and

¹⁰To be rigorous, these definitions would have to assume perfect play on both sides. Except for very restricted situations, however, optimal play in *Go* is unknown; working definitions must therefore be relative to some notion of “reasonable” play. For instance, in Figure 6.4(a) black could rescue ⑫ and ⑬ by joining them to an alive block with two eyes, but only if white were to play quite irrationally. Less clear-cut cases are what makes *Go* territory prediction so challenging.

take up to 200 seconds for a game they can solve [92, 93]. Therefore a heuristic approach is called for. Schraudolph et al. [112] use temporal difference learning in convolutional neural networks to predict territory; Wu and Baldi [142] employ a recursive neural network architecture. van der Werf et al. build a series of classifiers that learn to estimate potential territory [2004], score final positions [2005], and predict life and death [2005].

6.4.2 Go Positions as Graph Abstraction Hierarchies

Go positions are most naturally represented as graphs, suggesting the use of graphical models. These graphs can have various levels of abstraction, forming an *abstraction hierarchy* [35]. We will use different levels of that hierarchy for territory modelling and prediction.

Grid Graph

The *go* board defines a square grid graph $G(\mathcal{V}, \mathcal{E})$ whose intersections can be in one of three states: *black*, *white*, or *empty* (Figure 6.4(a)). Stern et al. [119] model the territory distribution with an MRF defined on G . Their system uses sampling and Loopy Belief Propagation (LBP) to learn just 6 parameters for node and edge potentials. We believe this is insufficient to model the complex interaction between stones in *go*.

van der Werf et al. [130, 131, 132] obtain better prediction accuracy by avoiding graphical models altogether. However, this forces them to describe each node of the grid with a plethora of hand-crafted, domain-specific features, which are prone to cause overfitting.

Common Fate Graph

It is the properties of blocks, rather than individual stones, that determines their fate: blocks always live or die as a unit. It is therefore inherently wasteful to model the individual stones of a block, as the grid graph does. Worse, it is counterproductive: interactions between neighboring blocks, which are essential to the game, may appear as hard-to-model long-range interactions at the grid level. It thus makes sense to merge all stones in a block into a single node, and move information such as the size and shape of the block—which would otherwise be lost—into the block’s node features (Figure 6.4(b)). We will refer to such blocks as *stone blocks*. Graepel et al. [40] use this *common fate graph* $G_f(\mathcal{V}_f, \mathcal{E}_f)$ to train SVMs to discriminate between good and bad moves. One of the strongest learning-based *go* program uses a neural network that also learns from a common fate graph representation [26, 27].

Block Graph

The common fate graph still represents empty grid intersections individually. Regions of empty intersections do not have the common fate property; they can be (and usually are) divided up between the players. Therefore collapsing them as well as stone blocks results in too impoverished a representation. On the other hand, not collapsing them produces unwieldy, large graphs, especially early in the game. We compromise by grouping empty intersections into three types before collapsing regions of the same type. Specifically, we construct our *block graph* as follows:

Classify the intersections of the grid graph $G(\mathcal{V}, \mathcal{E})$ (Figure 6.4(a)) into black stones ●, white stones ○, and three types of empty intersections: *black surround* ■, *white surround* □ and *neutral* ◇. To determine the type of all empty intersections we use a flood fill algorithm to find the Manhattan distances from each intersection to the nearest black *resp.* white stone. Each intersection is then colored ■, ◇, or □ depending on whether the distance to the nearest black stone is less than, equal to, or greater than the distance to the nearest white stone. For instance, intersection 19 on the grid graph has neutral type since its closest white stone ⑨ and closest black stones ⑫ and ⑰ all lie a Manhattan distance of 2 away (Figure 6.4(a)). Finally, we collapse all contiguous regions of intersections of the same type to obtain the block graph $G_b(\mathcal{V}_b, \mathcal{E}_b)$ (Figure 6.4(c)). From now on, blocks that are composed entirely of empty intersections will be called *empty blocks*.

Our block graph provides a more succinct representation than the common fate graph, yet by classifying empty blocks into three types preserves the kind of information needed for predicting territory. For instance, consider Figure 6.4(c): ⑫ and ⑰ are dead because they are in the opponent’s territory. However, if there were more friendly (black) stones in the neighbourhood then they could be alive. It is important to encode this potential for a status change (*aji* in *go* terminology), as it could affect our prediction for neighbouring opponent blocks. We encode this by having black surround blocks (blocks 16 and 18 in Figure 6.4(c)). The block graph also concisely encodes the notion of *eyes* — small empty regions surrounded by stones of a single color. Eyes are a literally vital concept in *go*, because a stone block with two or more eyes cannot be captured.

Group Graph

Stone blocks of one color that share the same surround are very unlikely to end up with different labels — in fact we have not found a single instance of such an occurrence in our set of 18 000 9×9 games. Since such groups of blocks often share the same fate (*e.g.*, all live or all die), we can group them together (dashed lines in Figure 6.4(c)). We use the resulting *group graph* $G_g(\mathcal{V}_g, \mathcal{E}_g)$ for prediction; for parameter estimation we use the block graph, because the fate of a group very much depends on the shape of its constituent blocks, albeit shared amongst them.

The grid G , block G_b , and group G_g graphs are the first three levels in a 5-stage graph *abstraction hierarchy* of *go* positions introduced by Friedenbach, Jr. [35], who also presents an algorithm for incrementally updating such a hierarchy. This hierarchical graph representation is intended to model how proficient *go* players evaluate positions. Integrating yet higher levels of Friedenbach’s abstraction hierarchy into a *go* learning program could be a promising direction for future work.

We note that the idea of graph abstraction hierarchies is also found in other fields, such as computer vision. For example, an image can be decomposed into homogenous regions (*e.g.*, sky, grass, building etc.), while each region can be decomposed into separate objects and finally each object is composed of individual pixels.

Planarity

We note that the *go* grid G , which is drawn on a plane board, is by definition planar. Since common fate graph, block graph, and group graph are all derived from the grid graph via edge contractions (which preserve planarity) they are also planar. This allows us to employ our exact polynomial-time algorithms

from Chapter 5 for both model parameter estimation on the block graph and territory prediction on the group graph.

6.4.3 Conditional Random Field Model

We will use the block graph G_b described in Section 6.4.2 to encode the conditional independence assumptions of our Ising CRF model. Following (6.1) the L_2 -regularized negative log-likelihood $L_{\text{ML}}(\theta)$ of the data under our model parameterized by θ can be written as

$$L_{\text{ML}}(\theta) := \frac{1}{2} \lambda \|\theta\|^2 + \ln Z(\theta) + \sum_{(i,j) \in \mathcal{E}_b} \mathbb{I}[y_i^b \neq y_j^b] E_{ij}^b, \quad (6.13)$$

$$\text{where} \quad Z(\theta) := \sum_y \exp \left(\sum_{(i,j) \in \mathcal{E}_b} \mathbb{I}[y_i^b \neq y_j^b] E_{ij}^b \right) \quad (6.14)$$

is the partition function, and y^b the binary labels specifying territory (BLACK or WHITE) of the nodes (blocks) in G_b . Note that the original *go* positions can also have “neutral” territory, which does not belong to any player. We deal with such cases separately in Section 6.4.4. $E_{ij}^b := \theta^\top \phi_{ij}^b$ is the *disagreement cost* of edge $(i, j) \in \mathcal{E}_b$. We now consider how to compute the *sufficient statistics* ϕ_{ij}^b for every edge (i, j) of our block graph.

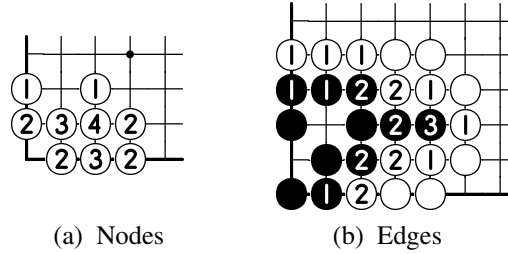


Figure 6.5: Computation of node and edge features of G_b . (a) Stone block, resulting in node feature vector $x_{\circ} = [2, 4, 2, 1, 9]$. For example, the 4th element is 1 since there is only one ④, *i.e.*, an intersection with 4 neighbours. (b) Two neighbouring stone blocks, resulting in edge feature vectors $x_{\bullet\circ} = [3, 3, 1, 0, 7]$ and $x_{\circ\bullet} = [6, 3, 0, 0, 9]$.

Node Features

The fate of a block is largely determined by its shape and the arrangement of its nearby blocks. Therefore we believe that it is crucial to accurately capture the shape of a block. *Neighbour classification* of Vilà and Cazenave [134] provides a powerful way to summarise a block’s shape, which is particularly important for empty blocks. Vilà and Cazenave [134] showed that in *go*, empty blocks with identical neighbour classification have identical eye-forming potential.

For our node features we use an extension of neighbour classification. We let $\mathbf{x}_i$ be the vector of raw node features for block $i \in \mathcal{V}_b$. For each intersection in a block (stone block or empty block) we compute the number of adjacent points that are in the same block (Figure 6.5(a)). A block’s shape feature is a vector of length five whose k^{th} element counts the number of points with exactly k neighbours

Node		Edge Params.	Edge Types	Edge (i, j) with type $\bullet \blacksquare$			
Node Params.	Node Types			x_i	$x_{i \rightarrow j}$	$x_{j \rightarrow i}$	x_j
$\theta_\diamond$	$\diamond$	$\theta_{\circ\circ}$	$\bullet \rightarrow \circ \ \& \ \circ \rightarrow \bullet$	$\theta_\circ$	$\theta_{\circ\blacksquare}$	$\theta_{\blacksquare\circ}$	$\theta_{\blacksquare}$
$\theta_\circ$	$\bullet \ \& \ \circ$	$\theta_{\circ\blacksquare}$	$\bullet \rightarrow \blacksquare \ \& \ \circ \rightarrow \square$	$\theta_\circ^\top x_i + \theta_{\circ\blacksquare}^\top x_{i \rightarrow j} + \theta_{\blacksquare\circ}^\top x_{j \rightarrow i} + \theta_{\blacksquare}^\top x_j$			
$\theta_\square$	$\blacksquare \ \& \ \square$	$\theta_{\blacksquare\circ}$	$\blacksquare \rightarrow \bullet \ \& \ \square \rightarrow \circ$	Edge (i, j) with type $\bullet \circ$			
		$\theta_{\blacksquare\diamond}$	$\blacksquare \rightarrow \diamond \ \& \ \square \rightarrow \diamond$				
		$\theta_{\diamond\circ}$	$\diamond \rightarrow \bullet \ \& \ \diamond \rightarrow \circ$				
		$\theta_{\diamond\blacksquare}$	$\diamond \rightarrow \blacksquare \ \& \ \square \rightarrow \diamond$				
		$\theta_{\diamond\square}$	$\diamond \rightarrow \diamond \ \& \ \square \rightarrow \square$	x_i	$x_{i \rightarrow j}$	$x_{j \rightarrow i}$	x_j
				$\theta_\circ$	$\theta_{\circ\circ}$	$\theta_{\circ\circ}$	$\theta_\circ$
				$\theta_\circ^\top x_i + \theta_{\circ\circ}^\top x_{i \rightarrow j} + \theta_{\circ\circ}^\top x_{j \rightarrow i} + \theta_\circ^\top x_j$			

Edge Features

- Two stone blocks can merge into one by connecting with each other.
- A stone block can become alive by forming two eyes in the nearby empty space.
- A stone block can surround an opponent block by reducing its liberties.

For this purpose we define our edge features using a variant of neighbour classification [134]. Let $\mathbf{x}_{i \rightarrow j}$ be the vector of raw edge features for the edge $(i, j) \in \mathcal{E}_b$ as viewed from block i . For each point in block i we compute the number of adjacent points that are in block j . We now let $\mathbf{x}_{i \rightarrow j}$ be a vector of length five whose k^{th} element counts the number of points in block i with exactly k neighbours in block j . For completeness, the 5th element of $\mathbf{x}_{i \rightarrow j}$ stores the length of the boundary between i and j . Vector $\mathbf{x}_{j \rightarrow i}$ is computed similarly, but with the roles of i and j reversed (Figure 6.5(b)). Note that the elements of $\mathbf{x}_{i \rightarrow j}$ and $\mathbf{x}_{j \rightarrow i}$ can be different. It is also worth noting that our edge features provide additional information that is not conveyed by the node features alone. This contrasts with the common approach of creating edge features that are simply some arithmetic combination (*e.g.*, dot-product, magnitude of difference) of node features.

Since (6.13) provides for edge disagreement costs only, we move our node features into the features of each adjoining edge. Because our edge features are non-commutative, we provide shape vectors for both orientations. This means that the full feature vector for an edge (i, j) comprises of four parts, two node feature vectors and two edge feature vectors: $\mathbf{x}_{ij} := [\mathbf{x}_i, \mathbf{x}_{i \rightarrow j}, \mathbf{x}_{j \rightarrow i}, \mathbf{x}_j]$ (Table 6.3, right). We

	Current Edge		Neighbour Edges	
	Parameters	Features	Parameters	Features
Nodes	$\theta_{\circ}$	$\mathbf{x}_i$	$\tilde{\theta}_{\diamond}$	$\mathbf{x}_k$
	$\theta_{\square}$	$\mathbf{x}_j$	$\tilde{\theta}_{\circ}$	$\mathbf{x}_m$
			$\tilde{\theta}_{\square}$	$\mathbf{x}_n$
Edges	$\theta_{\square\square}$	$\mathbf{x}_{i \Rightarrow j}$	$\tilde{\theta}_{\diamond\square}$	$\mathbf{x}_{k \rightarrow i}$
	$\theta_{\square\circ}$	$\mathbf{x}_{j \Rightarrow i}$	$\tilde{\theta}_{\square\diamond}$	$\mathbf{x}_{i \rightarrow k}$
			$\tilde{\theta}_{\square\square}$	$\mathbf{x}_{m \rightarrow i}$ $\mathbf{x}_{i \rightarrow m}$
			$\tilde{\theta}_{\square\square}$	$\mathbf{x}_{n \rightarrow j}$ $\mathbf{x}_{j \rightarrow n}$

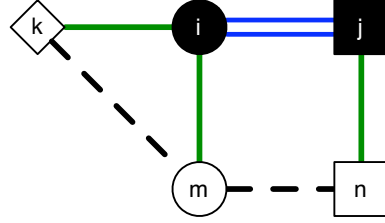


Table 6.4: Parameters and features (left) used to compute the disagreement cost of edge (i, j) in a small block graph (right). Neighbouring edges are shown in green.

perform normalisation and power expansion, replacing each feature value $x \in \mathbb{N}$ with $1/(x+1)$, $x/20$, and $x^2/400$, and provide a constant bias feature set to 1. Such an expansion allows the Ising CRF to capture non-linear interactions between the features.

Parameter Sharing

For each *type* of node and edge, we provide a *single* parameter vector that is shared between all nodes *resp.* edges of that kind. There are three types of node parameters, one for each node type: *stone* ($\theta_{\circ}$), *surround* ($\theta_{\square}$), and *neutral* ($\theta_{\diamond}$) (Table 6.3, left). There are eight types of edge parameters resulting from all possible types of node pairings (Table 6.3, center). We let $\theta_{t(i)}$ and $\theta_{t(i,j)}$ denote the parameter vectors for a node of type $t(i)$ and edge of type $t(i, j)$, respectively. We aggregate these 11 parameter vectors into a single vector θ , and match input features with the appropriate parameters via a *routing* matrix R that depends on the kind of edge present: $\phi_{ij} := R_{ij}\mathbf{x}_{ij}$.

Neighbour Features

To assist the propagation of *go* knowledge along the graph, for each edge we also include the features, weighted by $\tilde{\theta}$, of its neighbouring nodes and edges. Table 6.4 shows the resulting features and parameters for one particular edge (double blue line). With this new model, the final disagreement cost E_{ij} for edge (i, j) becomes

$$\begin{aligned}
E_{ij} = & \theta_{t(i)}^{\top} \mathbf{x}_i + \theta_{t(i,j)}^{\top} \mathbf{x}_{i \rightarrow j} + \theta_{t(j,i)}^{\top} \mathbf{x}_{j \rightarrow i} + \theta_{t(j)}^{\top} \mathbf{x}_j \\
& + \sum_{k \sim i; k \neq j} (\tilde{\theta}_{t(k)}^{\top} \mathbf{x}_k + \tilde{\theta}_{t(k,i)}^{\top} \mathbf{x}_{k \rightarrow i} + \tilde{\theta}_{t(i,k)}^{\top} \mathbf{x}_{i \rightarrow k}) \\
& + \sum_{k \sim j; k \neq i} (\tilde{\theta}_{t(k)}^{\top} \mathbf{x}_k + \tilde{\theta}_{t(k,j)}^{\top} \mathbf{x}_{k \rightarrow j} + \tilde{\theta}_{t(j,k)}^{\top} \mathbf{x}_{j \rightarrow k})
\end{aligned} \tag{6.15}$$

where $\sim$ denotes the adjacency relation between nodes, *i.e.*, $k \sim i$ is a node k that is adjacent to node i .

Parameter Estimation

We train our Ising CRF model on the block graph G_b by penalised maximum likelihood (ML) parameter estimation, which involves minimising the negative log-likelihood (6.13). Since this is a convex optimisation problem, efficient quasi-Newton solvers such as LBFGS can be employed to good effect [86, 95]. These solvers require the gradient of the log-likelihood

$$\frac{\partial}{\partial \theta} L_{\text{ML}}(\theta) = \lambda \theta + \sum_{(i,j) \in \mathcal{E}_b} \left(\mathbb{I}[y_i^b \neq y_j^b] - \frac{\partial \ln Z(\theta)}{\partial E_{ij}^b} \right) \phi_{ij}^b, \quad (6.16)$$

where we compute $\partial \ln Z(\theta) / \partial E_{ij}^b$ via the inverse of the Kasteleyn matrix (5.23).

Prediction

Recall that we train on the block graph G_b , but predict on the group graph G_g . Parameter estimation on the block graph provides us with $\theta^* := \operatorname{argmin}_{\theta} L_{\text{ML}}(\theta)$. We use θ^* to compute the disagreement costs for the block graph $E_{ij}^b := \theta^{*\top} \phi_{ij}^b$. These allow us to compute the disagreement costs of the group graph:

$$E_{g(i), g(j)}^g := \sum_{(i,j) \in \mathcal{E}_b} E_{ij}^b, \quad (6.17)$$

where $(g(i), g(j)) \in \mathcal{E}_g$ and $g(i)$ is the group of block $i \in \mathcal{V}_b$. See Figure 6.6 for an example. We can

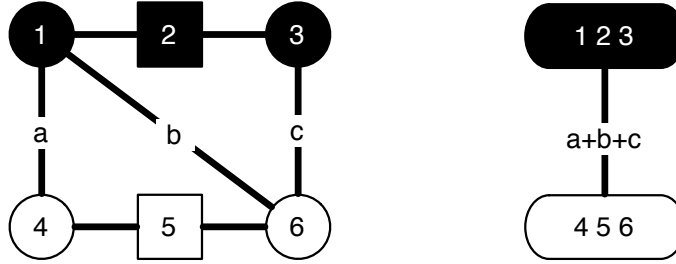


Figure 6.6: Left: Block graph G_b with edge disagreement costs a, b, c . Right: Corresponding group graph G_g .

now use blossom-shrinking to efficiently compute the MAP state of the group graph:

$$\mathbf{y}^{g*} := \operatorname{argmax}_{\mathbf{y}^g} \mathbb{P}(\mathbf{y}^g | \mathbf{x}; \theta^*). \quad (6.18)$$

We set the state of each block $\mathbf{y}_i^b$ to the state of its parent group $\mathbf{y}_{g(i)}^{g*}$. Similarly, we set the state of each intersection $\mathbf{y}_i$ to the state of its parent block. Since G_b is generally not outerplanar (Section 5.1.2), our parameter estimation model does not use a constant bias node, as was done in Section 5.1. For the same reason, our prediction model does not use a constant bias node, and thus due to label symmetry there are two possible MAP states. We resolve this ambiguity by a domain-specific heuristic: there are usually (*e.g.*, in 99.4% of our 9×9 games) more alive than dead stones. Of the two MAP states $\mathbf{y}^{g*}$ we therefore pick the one which maximises the number of alive stones, resolving ties randomly.

6.4.4 Experiments

We now evaluate our Ising CRF model and inference algorithms in terms of their territory prediction accuracy and speed, on several sets of final *go* positions at different board sizes.

Datasets

9 × 9 games. van der Werf et al. [130, 131, 132] have created a collection of about 18 000 9 × 9 games that were played between 1995 and 2002 on the No Name Go Server (NNGS), whose territory has been determined using a combination of manual labeling, GNU Go 3.6 [39], and their own classifiers. We use a subset of this collection of games for our experiments: 1 000 games played between 1996 and 2002 for training and validation, and 906 games played in 1995 for testing. The average rank of ranked players is 17 kyu in the training set and 12 kyu in the test set.

19 × 19 games. We had great difficulty in finding a sufficiently large collection of 19 × 19 games whose labels have been independently verified. Thus we developed our own heuristic scoring method to label a set of games from the Kiseido Go Server (KGS), and picked only games labeled identically by our scorer and the players themselves for the dataset. This heuristic scorer is completely independent from our Ising CRF classifier and thus does not cause any bias. The training and test set consist of 1 000 games each, played between January 2005 and January 2003, respectively. The average rank of ranked players is 11 kyu for both training and test sets.

Oversize games. We manually scored 22 games that were played on KGS in December 2004 with extra-large board sizes ranging from 21 × 21 to 38 × 38. Compared to the 9 × 9 and 19 × 19 datasets, these games had a higher average number of dead blocks and stones, making them a greater challenge to score. The average rank of ranked players in these games was 15 kyu.

Territory Prediction Accuracy

Task. Given an endgame board position G we want to predict the label (BLACK, NEUTRAL or WHITE) of each intersection in $\mathcal{V}$. However, our classifier can only make binary predictions. This is not a problem though: by predicting BLACK and WHITE we obtain the location of all dead stones; once those are removed identifying NEUTRAL intersections becomes simple. We use four measures of test error:

- *Stone error* is the percentage of misclassified stones in $\mathcal{V}$.
- *Block error* is the percentage of misclassified stone blocks in $\mathcal{V}_b$.
- *Game error* is the percentage of games that have at least one misclassified stone block in $\mathcal{V}_b$.
- *Winner error* is the percentage of games whose winner (using Chinese scoring) is predicted incorrectly.

Method. We trained our Ising CRF model on 1 000 block graphs until convergence, using LBFGS as the optimizer. The regularizer λ was tuned for best performance, which was found at $\lambda = 3/4$. Predicted territory was determined from the MAP state of the group graphs on the test set. We compared our approach to four control methods:

- The naive method of assuming that all stones are alive, *i.e.*, always labelling ● as BLACK and ○ as WHITE.
- [GNU Go 3.6](#), an open-source *go* program that scores endgame positions by combining *go*-specific knowledge with results from many local searches.
- A neural network (NN) that uses a total of 63 *go*-specific features of graphs at several levels of abstraction. This is the best classifier from the collection built by van der Werf et al. [131] to predict the final territory of 9×9 endgame positions.
- A simple MRF (just 6 parameters) trained by Stern et al. [119] on the grid graphs of 290 19×19 expert games. Inference is performed via 50 iterations of loopy belief propagation (LBP) to compute approximate marginal expectations at each intersection, which are then thresholded to yield a prediction. In contrast to our approach of using the MAP state, marginals can lead to inconsistent labelings (for example see Figure 6.7, left). Since this method has no notion of blocks and would therefore do poorly on the block error, we give it a helping hand in our comparison by setting each block's label to the label of the majority of its stones.

Results. Table 6.5 compares our approach with the controls on various board sizes. Our method (Ising_9 and Ising_19) significantly outperforms MRF, the only other learning method based on graphical models, on all board sizes. On 9×9 it appears inferior to approaches that heavily rely on domain knowledge (GnuGo and NN). However, it must be noted that this dataset was labelled using a combination of GnuGo and NN classifiers in the first place [131], hence is quite likely biased towards these methods.

It is interesting to note that MRF is relatively better at stone error than block error. This is due to the fact that MRF has no concept of blocks and treats all stones as individuals. Thus, unlike our method, an error in one stone does not cause errors in other stones of the same block.

On 19×19 our method is clearly worse than GnuGo. To put this in perspective though, consider that GnuGo takes about four orders of magnitude as long to make a prediction here (Section 6.4.4). Our model trained on 9×9 positions (Ising_9) does not perform much worse here than the natively trained Ising_19, indicating that we are able to transfer knowledge across board sizes. Figure 6.7, left shows a sample position, with the errors made by each method marked.

To investigate their scalability, we also tested the graphical models on our small set of oversize games. Even though players of differing skill levels on boards of differing sizes tend to reach different types of final positions, our architecture manages a fair amount of information transfer; Figure 6.7, right gives an example where our model made no error.

Territory Prediction Speed

Task. In this set of experiments, we compare the CPU times taken by various territory prediction methods as a function of $|\mathcal{V}_g|$. All experiments were performed on an Intel Pentium 4 running at 3GHz with 1GB RAM and 512KB level 2 cache. We compare the following five methods:

- [GNU Go 3.6](#) scoring the position in *aftermath* mode.
- The approach of Stern et al. [119]: 50 iterations of loopy belief propagation (LBP) to calculate approximate marginal expectations on the grid graph.

Size	Algorithm	Error (%)			
		Stone	Block	Game	Winner
9×9	Naive	8.80	17.57	75.70	30.79
	MRF	5.97	8.19	38.41	13.80
	Ising_9	2.32	2.57	9.05	2.32
	GnuGo*	0.05	—	1.32	—
	NN*	0.19	≤ 1.00	1.10	0.50
19×19	Naive	6.96	16.52	98.30	32.60
	MRF	3.80	4.91	63.90	20.50
	Ising_9	3.91	5.22	61.90	14.30
	Ising_19	3.81	3.93	43.40	9.30
	GnuGo	0.11	—	5.10	—
greater than 19×19	Naive	10.25	19.64	100.00	31.81
	MRF	6.83	7.80	100.00	22.73
	Ising_9	6.64	6.95	95.45	13.64
	Ising_19	5.02	4.52	81.82	9.09

*Bias alert: dataset creation involved these methods (see text for details).

Table 6.5: Prediction error of different methods for various board sizes. Naive is the naive error, MRF is the work of Stern et al. [119], GnuGo is GNU Go 3.6 [39], NN is the work of [131], Ising_9 and Ising_19 is our method trained on 9×9 and 19×19 games, respectively. We could not easily obtain the block and winner errors for GnuGo. The minimum in each result column is boldfaced.

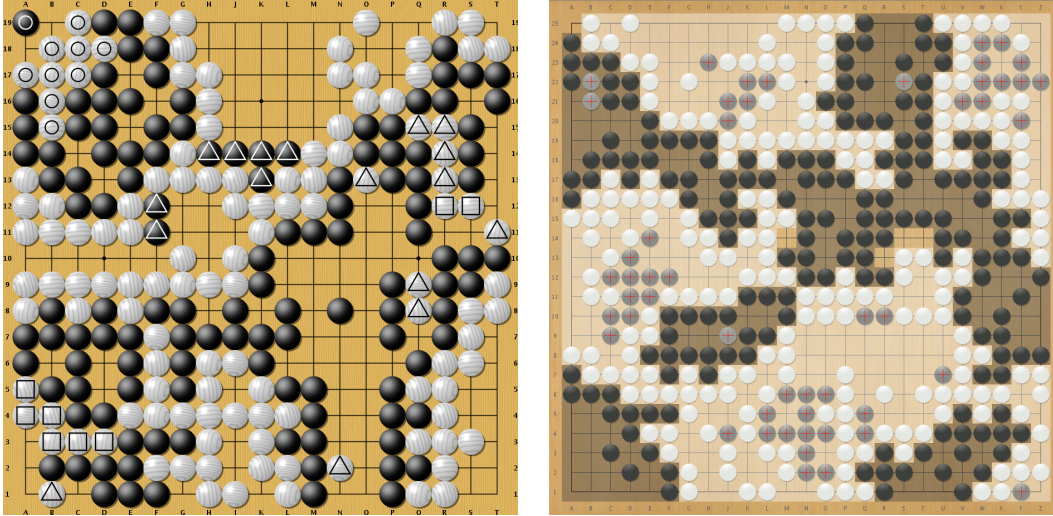


Figure 6.7: **Left:** a 19×19 endgame position showing the errors made by different methods. Stones marked with $\circ$ are misclassified by Naive, MRF and Ising CRF; $\square$ by Naive and MRF; $\triangle$ by Naive only. GnuGo made no errors in this game. Notice how MRF misclassifies stones at R12 and S12 and yet manages to get the rest of the block labelled correctly. **Right:** A 25×25 endgame position, whose territory assignment is predicted correctly by our system trained on 19×19 boards. By comparison, MRF on the same position predicts 33 stones (in 4 blocks) incorrectly.

- “Brute force”: MAP state calculation using variable elimination with arbitrary elimination ordering, on the group graph.
- MAP state calculation using variable elimination with elimination ordering determined by the min-fill heuristic [64], on the group graph.
- Our approach: MAP state calculation by running Blossom IV [17] on the expanded dual of the group graph.

Results. Figure 6.8 shows that our approach (blossom) is about three orders of magnitude faster than popular inference methods for graphical models, both exact (variable elimination) and approximate (LBP). The knowledge-intensive GnuGo is an order of magnitude slower still, though admittedly far more accurate. Only the primitive brute force method can compete in speed with the blossom-shrinking approach on small graphs, as encountered in 9×9 *go*. However, its exponential complexity means that it is useless for 19×19 *go* positions, let alone larger problems of any kind. Blossom-shrinking, by contrast, scales as $O(|\mathcal{E}_g| |\mathcal{V}_g|^3)$ in the worst case.

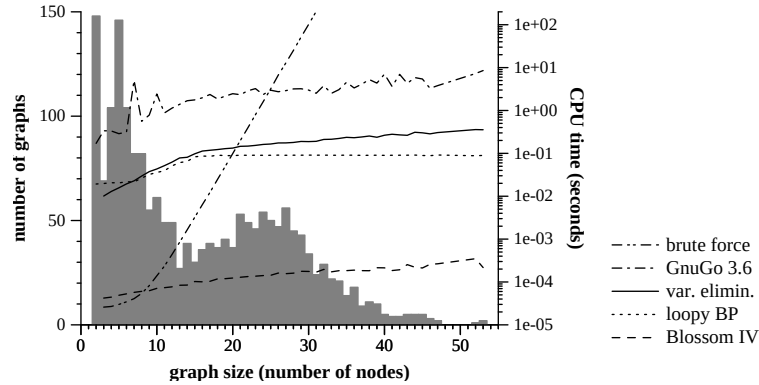


Figure 6.8: Average CPU time taken for territory prediction methods as a function of $|\mathcal{V}_g|$. Grey histogram indicates the number of graphs of a given size: the two broad peaks correspond to board sizes 9×9 and 19×19 .

6.4.5 Outlook and Discussion

We presented an exact and efficient method for territory prediction in *go*. Rather than the naive grid graph, our method uses higher-order representations, namely the block and group graphs, to encode dependencies. Our inference algorithm is highly efficient and scalable, yielding predictions orders of magnitude faster than conventional approximate and exact graphical inference techniques. As a result, we were able to predict territory on *go* boards as large as 38×38 .

Our Ising CRF model could be improved in terms of its prediction accuracy. We suspect that our generic shape-based features do not capture all the intricacies of life and death in *go*. Some domain-specific feature engineering could greatly boost performance in that regard.

On the other hand, we note that Monte-Carlo *go* programs, which currently dominate computer *go*, rely on fast rather than accurate static evaluation of their leaf nodes. While we have applied our model to territory prediction, it is generic enough to be adapted to a wide range of computer *go* subproblems.

The speed of inference in our model makes it suitable for leaf evaluation in game-tree searches and Monte-Carlo *go*.

6.5 Conclusion

In this chapter we have demonstrated the suitability of our planar Ising CRF on three different tasks. Although these are relatively simple tasks, they are sufficiently large to show that our approach is correct and efficient. For parameter estimation we have used both penalized maximum likelihood and maximum margin methods. We have shown that maximum margin is orders of magnitude faster than penalized maximum likelihood. Furthermore, maximum margin converges to parameters that are superior for prediction.

In terms of prediction, we have shown that the computation of MAP states via blossom-shrinking is orders of magnitude faster than the computation of M^3P states. In particular, we were able to obtain the MAP state for a random 1500×1500 Ising grid in 200 CPU seconds. We have found reconstruction via MAP to be similar to that of M^3P , despite the latter's theoretical advantages [33].

Non-Planar Ising Graphical Models

The algorithms described in Chapter 5 provide a foundation for *approximate* inference algorithms for non-planar Ising graphical models. Importantly the method for calculating the ground state (Section 5.2) also works exactly for non-planar graphs whose ground state does not contain frustrated non-contractible cycles. In this chapter we discuss approximative algorithms for Ising graphical models whose ground state does contain frustrated non-contractible cycles.

7.1 Edmond's Blossom-Shrinking Algorithm

In Chapter 5 we have used the blossom-shrinking algorithm [23, 24] for computing the maximum-weight perfect matching $\mathcal{M}$ (Definition 14) of our expanded dual graph G_E^* . We then used $\mathcal{M}$ to obtain the exact ground state of a planar model graph G . To understand the process for non-planar graphs G , we begin by reviewing the blossom-shrinking algorithm, closely following the description of Kolmogorov [67].

Let $G(\mathcal{V}, \mathcal{E})$ be an undirected weighted graph. Each edge $e \in \mathcal{E}$ has a corresponding weight c_e . Our goal is to compute a perfect matching $\mathcal{M}^* \subseteq \mathcal{E}$ with the maximum weight $\sum_{e \in \mathcal{M}^*} c_e$. We can represent a perfect matching $\mathcal{M} \subseteq \mathcal{E}$ with an incidence vector $\mathbf{x} \in \{0, 1\}^{|\mathcal{E}|}$, where $x_e = \mathbb{I}[e \in \mathcal{M}]$. For a subset $\mathcal{S} \subseteq \mathcal{V}$, let $\delta(\mathcal{S})$ be the set of boundary edges of $\mathcal{S}$: $\delta(\mathcal{S}) := \{(i, j) \in \mathcal{E} : i \in \mathcal{S}, j \in \mathcal{V} \setminus \mathcal{S}\}$. Let $\mathcal{O}$ be the set of all subsets of $\mathcal{V}$ that have an odd number of nodes greater than one. The blossom-shrinking algorithm efficiently solves the following integer linear program:

$$\begin{aligned} & \underset{\mathbf{x}}{\text{maximize}} && \sum_{e \in \mathcal{E}} c_e x_e \end{aligned} \tag{7.1}$$

$$\text{subject to} \quad \sum_{e \in \delta(\{i\})} x_e = 1 \quad \forall i \in \mathcal{V}, \tag{7.2}$$

$$\sum_{e \in \delta(\mathcal{S})} x_e \geq 1 \quad \forall \mathcal{S} \in \mathcal{O}, \tag{7.3}$$

$$x_e \geq 0 \quad \forall e \in \mathcal{E}. \tag{7.4}$$

(7.2) ensures that there is at most one edge in $\mathcal{M}^*$ incident upon any node $i \in \mathcal{V}$. If we select any subset $\mathcal{S}$ with an odd number of nodes then at least one of those nodes must be matched to a node not in $\mathcal{S}$,

hence (7.3). Let $\mathbf{y} \in \mathbb{R}^{|\mathcal{V}|+|\mathcal{O}|}$ be a feasible dual vector, then the dual program is given as:

$$\underset{\mathbf{y}}{\text{minimize}} \quad \sum_{i \in \mathcal{V}} y_i + \sum_{S \in \mathcal{O}} y_S \quad (7.5)$$

$$\text{subject to} \quad \text{slack}(e) \geq 0 \quad \forall e \in \mathcal{E}, \quad (7.6)$$

$$y_S \geq 0 \quad \forall S \in \mathcal{O}, \quad (7.7)$$

where $\text{slack}(e)$ in Equation 7.6 denotes the *reduced cost* of edge $e = (i, j)$:

$$\text{slack}(e) := c_e - y_i - y_j - \sum_{S \in \mathcal{O} : e \in \delta(S)} y_S. \quad (7.8)$$

An edge e is said to be *tight* if $\text{slack}(e) = 0$. A set $S \in \mathcal{O}$ is called *full* if $\sum_{e \in \delta(S)} x_e = 1$. Blossom-shrinking is a primal-dual algorithm that maintains both an infeasible primal solution $\mathbf{x}$ (i.e., a non-perfect matching) and a feasible dual solution $\mathbf{y}$. The algorithm alternates dual updates that create new tight edges with primal updates that increase the size of the proposed matching. Primal feasibility is achieved when the matching becomes perfect, i.e., when $\sum_{e \in \mathcal{E}} x_e = |\mathcal{V}|/2$. At convergence, $\mathbf{x}$ corresponds to a perfect matching of maximal cost and the *complementary slackness conditions* are satisfied:

$$\text{slack}(e) > 0 \Rightarrow x_e = 0 \quad \text{and} \quad (7.9)$$

$$y_S > 0 \Rightarrow \sum_{e \in \delta(S)} x_e = 1. \quad (7.10)$$

(7.9) states that edges in the matching must be tight. One potential concern is that the dual problem (7.5) has an exponential number of variables y_S . The algorithm's efficiency is due to there being at most $O(|\mathcal{V}|)$ subsets $S \in \mathcal{O}$ with a non-zero variable y_S at any given moment. These subsets are called *blossoms*:

Definition 42 A blossom of a graph $G(\mathcal{V}, \mathcal{E})$ is a subgraph $G_b(\mathcal{V}_b, \mathcal{E}_b)$ such that the removal of any single node $i \in \mathcal{V}_b$ (and its incident edges) leaves G_b with at least one perfect matching.

A blossom can also be defined recursively: it is a cycle containing an odd number of “pseudonodes,” where a pseudonode is either a node in $\mathcal{V}$ or another blossom (dashed ovals in Figure 7.1(a)). Two adjacent pseudonodes in a blossom are connected via a *blossom-forming edge* (red edges in Figure 7.1). The key idea is that if we can match an edge coming out of a node inside a blossom, then we can recursively match the remaining even number of nodes using only blossom-forming edges (Figures 7.1(b) and 7.1(c)). Since blossom-forming edges are tight, such a completion of the matching is guaranteed to be optimal.

7.1.1 Application to Non-Planar Graphs

For convenience we define an *edge state* of an Ising graphical model:

Definition 43 An edge state of an Ising graphical model defined on graph $G(\mathcal{V}, \mathcal{E})$ is a binary state $\mathbf{y}^e \in \{-1, 1\}^{|\mathcal{E}|}$ of its edges; its energy is given by $E(\mathbf{y}^e) := \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} (1 - y_{ij}^e) E_{ij}$.

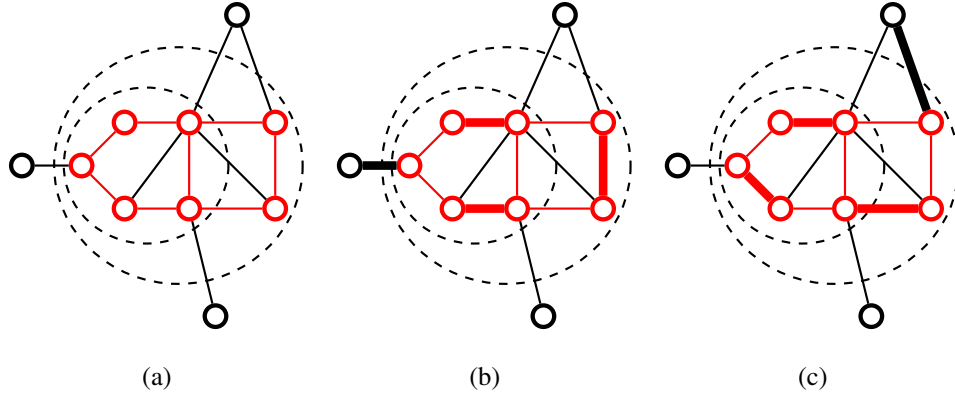


Figure 7.1: (a) A blossom containing another blossom as a pseudonode (dashed ovals). Blossom nodes and blossom-forming edges are red, while all other nodes and edges are black. (b) and (c): optimal completion of the matching (bold edges) when a node in a blossom is matched to an exterior node.

Edge states are an extension of node states: given any node state $\mathbf{y}$ with energy $E(\mathbf{y})$, we can construct an edge state $\mathbf{y}^e$ with identical energy $E(\mathbf{y}^e) = E(\mathbf{y})$ by assigning $y_{ij}^e := 2\llbracket y_i = y_j \rrbracket - 1$, $\forall (i, j) \in \mathcal{E}$. However, the converse is not necessarily true, since there are edge states with no corresponding node state. An edge state is *globally consistent* if it has a corresponding node state:

Definition 44 An edge state $\mathbf{y}^e$ of an Ising graphical model defined on graph $G(\mathcal{V}, \mathcal{E})$ is globally consistent iff $\prod_{(i,j) \in C} y_{ij}^e = 1$ for all cycles $C \subseteq \mathcal{E}$. An edge state $\mathbf{y}^e$ of an Ising graphical model with an embedded graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ is locally consistent iff $\prod_{(i,j) \in C_\Pi} y_{ij}^e = 1$ for all those cycles $C_\Pi \subseteq \mathcal{E}$ that are contractible in Π .

In other words, if an edge state $\mathbf{y}^e$ is locally consistent then every contractible cycle in G (e.g., green line in Figure 7.2) will have an even number of disagreement edges. However, the definition of local consistency does not guarantee any consistency for edges on non-contractible cycles¹¹ of G (e.g., blue and red lines in Figure 7.2): if we label the nodes by walking around the hole of the torus, the label of the final node may end up in contradiction to its starting label.

As described in Chapter 5, the maximum-weight perfect matching $\mathcal{M}$ of our expanded dual graph G_E^* can be used to obtain the exact ground state of a planar model graph G . When G is non-planar, however, blossom-shrinking only recovers the *extended ground state* of G :

Definition 45 An extended ground state of an Ising graphical model with an embedded graph $G(\mathcal{V}, \mathcal{E}, \Pi)$ is a locally consistent edge state with minimum energy.

7.2 Heuristics

For non-planar graphs blossom-shrinking only recovers the extended ground state, which in general does not uniquely determine a ground state. Since the computation of the ground state of a non-planar

¹¹ A surface of genus g contains $2g$ kinds of non-contractible cycles. By applying homology transformations to these cycles we can obtain every other cycle in the surface [28].

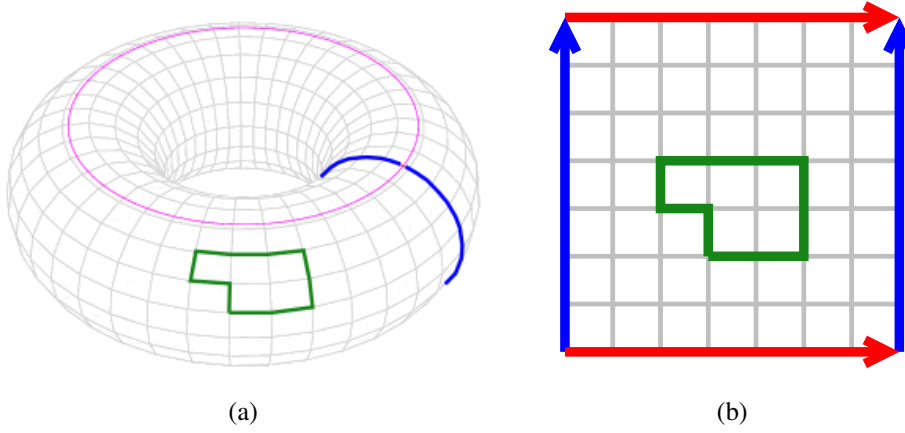


Figure 7.2: A contractible (green) and two non-contractible (blue and red) cycles on the surface of a torus. (a) 3D view of the torus (courtesy of [Wikipedia](#)). (b) the torus represented as a regular grid with periodic boundary conditions: opposite boundary edges are identified with each other.

Ising graphical model is NP-complete [2], we must resort to approximative algorithms. In this section we propose heuristic methods for obtaining ground states from the extended ground state of a non-planar Ising graphical model.

7.2.1 Low-Genus Embedding

Non-planar graphs cannot be embedded in the plane, but every graph can be drawn on some orientable surface such that no edges cross and each face is topologically a 2-disk, *i.e.*, has no holes [138].

To reduce the number of non-contractible cycles in G , we would like to obtain an embedding of as low a genus as possible. Unfortunately finding the lowest-genus embedding of a graph is NP-complete [128]. However, for regular grids we can use a simple method for constructing a (not necessarily optimal) low-genus embedding:

Consider an undirected binary graphical model with energy function (5.3) defined on a 6×6 grid, as shown in Figure 7.3(a). To construct an equivalent Ising graphical model we must connect every node to a bias node (see Section 5.1). We connect all the nodes on the perimeter to the bias node by routing the edges around the surface, as shown in Figure 7.3(b). We connect the remaining nodes through the holes in the surface, as shown in Figure 7.3(c). Each hole in Figure 7.3(c) can route up to 4 edges (from 4 nodes) without any crossings (see Figure 7.3(b)). With this method an $N \times M$ grid can be embedded on a surface with genus $\lceil N/2 - 1 \rceil \lceil M/2 - 1 \rceil$.

7.2.2 Minimax Spanning Tree

In our Ising graphical model the extended ground state is an edge state $\mathbf{y}^e \in \{-1, 1\}^{|\mathcal{E}|}$, such that the state of edge (i, j) is $y_{ij}^e := 1 - 2\mathbb{I}[(i, j) \in \mathcal{E} \setminus \mathcal{M}]$. Since the model graph is not planar, the complement $\mathcal{E} \setminus \mathcal{M}$ may not be a cut. To disambiguate the node state we use a spanning tree, as in Section 6.2.3, to obtain a minimax solution. We construct a maximum-weight spanning tree $T(\mathcal{V}, \mathcal{E}')$ of the model graph $G(\mathcal{V}, \mathcal{E})$ whose edge weights are set to $-E_{ij}y_{ij}^e$, which is the local cost of disobeying the edge state $\mathbf{y}^e$.

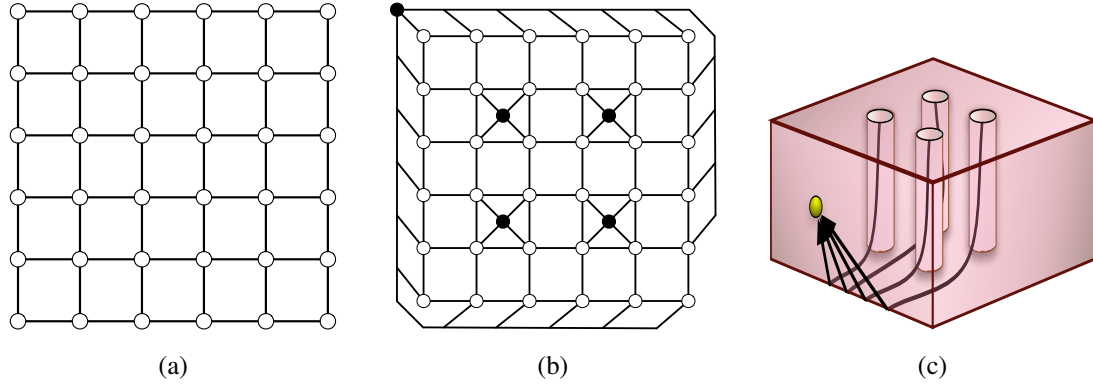


Figure 7.3: Construction of a low-genus embedding for a regular 6×6 grid (a). (b) 6×6 grid with every node (white circles) connected to the bias node (represented by multiple black disks). (c) The graph in (b) (omitted for clarity) lies on top of the embedding surface (represented as a cube). Arrows indicate the routing of connections to the bias node (yellow ball) through the 4 holes.

As before, we run Algorithm 1 on T (bold edges in Figure 7.5) to find the ground state $\mathbf{y}$ of the model graph G .

We can also take advantage of the dual slack information provided by Blossom to construct a min-max spanning tree. For each edge (i, j) of the expanded dual graph G_E^* the cost of disobeying Blossom's matching is the dual slack (i, j) (7.8). Consider the matching of G_E^* shown in Figure 7.4(a). If we remove edge (i, j) from the matching $\mathcal{M}$, then we must add two of its neighbouring edges to $\mathcal{M}$ —one from each side (one possible matching is shown in Figure 7.4(b)). Although we do not know which of the neighbouring edges of (i, j) will be added to $\mathcal{M}$, we can compute a lower bound on their cost. Hence we can set the weight of edge $(i, j) \in \mathcal{E}'$ to

$$\text{slack}(i, j) + \min_{k \in N(i) \setminus \{j\}} \text{slack}(i, k) + \min_{k \in N(j) \setminus \{i\}} \text{slack}(j, k), \quad (7.11)$$

where $N(i) \subseteq \mathcal{V}$ is the set of neighbours of node i . Since $\text{slack}(i, j)$ is always non-negative, (7.11) gives a lower bound on the cost of changing the state of (i, j) , *i.e.*, it is a min-marginal for (i, j) [65]. This is in contrast to using edge weights of the model graph G , which can be either positive or negative and hence does not provide any bound.

7.2.3 Node Quenching with Implicit Lookahead

We propose a method for improving an approximate ground state via small, local moves which we call *node quenching*. This method is based on Iterated Conditional Modes (ICM), which we describe below.

Iterated Conditional Modes

ICM [8] is a method that finds an approximate MAP state of a MRF. The algorithm begins with an initial node state $\hat{\mathbf{y}}$. During each iteration, for each node ICM selects the state that produces the largest decrease in energy. This process is repeated until convergence, which is guaranteed to occur, and is usually rapid in practice. The order of node updates can be arbitrary; Besag [8] proposes a raster scan

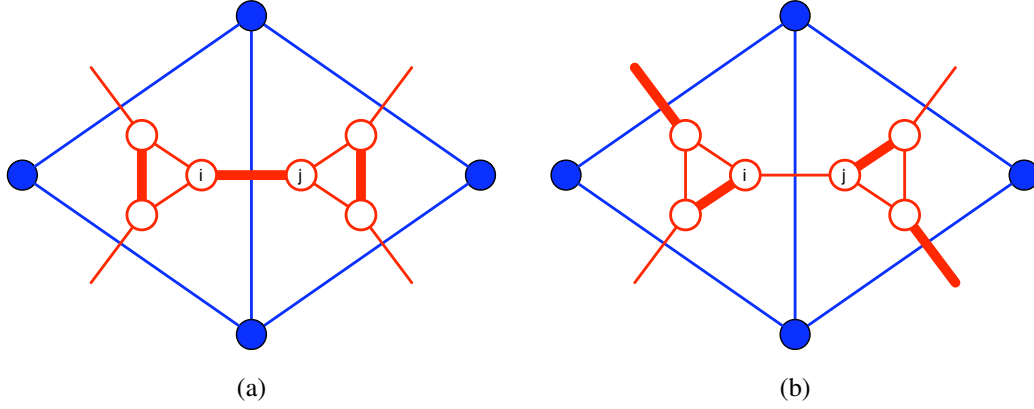


Figure 7.4: (a) Blossom's matching $\mathcal{M}$ (bold edges) of the expanded dual graph G_E^* (red). (b) a possible matching of G_E^* when the edge (i, j) is removed from $\mathcal{M}$.

whose direction is varied from one iteration to the next, thus avoiding possible directional effects. ICM is sensitive to the initial estimate $\hat{\mathbf{y}}$, especially in high-dimensional spaces where there are many local minima. Besag [8] initialises $\hat{\mathbf{y}}$ to the state which is best locally:

$$\hat{y}_i = \operatorname{argmax}_{y_i \in \mathcal{Y}} \mathbb{P}(Y_i = y_i | \mathbf{x}, \boldsymbol{\theta}). \quad (7.12)$$

We modify ICM as follows:

1. We initialize $\hat{\mathbf{y}}$ to the state given by the minimax spanning tree, which we know to be a good starting point;
2. We prioritise the order in which nodes are considered;
3. We include an implicit one-step lookahead, which allows us to make certain uphill moves.

Prioritisation

For each node $i \in \mathcal{V}$ we compute the change in energy f_i incurred by *flipping* that node's state, *i.e.*, changing it to the opposite state (we assume binary node states):

$$f_i := \sum_{(i,j) \in \mathcal{E}} E_{ij} y_{ij}^e. \quad (7.13)$$

We control the order of the node flips by using a priority queue $q = (f_{i_1}, \dots, f_{i_n})$, where f_{i_1} is at the head of the queue and node i_1 will be considered next. This queue prioritises nodes whose flipping produces the greatest decrease in energy, *i.e.*, $f_{i_k} \leq f_{i_j} \forall k < j$. The use of a priority queue ensures that promising states are explored first, so that the method is less likely to get stuck in local minima. Once node i is flipped the global energy changes by f_i and now flipping node i will incur a cost of $-f_i$. We also need to update the change in energy of its neighbouring nodes, *i.e.*, $f_k \forall (i, k) \in \mathcal{E}$; the cost of flipping all other nodes is unaffected.

Implicit One-Step Lookahead

Now suppose we make an uphill move $f_i > 0$ that leads to a larger downhill move $f_k < -f_i$, where node k is the best node to flip after flipping node i . However, if an uphill move $f_i > 0$ *does not* immediately lead to a larger downhill move, then $-f_i$ will remain at the head of the queue and we will have a never-ending 2-cycle. We detect such cycles when we see node i twice in a row, and break them by setting $f_i := \infty$ and $y_i := -y_i$. This method is equivalent to a one-step lookahead, but without the cost of checking every pair of nodes. We terminate the method when $f_{i_1} = \infty$, *i.e.*, when there is no node or pair of nodes left whose flipping decreases the energy.

7.2.4 Tree Quenching

Node quenching makes small changes to the node state, as it only considers flipping individual or pairs of nodes. We now introduce tree quenching, which is able to make large, global changes to the node state. During each iteration, tree quenching flips the state of an edge of the minimax spanning tree $T(\mathcal{V}, \mathcal{E}')$ (Section 7.2.2) that produces the greatest decrease in energy. We now show how to compute the potential change in energy f_{e_k} if the state of edge $e_k = (i, j) \in \mathcal{E}'$ is flipped. Split T into edge (i, j) and the two disjoint subtrees it connects: $T_1(\mathcal{V}_1, \mathcal{E}'_1)$ and $T_2(\mathcal{V}_2, \mathcal{E}'_2)$, as shown in Figure 7.5. To summarize:

$$\begin{aligned} i &\in \mathcal{V}_1, \quad j \in \mathcal{V}_2, \\ \mathcal{V}_1 \cup \mathcal{V}_2 &= \mathcal{V}, \quad \mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset, \\ \mathcal{E}'_1 \cup \{(i, j)\} \cup \mathcal{E}'_2 &= \mathcal{E}', \quad \mathcal{E}'_1 \cap \mathcal{E}'_2 = \emptyset. \end{aligned} \tag{7.14}$$

We want to flip the state of e_k while keeping the state of all other edges in $\mathcal{E}'$ unchanged. This corresponds to flipping the state of all the nodes of one of the subtrees. For efficiency, we flip the nodes of the smaller subtree. The edges of G whose state will change are those that connect the two subtrees, *i.e.*, the edges in the set $\mathcal{E}_f = \{(i, j) \in \mathcal{E} : i \in \mathcal{V}_1, j \in \mathcal{V}_2\}$, which is a cut of G (dashed line in Figure 7.5). The change in energy incurred by flipping the state of edge e_k is

$$f_{e_k} := \sum_{(i,j) \in \mathcal{E}_f} E_{ij} y_{ij}^e. \tag{7.15}$$

Once we have flipped the state of edges in $\mathcal{E}_f$ we must update $f_{e_k} \forall e_k \in \mathcal{E}'$. This can be done efficiently by noting that flipping the state of a non-tree edge $(i, j) \in \mathcal{E} \setminus \mathcal{E}'$ only affects f_{e_k} for edges e_k that lie on the path $\Pi(i, j) \subseteq \mathcal{E}'$, which can be precomputed.

As in node quenching we use a priority queue $q = (f_{e_1}, \dots, f_{e_n})$, where f_{e_1} is at the head of the queue and edge e_1 will be considered next. The tree quenching algorithm is shown in Algorithm 5. After every iteration of the main loop Algorithm 5 needs to recompute the cost of many edges in T , making it considerably slower than node quenching. Hence we propose two versions of tree quenching, which differ only in their termination criterion:

1. Fast: terminate when there is no edge whose flipping decreases the energy, *i.e.*, when $f_{e_1} > 0$.

2. Slow: terminate when there are no edge or pair of edges whose flipping decreases the energy, *i.e.*, when $f_{e_1} = \infty$.

Note that slow tree quenching can be up to a magnitude slower than fast tree quenching (see right column in Figure 7.9).

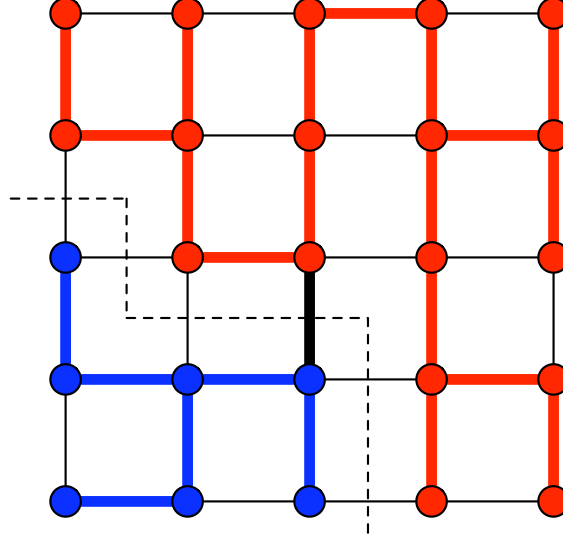


Figure 7.5: 5×5 model grid graph G with edges of the minimax spanning tree T shown in bold. The two subtrees T_1 and T_2 and the edge to be flipped are shown in blue, red and black respectively. The cut of G (dashed line) crosses the edges whose state will change if the state of the bold black edge is flipped.

Algorithm 5 Tree Quenching

Input: model graph $G(\mathcal{V}, \mathcal{E})$, minimax spanning tree $T(\mathcal{V}, \mathcal{E}')$

1. $\forall (i, j) \in \mathcal{E} \setminus \mathcal{E}'$:
 Compute and store the path $\Pi(i, j) \subseteq \mathcal{E}'$
 $\forall k \in \Pi(i, j)$: (initialize edge priorities in q)
 $f_{e_k} := f_{e_k} + E_{ij} y_{ij}^e$
 2. Main Loop:
 $(i, j) := q.\text{pop}()$
 Divide T into T_1 and T_2 as shown in (7.14)
 Compute the cut $\mathcal{E}_f = \{(i, j) \in \mathcal{E} : i \in \mathcal{V}_1, j \in \mathcal{V}_2\}$
 $\forall (p, q) \in \mathcal{E}_f$:
 $\forall k \in \Pi(p, q)$: (update edge priorities in q)
 $f_{e_k} := f_{e_k} - 2E_{pq} y_{pq}^e$
 $y_{pq}^e := -y_{pq}^e$ (flip the state of edges along the cut)
-

7.3 Experiments

In Section 7.2 we proposed a number of heuristics for obtaining ground states from the extended ground state of a non-planar Ising graphical model. We described a method for obtaining a low-genus embedding (Section 7.2.1) and discussed methods for constructing a consistent ground state using a minimax spanning tree (Section 7.2.2). We also suggested algorithms for finding a lower-energy state from a given ground state (Sections 7.2.3 and 7.2.4). To measure the effect of each heuristic, we designed inference algorithms with increasing levels of prediction accuracy:

- IS0: Compute the extended ground state using Blossom V. Approximate the ground state using the minimax spanning tree with disagreement edge costs (Section 7.2.2).
- IS: Compute the extended ground state using Blossom V. Approximate the ground state using the minimax spanning tree with blossom’s slacks (Section 7.2.2).
- IS0-N: Run IS0 and approximate the ground state using node quenching (Section 7.2.3).
- IS-N: Run IS and approximate the ground state using node quenching (Section 7.2.3).
- IS-FN: Run IS and approximate the ground state using fast tree quenching followed by node quenching (Sections 7.2.3 and 7.2.4).
- IS-SN: Run IS and approximate the ground state using slow tree quenching followed by node quenching (Sections 7.2.3 and 7.2.4).

We compared our inference algorithms to a variety of control algorithms, that are considered state-of-the-art in MAP approximation:

- Iterated Conditional Modes (ICM) (see Section 7.2.3 and [8]). Implementation from [121].
- Primal-dual algorithm FastPD [71, 72].
- Sequential Tree Reweighting (TRW-S) (see Section 4.3.6 and [66]). Implementation from [121].
- Sequential max-product loopy belief propagation (BP-S) [66]. Implementation from [121].
- Max-product loopy belief propagation (BP-M) [122]. Implementation from [121].

In our experiments we found that FastPD does not converge to low energies. We believe that in our case of random potentials, the LP relaxation used by FastPD is a poor approximation to the true energy. We also found that ICM converges to poor solutions. Therefore, for the sake of clarity we omit the results of FastPD and ICM.

7.3.1 Experimental Setup

We tested the algorithms on undirected binary graphical models with the energy function (5.3) defined on a regular grid. As shown in Theorem 33, these models correspond to Ising graphical models defined on the same grid, with every node connected to an additional bias node $y_0 := 0$. We use the embedding method described in Section 7.2.1. As identified in the literature [51, 66, 105, 121], the performance of the control algorithms depends on the following factors:

1. Number of cycles in the model graph.
2. Percentage of frustrated cycles (Definition 25) in the model graph.
3. Strength of unary versus pairwise potentials.

Computing the number of cycles in a graph is a difficult problem. For grids, however, the size of the grid is a good measure for the number of cycles (Figure 7.6(a)). We can also use the size of the grid to determine the genus of the embedding, which our algorithms depend upon. Similarly it is difficult to compute the percentage of frustrated cycles in a graph. In grids the percentage of frustrated cycles is dependent on the percentage of non-submodular edges (Figure 7.6(b)). Note that when the percentage of non-submodular edges is 100% there are no frustrated cycles, since all cycles in the regular grid have an even length. With the knowledge of these dependencies, we vary three parameters in our experiments: the size of the grid N , the percentage of non-submodular edges α , and the strength of the unary potentials E'_i relative to that of the pairwise potentials E'_{ij} . The latter is controlled by the variable σ and can be interpreted as a type of signal-to-noise ratio (SNR). For each parameter set, we generate 100 grids, following the construction of Kolmogorov and Wainwright [69]:

- Sample node potentials from the normal distribution: $E'_i(0) \sim \mathcal{N}(0, 1)$ and $E'_i(1) \sim \mathcal{N}(0, 1)$.
- Set edge potential to $E'_{ij}(0, 0) = E'_{ij}(1, 1) := 0$ and $E'_{ij}(0, 1) = E'_{ij}(1, 0) := \lambda_{ij}$, where the random variable $\lambda_{ij} \sim -|\mathcal{N}(0, \sigma^2)|$ with probability α and $\lambda_{ij} \sim |\mathcal{N}(0, \sigma^2)|$ with probability $(1 - \alpha)$.

All experiments were carried out on a Linux PC with 3 GB RAM and dual Intel Pentium 4 processors running at 3.6 GHz, each with 2 MB of level 2 cache.

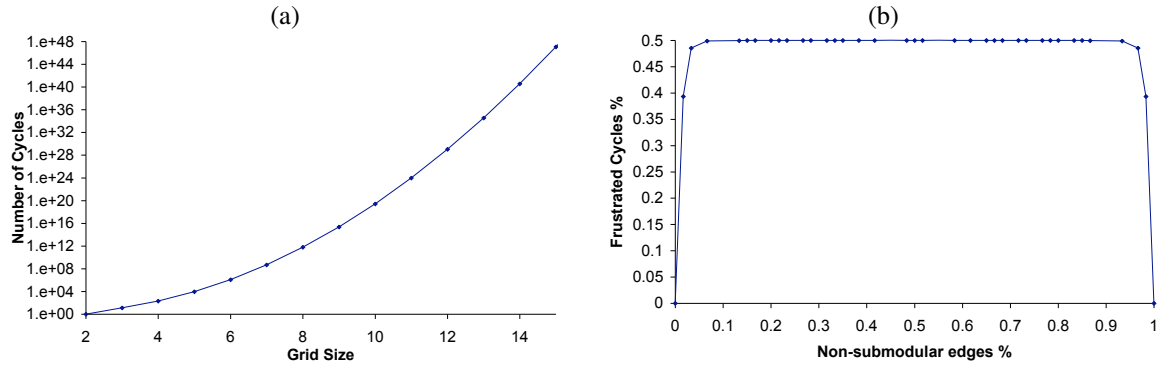


Figure 7.6: (a) Number of cycles in a regular grid as a function of its size (based on A140517 in [118]). (b) Percentage of frustrated cycles in a regular grid as a function of the percentage of non-submodular edges.

7.3.2 Comparison with the Ground State

We first compared different algorithms relative to the true ground state. This enabled us to measure the node error and the energy achieved by each algorithm relative to the minimal energy. Since the computation of ground states in our large 3-dimensional parameter space is rather time consuming, we

decided to explore only certain slices of our parameter space. In each such slice we varied one of the three parameters, while keeping the other two parameters fixed:

- Fix $\sigma = 1.0$, $\alpha = 0.5$ and vary the grid size $N \in \{4, 8, 16, 24, 32, 48, 64\}$, which corresponds to the embedding genus of $\{1, 9, 49, 121, 225, 529, 961\}$.
- Fix $\sigma = 1.0$, $N = 32$ and vary $\alpha \in \{0.01, 0.1\} \times \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.
- Fix $\alpha = 0.5$, $N = 32$ and vary $\sigma \in \{0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5\}$.

The relative energy was computed by comparing to the energy of the ground state, *i.e.*, the minimal energy. Let E^t be the energy of the true ground state in trial t and E_i^t be the energy achieved by algorithm i in trial t . The relative energy for algorithm i is computed as

$$E_i = \frac{1}{T} \sum_{t=1}^T \frac{E^t}{E_i^t}, \quad (7.16)$$

where T is the total number of trials. In the above we assume that E^t and E_i^t have the same sign, which is the case in our experiments.

Energy vs Distance to the Ground State

We also looked at the Hamming distance of the approximate ground states obtained by the various methods to the true ground state, normalized by the total number of nodes. Interestingly, we found that while our heuristics tended to find good local minima farther from the true ground state, the control methods preferred inferior local minima closer to the true ground state (see Figure 7.7). This may reflect different biases in these approximate optimizers, for instance in their disposition towards making small vs. large moves in state space. The effect is pronounced here because we are optimizing over random energy landscapes, which are very non-smooth.

In a machine learning application the edge potentials would reflect some underlying ground truth — for instance, they might be derived from training data by maximum-margin parameter estimation (Section 6.1.2). Unless the model is poorly chosen, good solutions (in terms of energy) would then necessarily lie close (in state space) to the ground truth, and points close to the ground truth would necessarily be good solutions. Under those circumstances distance to the optimum may be a useful stand-in for the criterion that is actually being optimized, the energy. Here, however, there is no ground truth, and in a random energy landscape these two quantities are effectively decoupled.

Minimax Spanning Tree

To determine whether it is best to use blossom’s slacks for the minimax spanning tree (Section 7.2.2) we compared IS0, IS, IS0-N and IS-N in terms of the achieved energy, as shown in Figure 7.8. In terms of the CPU running time all four algorithms were very similar, so we do not report it. Without any quenching there is no significant difference between IS and IS0. However, once node quenching is used IS-N overtakes IS0-N, showing that the minimax spanning tree is more powerful with blossom’s slacks. Given this, we choose to use blossom’s slack variables for all further experiments. We also notice that

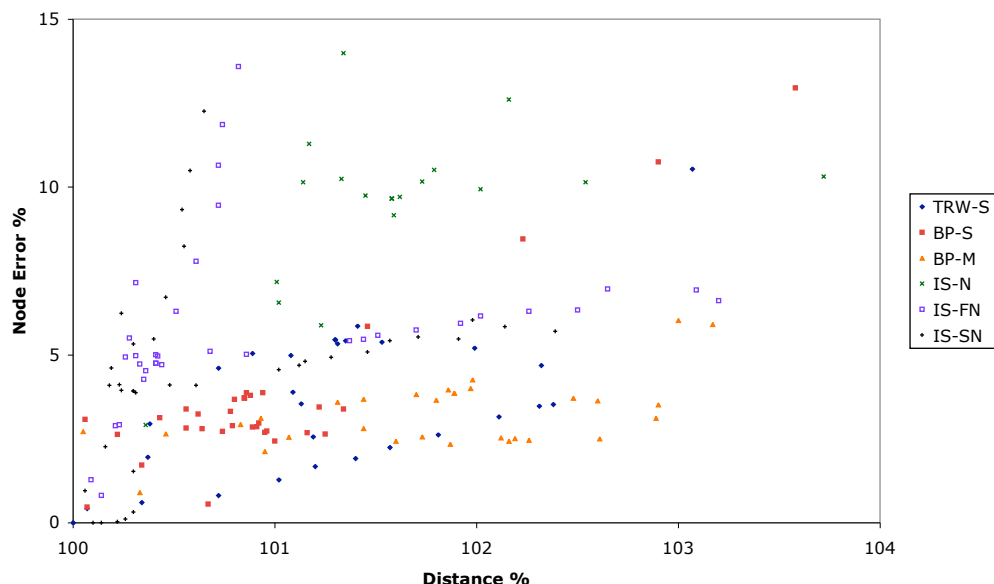


Figure 7.7: A scatter plot of distance vs. energy, relative to the true ground state, for the approximate ground states obtained by a range of methods and parameter settings.

IS-N is significantly better than IS. This shows that node quenching is quite beneficial. Hence, we also omit IS from all further experiments.

Quenching: Order and Iterations

We have two types of quenching (node and tree) at our disposal, so how many times and in what order should we use them? To answer the latter, we begin by noticing that it does not make sense to run two quenchantings of the same type in a row. This is because the second quenching will not have any available moves. Thus the only remaining option is to alternate node quenching with tree quenching. We verified that it is never beneficial to perform node quenching before tree quenching. This makes sense, because one should always make large moves, as those of tree quenching, before applying small local optimisations.

The next question we wanted to answer is whether running more quenching iterations is necessary. Tree quenching followed by node quenching, followed by another tree quenching was only marginally better than IS-SN in terms of the achieved energy, while being significantly slower. Similarly, we do not expect that further quenching iterations can significantly decrease the energy, since most of the work is already done during previous quenching iterations.

Tree Quenching

In our next set of experiments we analyzed the effects of tree quenching and compared its two running modes. To answer the former, we measured the achieved reduction in energy when tree quenching was used on top of node quenching (Figure 7.9). For all parameter settings we found that the reduction in energy was significant. In particular, the amount of reduction increases with σ and the size of the

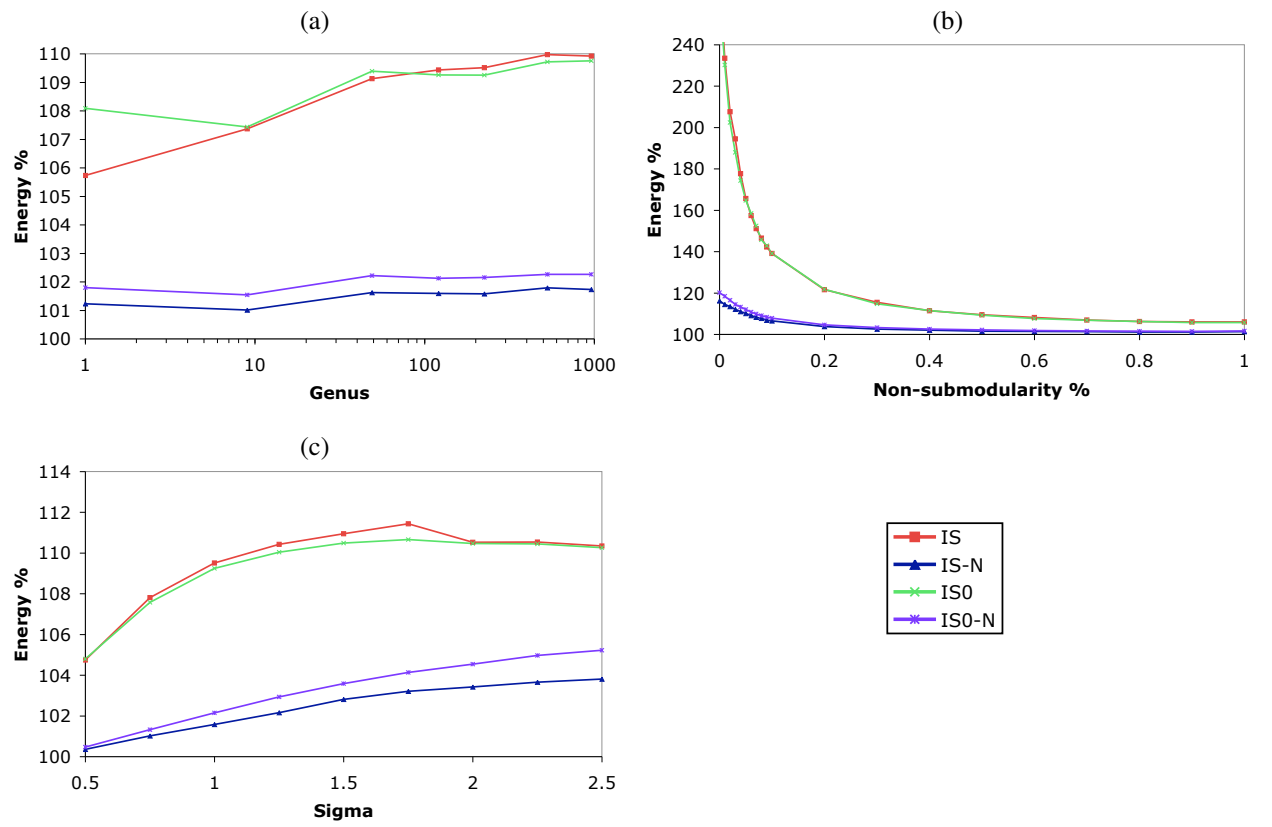


Figure 7.8: Comparison of IS0, IS, IS0-N and IS-N. y-axis shows the energy as a function of (a) genus of the embedding surface, (b) percentage of non-submodular edges and (c) sigma.

grid (Figures 7.9(a) and 7.9(e)). Given the significant improvement in energy, the extra running time incurred by tree quenching is a small price to pay. Hence we can confidently recommend the use of tree quenching.

We also wanted to determine whether slow tree quenching is worth the additional running time (Figure 7.9). As expected, IS-SN is better than IS-FN in terms of the achieved energy in all regimes. Furthermore, the margin of improvement is nearly constant per experiment. The biggest improvement is obtained when we vary α , in particular when $\alpha < 0.2$. The ratio between the running times of the two methods remains constant for a fixed grid size (Figures 7.9(d) and 7.9(f)). As we vary the grid size the ratio varies between 2 and 6 for small and large grids, respectively (Figures 7.9(a) and 7.9(b)). Given these results, one should prefer IS-SN over IS-FN for smaller grids (*e.g.*, $N \leq 32$), where the running time is not an issue. For larger grids, the lower energy achieved by IS-SN is overshadowed by its slower running time, and hence in such situations one might consider using IS-FN.

Comparison to Control Methods

After removing all our inferior algorithms we were left with just two Ising algorithms: IS-FN, IS-SN. We decided to compare these to the control algorithms (TRW-S, BP-S and BP-M) in terms of the achieved energy and CPU running time, as shown in Figure 7.9.

As discussed in Section 7.2.1, the performance of our algorithms should decrease as the genus increases. However, our experiments did not confirm this, since IS-FN and IS-SN outperformed the control algorithms for all genera (Figure 7.9(a)). One explanation is that all algorithms were affected by the increased genus, therefore their rank remained relatively unchanged. In the range $0.1 < \alpha < 0.9$ our algorithms were better than the control algorithms in terms of the achieved energy (Figure 7.9(c)). This shows that the control algorithms are best suited for submodular or nearly-submodular problems, and confirms the results of previous studies [121]. We also found that the control algorithms become significantly worse for larger σ (Figure 7.9(e)), while the performance of our algorithms is unaffected.

The running time of all algorithms increased with the genus, although it increased at a higher rate for our algorithms. Nevertheless, our algorithms were faster than the control algorithms for genus 121 and lower. Unlike our algorithms, the running time of the control algorithms was also affected by the percentage of non-submodularity (Figure 7.9(d)). We wanted to see what would happen if the control algorithms were given the same CPU time as our methods. We increased the running time of the control algorithms by a factor of 10 by changing their default configuration (see MRF-benchmarks code [121]) from 500 outer and 1 inner iterations to 1000 outer and 5 inner iterations, respectively. However this did not significantly improve their performance in terms of the achieved energy, and often made it worse.

7.3.3 Overall Comparison

For our final set of experiments we wanted to answer the following important question: under what set of conditions do our algorithms perform better *resp.* worse than the control algorithms? To answer this question we compared our best algorithm (IS-SN) to the control algorithms TRW-S, BP-S and BP-M in a large parameter space. This was feasible, because we avoided the cost of computing the ground states. We explored all possible sets of parameters:

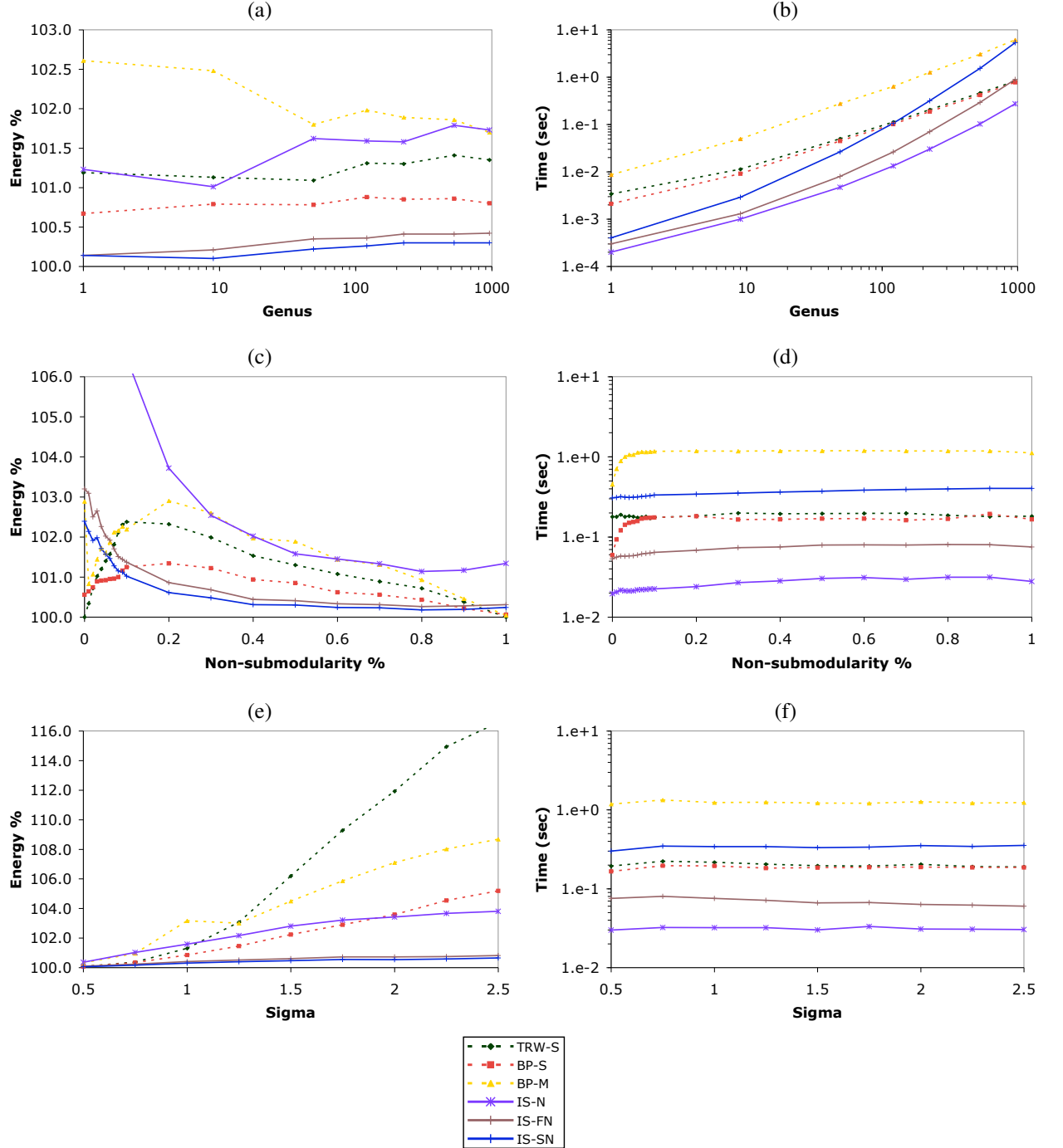


Figure 7.9: Comparison of IS-N, IS-FN and IS-SN to the three control algorithms. y-axis is a function of the genus in row 1, percentage of non-submodular edges in row 2 and sigma in row 3. Left column: y-axis is the achieved energy. Right column: y-axis is the CPU running time in seconds.

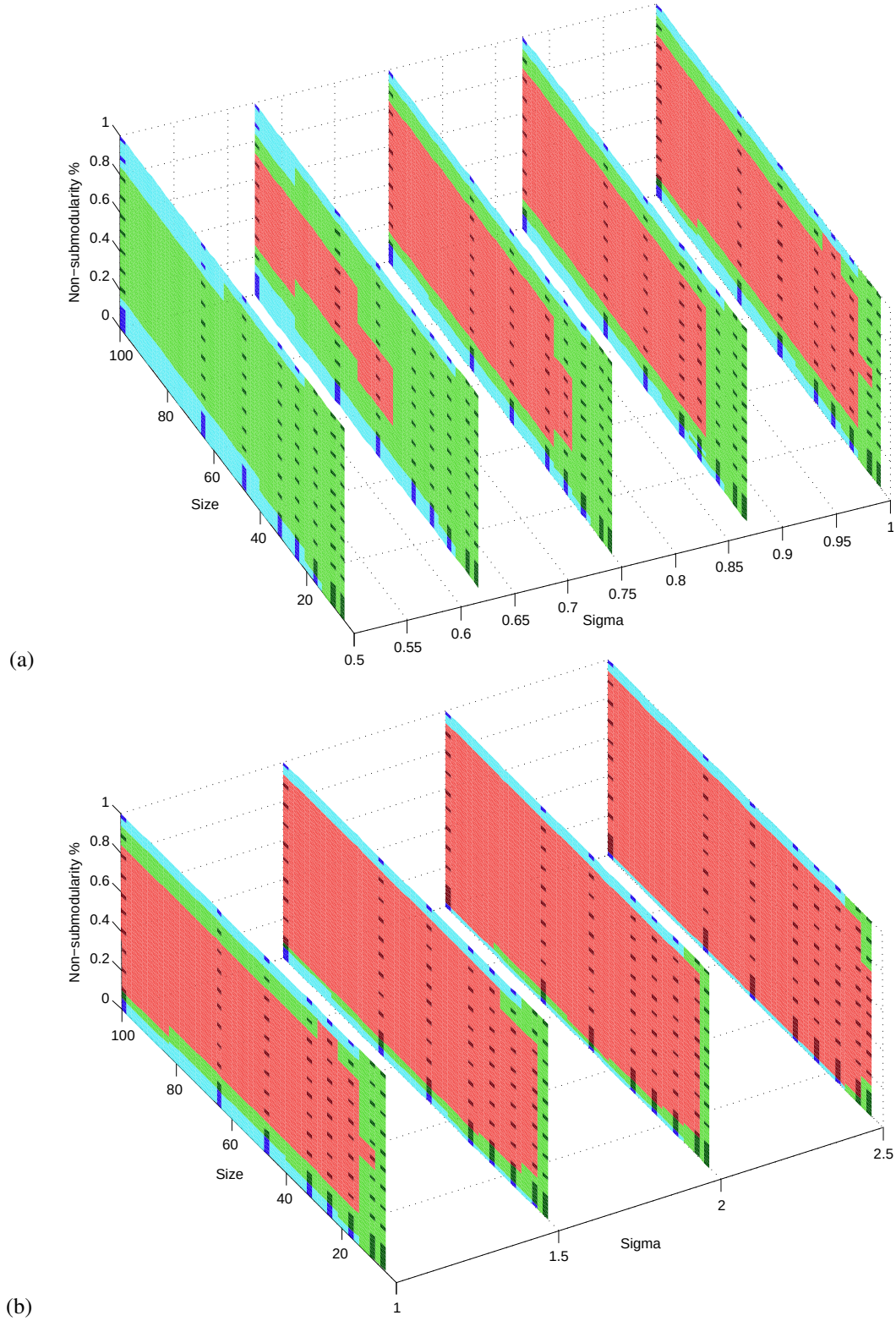


Figure 7.10: Comparison of IS-SN to the three control algorithms. For every combination of parameters we compute at 95% confidence level whether IS-SN is significantly better (dark red), significantly worse (dark blue) or equal (dark green) to the best control algorithm. We use these known data points to extrapolate the result of the untested parameters settings, *i.e.*, the remaining area (shown with corresponding lighter colors). (a) $\sigma \in \{0.5, 0.625, 0.75, 0.875, 1.0\}$. (b) $\sigma \in \{1.0, 1.5, 2.0, 2.5\}$.

k	2	3	4	5	6	7	8	9	10
q_k	1.960	2.241	2.394	2.498	2.576	2.638	2.690	2.724	2.773

Table 7.1: Critical values q_k for the two-tailed Bonferroni-Dunn test at the 95% confidence level. k is the number of classifiers and includes the control classifier [20].

- Grid size $N \in \{4, 8, 16, 24, 32, 48, 64, 100\}$.
- $\sigma \in \{0.5, 0.625, 0.75, 0.875, 1.0, 1.5, 2.0, 2.5\}$.
- Non-submodularity $\alpha \in \{0.0, 0.02, 0.04, 0.06, 0.08, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$.

It was not feasible to plot the results of each experiment, since that would require a 4-dimensional graph¹². Instead, we decided to compare all the inference algorithms based on their average ranking, and plot only the best ranked algorithms on a 3-dimensional graph. The rank of algorithm i for trial t is computed as

$$R_i^t := 1 + |\{k : E_k^t < E_i^t\}| + \frac{1}{2} |\{k : E_k^t = E_i^t, k \neq i\}|, \quad (7.17)$$

where E_i^t is the energy achieved by algorithm i in trial t . The average rank of algorithm i is then $R_i := \sum_t E_i^t / T$, where T is the total number of trials. Since we were comparing multiple algorithms, rather than just two, we chose to use the Bonferroni-Dunn test over any variant of the paired t-test [20, 22]. In particular, two algorithms i and j were considered significantly different at a 95% confidence confidence level if

$$|R_i - R_j| \geq q_k \sqrt{\frac{k(k+1)}{6T}}, \quad (7.18)$$

where k is the number of algorithms and q_k is the critical value given in Table 7.1.

We plotted the regions where IS-SN was significantly better, worse or equal (shown in red, blue and green respectively) to the best of the control algorithms for $\sigma \in \{0.5, 0.625, 0.75, 0.875, 1.0\}$ in Figure 7.10(a) and for $\sigma \in \{1.0, 1.5, 2.0, 2.5\}$ in Figure 7.10(b). In these figures we set the value of the untested region to the value of the closest (in Euclidean space) tested region.

In general, the region where IS-SN is better becomes larger as σ increases. When the node potentials dominate (*i.e.*, at $\sigma = 0.5$), our algorithm does not outperform the control algorithms. The situation changes rapidly, however, as soon as the strength of the edge potentials increases relative to the node potentials. Already at $\sigma = 0.75$, IS-SN is best in more than half of the available space, while for $\sigma > 1.5$ it is best nearly everywhere. Interestingly, at $\sigma = 2.5$ IS-SN outperformed the control methods even on grids as small as 4×4 . We expect these trends to continue for larger values of σ .

As we have observed previously (see Section 7.3.2), the relative performance of IS-SN does not get worse with the increase in the grid's size. The most likely explanation is that the size of the grid affects all methods equally, meaning that their ranking relative to each other remains constant. This demonstrates the potential of our approach to scale well to larger problems.

The only parameter setting where IS-SN was significantly worse than the control methods occurred when the model had a low percentage of frustrated cycles, *i.e.*, when $\alpha \leq 0.1$ and $\alpha \geq 0.9$. This should

¹²we need one axis for each of the three parameters, as well as another axis to display the value of the relative energy.

come at no surprise, since the control methods are specifically designed to work for such submodular models. Notice that for all other parameter settings, IS-SN was equal or better than the control algorithms.

In conclusion, we have found a large region in the parameter space where one should prefer IS-SN over any other state-of-the-art algorithm. In particular, one should use our IS-SN for any binary 4-connected MRF whose edge potentials are either

- strongly non-submodular: $0.2 \leq \alpha \leq 0.8$; or
- slightly non-submodular and stronger than the node potentials: $0.1 \leq \alpha \leq 0.9$ and $\sigma \geq 1.5$.

7.4 Conclusion

In this chapter we have proposed a number of heuristics that can be used for approximating the ground state from the extended ground state of an Ising graphical model. These included: obtaining a low-genus embedding, computing a minimax spanning tree with consistent edge states and methods for minimizing the energy of a node state (quenching). We have shown experimentally that all these heuristics benefit our framework. Some of these heuristics are specific to our framework, while others, such as quenching can be easily combined with any other inference framework. For example, one can compute an approximate ground state with any given inference algorithm and then optimise the result with a combination of node and tree quenching. This will be the focus of our future work.

We also compared our inference algorithms to state-of-the-art algorithms on random grid MRFs. Our algorithms were able to achieve lower-energy states on a range of parameter settings, without being significantly slower. We believe that there exist a range of non-trivial problems that are suitable for our algorithms. One such problem is viral marketing in social networks [141]. In viral marketing the task is to market a product to the “best” set of individuals in order to trigger the most widespread adoption of the product. This task has been successfully modelled as a MRF, whose nodes represent customers and edges correspond to relationships between customers [21, 103]. The network value of a customer is the expected profit from sales to that customer. Importantly, this value not only depends on the customer’s probability of buying the product, but also on his/her ability to persuade his friends, family and colleagues in buying the product. Hence if a customer is highly connected or is a well-know opinion leader (*e.g.*, celebrity) then his/her network value is dominated by his/her ability to persuade others. In terms of the underlying MRF, this corresponds to large edge potentials, *i.e.*, $\sigma \geq 1.5$ in our model. Therefore, we believe that our inference algorithms would be ideally suitable for such a task, especially if the underlying graph was large, *e.g.*, Facebook, MySpace or LinkedIn.

A number of non-submodular computer vision applications have been proposed recently: texture restoration, image stitching, diagram recognition, super resolution, new view synthesis and image deconvolution [68, 105]. In all of these applications, however, either the node potentials are too strong or there is not enough non-submodularity, making them unsuitable for our algorithms. We believe that current computer vision problems are designed to suit current available inference algorithms and vice-versa. This selection bias impedes progress in both applications and in the development of inference algorithms for non-submodular problems. The presence of our framework, that does not rely on submodularity, should help to dissolve this bias and open doors for new applications in computer vision and other fields.

Conclusion

This thesis introduced the Ising graphical model. This is an undirected graphical model that obeys the constraints of the Ising model, *i.e.*, a binary-labeled energy function that is a sum of edge disagreement costs. Despite its simplicity, we proved that the Ising graphical model with an additional bias node is equivalent to any binary-labeled pairwise MRF (Theorem 33).

Our work can be described in two parts. In the first part (Chapters 5 and 6) we discussed exact inference algorithms for the planar Ising model, while in the second we looked at approximative algorithms for ground states of non-planar Ising models.

In contrast to previous approaches, our construction uses the complementary mapping (Section 3.2.2) between even subgraphs in G_0 and perfect matchings in the decorated graph G_d , where G_0 is the dual graph G^* with unmodified edge weights. This gives us a complementary connection between graph cuts of the model graph G and perfect matchings in G_d . We explain the role of planarity (*resp.* triangulation) in making this relation a surjection (*resp.* bijection). Our method for decorating the graph is similar to Kasteleyn’s (Definition 24), but does not need dummy nodes since we plane triangulate G . Our construction is conceptually simpler than previous constructions, since we do not explicitly construct the dual graph and perform all operations in the model graph G .

We show how our framework can be used for the exact computation of the partition function, worst margin violators, marginal edge probabilities and lowest-energy (ground) states in the planar Ising graphical model. This allows us to implement popular parameter estimation frameworks of maximum-margin and maximum-likelihood. By reusing the proto-Kasteleyn matrix $\mathbf{H}$ (Section 5.3.3) we speed up the parameter estimation loop by orders of magnitude. We demonstrated that our inference algorithms are efficient and effective on a number of real-world machine learning problems; in particular, we found that maximum margin is orders of magnitude faster than penalized maximum likelihood, as well as converging to superior parameters.

In the second part, we proposed a number of heuristic methods for approximating the ground state of a non-planar Ising graphical model. These novel contributions included: obtaining a low-genus embedding, computing a minimax spanning tree with consistent edge states and methods for minimizing the energy of a node state (quenching). We conducted a thorough experimental comparison to show that our framework benefits from every heuristic. When combined with our best heuristics, our inference algorithm outperforms current state-of-the-art algorithms by achieving lower-energy states on a range of parameter settings. Finally, we characterise the set of MRF models for which our inference algorithms are preferable over other algorithms.

8.1 Future Work

There are a number of extensions for this work. These can be divided into two categories: new applications and algorithmic developments. Some possible new applications are the following:

- The Ising graphical model can also implement α -expansion moves and α - β -swaps [12]. This will allow us to deal with multi-labeled models and give rise to a number of new applications.
- Our exact inference algorithms replace the submodularity constraint of graph cut methods with a planarity constraint. The existence of two distinct tractable frameworks for inference in binary graphical models implies a powerful hybrid. Such a hybrid could be used when the model graph has biconnected components that are either planar or submodular. It would be interesting to see whether such a hybrid approach can be used for less obvious situations.
- As we discussed in Chapter 7, our algorithms for non-planar Ising graphical models are well-suited for the task of viral marketing [141]. This is because such tasks can be modeled as a binary pairwise MRF with large edge potentials, *i.e.*, $\sigma \geq 1.5$ in our model (Section 7.3.1). These tasks require efficient inference algorithms for large graphs (social networks).

Some possible algorithmic developments are the following:

- There may still be other constructions for computing the partition function and ground states of an Ising model. For example, to our knowledge nobody has tried to use the complementary mapping (Section 3.2.2) where G_0 is the model graph G .
- Our quenching methods (Sections 7.2.3 and 7.2.4) can be used to improve the node state computed by any inference algorithm. This opens up many possibilities for new algorithms, which may turn out to be better than current algorithms on particular types of models.
- It is possible to use our framework to efficiently compute the exact min-marginals [4], instead of approximate min-marginals that we are currently computing (Section 7.2.2). These can be used to construct a better minimax spanning tree, which may lead to stronger performance of our algorithms.

Bibliography

- [1] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. van der Vorst. *Templates for the solution of Algebraic Eigenvalue Problems: A Practical Guide*. Society for Industrial and Applied Mathematics, Philadelphia, USA, 2000.
- [2] F. Barahona. On the computational complexity of Ising spin glass models. *Journal of Physics A: Mathematical, Nuclear and General*, 15(10):3241–3253, 1982.
- [3] F. Barahona, R. Maynard, R. Rammal, and J. P. Uhry. Morphology of ground state of two-dimensional frustration model. *Journal of Physics A: Mathematical and General*, 15:673–699, 1982.
- [4] Dhruv Batra and Tsuhan Chen. Dynamic planar-cuts: Efficient computation of min-marginals for outer-planar models. *NIPS Workshop on Discrete Optimization in Machine Learning*, 2009. http://www.ece.cmu.edu/~dbatra/publications/assets/batra_discml_nips09.pdf.
- [5] Peter Benner, Ralph Byers, Heike Fassbender, Volker Mehrmann, and David Watkins. Cholesky-like factorizations of skew-symmetric matrices. *Electronic Transactions on Numerical Analysis*, 11:85–93, 2000.
- [6] A. Bertoni, P. Campadelli, G. Gangai, and R. Posenato. Approximability of the ground state problem for Ising spin glasses. *Journal of Complexity*, 13:326–339, 1997.
- [7] Julian Besag. Spatial interaction and the statistical analysis of lattice systems (with discussion). *Journal of the Royal Statistical Society. Series B*, 36(2):192–236, 1974.
- [8] Julian Besag. On the statistical analysis of dirty pictures. *Journal of the Royal Statistical Society B*, 48(3):259–302, 1986.
- [9] I. Bieche, R. Maynard, R. Rammal, and J. P. Uhry. On the ground states of the frustration model of a spin glass by a matching method of graph theory. *Journal of Physics A: Mathematical and General*, 13:2553–2576, 1980.
- [10] S. Boettcher and A. G. Percus. Optimization with extremal dynamics. *Physical Review Letters*, 86:5211–5214, 2001.
- [11] John M. Boyer and Wendy J. Myrvold. On the cutting edge: Simplified $O(n)$ planarity by edge addition. *Journal of Graph Algorithms and Applications*, 8(3):241–273, 2004. Reference implementation (C source code): <http://jgaa.info/accepted/2004/BoyerMyrvold2004.8.3/planarity.zip>.

- [12] Yuri Boykov, Olga Veksler, and Ramin Zabih. Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(11):1222–1239, 2001.
- [13] Stephen G. Brush. History of the Lenz-Ising model. *Reviews of modern physics*, 39:883–893, 1967.
- [14] James R. Bunch. A note on the stable decomposition of skew-symmetric matrices. *Mathematics of Computation*, 38(158):475–479, April 1982.
- [15] Arthur Cayley. Sur les déterminants gauches. *Journal für die reine und angewandte Mathematik*, 38:93–96, 1849.
- [16] Chandra Chekuri, Sanjeev Khanna, Joseph (Seffi) Naor, and Leonid Zosin. Approximation algorithms for the metric labeling problem via a new linear programming formulation. In *SODA '01: Proceedings of the twelfth annual ACM-SIAM symposium on Discrete algorithms*, pages 109–118, Philadelphia, PA, USA, 2001. Society for Industrial and Applied Mathematics.
- [17] William Cook and André Rohe. Computing minimum-weight perfect matchings. *INFORMS Journal on Computing*, 11(2):138–148, 1999. C source code available at <http://www.isye.gatech.edu/~wcook/blossom4/>.
- [18] Marcel Crasmaru and John Tromp. Ladders are PSPACE-complete. In *Computers and Games*, pages 241–249, 2000.
- [19] James W. Demmel, Stanley C. Eisenstat, John R. Gilbert, Xiaoye S. Li, and Joseph W. H. Liu. A supernodal approach to sparse partial pivoting. *SIAM Journal on Matrix Analysis and Applications*, 20(3):720–755, 1999. C source code available from <http://crd.lbl.gov/~xiaoye/SuperLU/>.
- [20] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [21] Pedro Domingos and Matthew Richardson. Mining the network value of customers. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 57–66, New York, NY, USA, 2001. ACM.
- [22] Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, March 1961.
- [23] Jack Edmonds. Maximum matching and a polyhedron with 0,1-vertices. *Journal of Research of the National Bureau of Standards*, 69B:125–130, 1965.
- [24] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [25] S. F. Edwards and P. W. Anderson. Theory of spin glasses. *Journal of physics F: metal physics*, 5:965–974, 1975.
- [26] Markus Enzenberger. Evaluation in Go by a neural network using soft segmentation. In *Advances in Computer Games*, pages 97–108, 2003.

-
- [27] Markus Enzenberger. The integration of a priori knowledge into a Go playing neural network. Technical report, University of Munich, September 1996.
- [28] Jeff Erickson and Kim Whittlesey. Greedy optimal homotopy and homology generators. In *Proc. 16th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1038–1046, Vancouver, BC, 2005. SIAM. ISBN 0-89871-585-7.
- [29] Richard P. Feynman. *Statistical Mechanics. A Set Of Lectures*. Addison-Wesley, Reading, MA, 1972.
- [30] Michael E. Fisher. Statistical mechanics of dimers on a plane lattice. *Physical Review*, 124(6): 1664–1672, 1961.
- [31] Michael E. Fisher. On the dimer solution of planar Ising models. *Journal of Mathematical Physics*, 7(10):1776–1781, 1966.
- [32] L. R. Ford and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, Princeton, NJ, 1962.
- [33] C. Fox and G. K. Nicholls. Exact map states and expectations from perfect sampling: Greig, Porteous and Seheult revisited. In A. Mohammad-Djafari, editor, *Bayesian Inference and Maximum Entropy Methods in Science and Engineering*, volume 568 of *American Institute of Physics Conference Series*, pages 252–263, 2001.
- [34] Brendan J. Frey and David J. C. MacKay. A revolution: Belief propagation in graphs with cycles. In M. I. Jordan, M. J. Kearns, and S. A. Solla, editors, *Advances in Neural Information Processing Systems 10*, volume 10, pages 479–485. MIT Press, 1998.
- [35] K. J. Friedenbach, Jr. *Abstraction hierarchies: a model of perception and cognition in the game of Go*. PhD thesis, University of California, Santa Cruz, 1980.
- [36] Anna Galluccio and Martin Loeb. On the theory of Pfaffian orientations. i. perfect matchings and permanents. *Journal of Combinatorics*, 6:81–100, 1999.
- [37] Anna Galluccio, Martin Loeb, and Jan Vondrák. New algorithm for the Ising problem: Partition function for finite lattice graphs. *Physical Review Letters*, 84(26):5924–5927, Jun 2000.
- [38] Amir Globerson and Tommi Jaakkola. Approximate inference using planar graph decomposition. In B. Schölkopf, J. Platt, and T. Hofmann, editors, *Advances in Neural Information Processing Systems 19*, pages 473–480, Cambridge, MA, 2007. MIT Press.
- [39] GNU Go 3.6. <http://www.gnu.org/software/gnugo>, 2004.
- [40] Thore Graepel, Mike Goutrié, Marco Krüger, and Ralf Herbrich. Learning on graphs in the game of Go. In *International Conference on Artificial Neural Networks*, pages 347–352, London, UK, 2001. Springer-Verlag. ISBN 3-540-42486-5.
- [41] D. M. Greig, B. T. Porteous, and A. H. Seheult. Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society B*, 51(2):271–279, 1989.

- [42] Carsten Gutwenger and Petra Mutzel. Graph embedding with minimum depth and maximum external face. In G. Liotta, editor, *Graph Drawing 2003*, volume 2912 of *Lecture Notes in Computer Science*, pages 259–272. Springer Verlag, 2004.
- [43] P. L. Hammer, P. Hansen, and B. Simeone. Roof duality, complementation and persistency in quadratic 0-1 optimization. *Mathematical Programming*, 28(2):121–155, February 1984.
- [44] J. M. Hammersley and P. E. Clifford. Markov fields on finite graphs and lattices. unpublished manuscript, 1971.
- [45] F. Harary. Feynman’s simplification of the Kac-Ward treatment of the two-dimensional Ising model, 1958. In the draft chapter of Harary’s book on graph theory.
- [46] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2002.
- [47] John E. Hopcroft and Robert E. Tarjan. Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.
- [48] J. Houdayer and O. C. Martin. Renormalization for discrete optimization. *Physical Review Letters*, 83(5):1030–1033, Aug 1999.
- [49] Koji Hukushima and Koji Nemoto. Exchange Monte Carlo method and application to spin glass simulations. *Journal of the Physical Society of Japan*, 65, 1996.
- [50] Charles A. Hurst and Herbert S. Green. New solution of the Ising problem for a rectangular lattice. *Journal of chemical physics*, 33:1059–1062, 1960.
- [51] Alexander T. Ihler, John W. Fischer III, and Alan S. Willsky. Loopy belief propagation: Convergence and effects of message errors. *J. Mach. Learn. Res.*, 6:905–936, 2005.
- [52] Ernst Ising. *Beitrag zur Theorie des Ferro- und Paramagnetismus*. PhD thesis, Hamburg University, 1924.
- [53] Ernst Ising. Beitrag zur Theorie des Ferromagnetismus. *Zeitschrift für Physik*, 31:253–258, February 1925.
- [54] Sorin Istrail. Statistical mechanics, three-dimensionality and NP-completeness: I. universality of intractability of the partition functions of the Ising model across non-planar lattices. In *Proceedings of the 32nd ACM Symposium on the Theory of Computing*, pages 87–96, Portland, Oregon, 2000. ACM Press.
- [55] Mark Jerrum and Alistair Sinclair. Polynomial-time approximation algorithms for the Ising model. *SIAM Journal on Computing*, 22:1087–1116, 1993.
- [56] Mark Kac and John C. Ward. A combinatorial solution of the two dimensional Ising model. *Physical Review*, 88:1332–1337, 1952.
- [57] Dmitry Kamenetsky, Nicol N. Schraudolph, Simon Günter, and S. V. N. Vishwanathan. Modelling Go positions with planar CRFs. *NIPS workshop on Statistical Models of Networks*, 2007. <http://www.cs.ubc.ca/~murphyk/nips07NetworkWorkshop/abstracts/kamenetsky.pdf>.

-
- [58] Dmitry Kamenetsky, Nicol N. Schraudolph, Simon Günter, and S. V. N. Vishwanathan. Modelling Go positions with planar CRFs. *NIPS workshop on Machine Learning and Games*, 2007. <http://www.cs.ubc.ca/~murphyk/nips07NetworkWorkshop/abstracts/kamenetsky.pdf>.
- [59] Pieter W. Kasteleyn. The statistics of dimers on a lattice: I. the number of dimer arrangements on a quadratic lattice. *Physica*, 27(12):1209–1225, 1961.
- [60] Pieter W. Kasteleyn. Dimer statistics and phase transitions. *Journal of Mathematical Physics*, 4(2):287–293, 1963.
- [61] Pieter W. Kasteleyn. Graph theory and crystal physics. In Frank Harary, editor, *Graph Theory and Theoretical Physics*, chapter 2, pages 43–110. Academic Press, London and New York, 1967.
- [62] KGS. The Kiseido Go Server, 2000. <http://www.gokgs.com/>.
- [63] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220:671–680, 1983.
- [64] Uffe Kjærulff. Triangulation of graphs — algorithms giving small total state space. Technical Report R-90-09, Department of Mathematics and Computer Science, Aalborg University, 1990. citeseer.ist.psu.edu/111946.html.
- [65] Pushmeet Kohli and Philip H. S. Torr. Measuring uncertainty in graph cut solutions. *Computer Vision and Image Understanding*, 112(1):30–38, 2008.
- [66] Vladimir Kolmogorov. Convergent tree-reweighted message passing for energy minimization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(10):1568–1583, 2006.
- [67] Vladimir Kolmogorov. Blossom V: A new implementation of a minimum cost perfect matching algorithm. *Mathematical Programming Computation*, 1(1):43–67, 2009. <http://www.cs.ucl.ac.uk/staff/V.Kolmogorov/papers/BLOSSOM5.html>.
- [68] Vladimir Kolmogorov and Carsten Rother. Minimizing nonsubmodular functions with graph cuts — a review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(7):1274–1279, July 2007.
- [69] Vladimir Kolmogorov and Martin J. Wainwright. On the optimality of tree-reweighted max-product message-passing. In *Proc. 21st Annual Conf. Uncertainty in Artificial Intelligence (UAI)*, pages 316–323. AUAI Press, 2005.
- [70] Vladimir Kolmogorov and Ramin Zabih. What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):147–159, February 2004.
- [71] Nikos Komodakis, Georgios Tziritas, and Nikos Paragios. Fast, approximately optimal solutions for single and dynamic MRFs. In *IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2007.

- [72] Nikos Komodakis, Georgios Tziritas, and Nikos Paragios. Performance vs computational efficiency for optimizing single and dynamic MRFs: Setting the state of the art with primal-dual strategies. *Computer Vision and Image Understanding*, 112(1):14–29, 2008. C++ source code available from <http://www.csd.uoc.gr/~komod/FastPD/>.
- [73] Arie M. C. A. Koster, Stan P. M. van Hoesel, and Antoon W. J. Kolen. The partial constraint satisfaction problem: facets and lifting theorems. *Operations Research Letters*, 23:89–97, October 1998.
- [74] H. A. Kramers and G. H. Wannier. Statistics of the two-dimensional ferromagnet. part I. *Physical Review*, 60:252–262, 1941.
- [75] Frank Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [76] Sanjiv Kumar and Martial Hebert. Discriminative fields for modeling spatial dependencies in natural images. In S. Thrun, L. Saul, and B. Schölkopf, editors, *Advances in Neural Information Processing Systems 16*, 2004.
- [77] Sanjiv Kumar and Martial Hebert. Discriminative random fields. *International Journal of Computer Vision*, 68(2):179–201, 2006.
- [78] Kazimierz Kuratowski. Sur le problème des courbes gauches en topologie. In *Fundamenta Mathematicae*, volume 15, pages 217–283, 1930.
- [79] J. D. Lafferty, A. McCallum, and F. Pereira. Conditional random fields: Probabilistic modeling for segmenting and labeling sequence data. In *Proceedings of International Conference on Machine Learning*, volume 18, pages 282–289, San Francisco, CA, 2001. Morgan Kaufmann.
- [80] David Landau and Kurt Binder. *A Guide to Monte Carlo Simulations in Statistical Physics*. Cambridge University Press, New York, NY, USA, 2005.
- [81] Steffen L. Lauritzen and David J. Spiegelhalter. Local computations with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society*, B 50(2):157–224, 1988.
- [82] Wilhelm Lenz. Beiträge zum Verständnis der Magnetischen Eigenschaften in festen Körpern. *Physikalische Zeitschrift*, 21:613–615, 1920.
- [83] Vasilica Lepar and Prakash P. Shenoy. A comparison of Lauritzen-Spiegelhalter, Hugin, and Shenoy-Shafer architectures for computing marginals of probability distributions. In *Proceedings of the 14th Conference on Uncertainty in Artificial Intelligence*, pages 328–337. Morgan Kaufmann, 1998.
- [84] Frauke Liers and Gregor Pardella. A simple max-cut algorithm for planar graphs. Technical Report zaik2008-579, Zentrum für Angewandte Informatik, Universität Köln, September 2008.

-
- [85] Charles H. C. Little. An extension of Kasteleyn's method of enumerating the 1-factors of planar graphs. In *Combinatorial Mathematics*, volume 403 of *Lecture Notes in Mathematics*, pages 63–72. Springer Berlin, Heidelberg, 1974.
- [86] Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(3):503–528, 1989.
- [87] László Lovász and Michael D. Plummer. *Matching Theory*, volume 29 of *Annals of Discrete Mathematics*. North-Holland, Amsterdam, 1986.
- [88] E. Marinari and G. Parisi. Simulated tempering: A new Monte Carlo scheme. *Europhysics Letters*, 19(6), 1992.
- [89] Kurt Mehlhorn and Guido Schäfer. Implementation of $O(nm \log n)$ weighted matchings in general graphs: The power of data structures. *ACM Journal of Experimental Algorithms*, 7:138–148, 2002. Article and C source code (requires LEDA 4.2 or higher) at <http://www.jea.acm.org/2002/MehlhornMatching/>.
- [90] Talya Meltzer, Chen Yanover, and Yair Weiss. Globally optimal solutions for energy minimization in stereo vision using reweighted belief propagation. In *International Conference On Computer Vision*, pages 428–435, 2005.
- [91] Martin Müller. Computer Go. *Artificial Intelligence*, 134(1-2):145–179, 2002.
- [92] Xiaozhen Niu and Martin Müller. An improved safety solver for Computer Go. In *Computers and Games 2004, Ramat-Gan, Israel*, pages 97–112. Springer-Verlag, 2006.
- [93] Xiaozhen Niu and Martin Müller. An improved safety solver in Go using partial regions. In *6th International Conference on Computers and Games*, pages 102–112. Springer-Verlag, 2008.
- [94] NNGS. No Name Go Server, 1995–2005. <http://senseis.xmp.net/?NNGS>.
- [95] Jorge Nocedal. Updating quasi-Newton matrices with limited storage. *Mathematics of Computation*, 35:773–782, 1980.
- [96] Sergey Norin. Drawing Pfaffian graphs. In *International Symposium on Graph Drawing, Lecture Notes in Computer Science*, pages 371–376, 2005.
- [97] Lars Onsager. Crystal statistics. I. a two-dimensional model with an order-disorder transition. *Physical Review*, 65:117–149, 1944.
- [98] Gregor Pardella and Frauke Liers. Exact ground states of large two-dimensional planar Ising spin glasses. *Physical Review E*, 78(5):056705(10), November 2008.
- [99] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufman, 1988.
- [100] Rudolf Peierls. On Ising's model of ferromagnetism. In *Proceedings of the Cambridge Philosophical Society*, volume 32, pages 477–481, 1936.

- [101] Renfrey B. Potts. Some generalized order-disorder transformations. *Proceedings of the Cambridge Philosophical Society*, 48:106–109, 1952.
- [102] Renfrey B. Potts and John C. Ward. The combinatorial method and the two-dimensional Ising model. *Progress of Theoretical Physics*, 13:38–46, 1955.
- [103] Matthew Richardson and Pedro Domingos. Mining knowledge-sharing sites for viral marketing. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 61–70, New York, NY, USA, 2002. ACM.
- [104] Carsten Rother, Vladimir Kolmogorov, Andrew Blake, and Matthew Brown. GrabCut ground truth database. <http://research.microsoft.com/en-us/um/cambridge/projects/visionimagevideoediting/segmentation/grabcut.htm>, 2007.
- [105] Carsten Rother, Vladimir Kolmogorov, Victor Lempitsky, and Martin Szummer. Optimizing binary MRFs via extended roof duality. In *IEEE Conference on Computer Vision and Pattern Recognition*, Minneapolis, MN, June 2007.
- [106] Lawrence K. Saul. *Exact computations in the statistical mechanics of disordered systems*. PhD thesis, Department of Physics, Massachusetts Institute of Technology, 1994.
- [107] Lawrence K. Saul and Mehran Kardar. The 2D +/-J Ising spin glass: exact partition functions in polynomial time. *Nuclear Physics B*, 432:641–667, December 1994.
- [108] Michail I. Schlesinger. Syntactic analysis of two-dimensional visual signals in noisy conditions. *Kibernetika*, 4:113–130, 1976. In Russian.
- [109] Helga Schramm and Jochem Zowe. A version of the bundle idea for minimizing a nonsmooth function: Conceptual idea, convergence analysis, numerical results. *SIAM J. Optimization*, 2: 121–152, 1992.
- [110] Nicol N. Schraudolph and Dmitry Kamenetsky. Efficient exact inference in planar Ising models. Technical Report 0810.4401, arXiv, October 2008. <http://aps.arxiv.org/abs/0810.4401>.
- [111] Nicol N. Schraudolph and Dmitry Kamenetsky. Efficient exact inference in planar Ising models. In D. Koller, Y. Bengio, D. Schuurmans, L. Bottou, and A. Culotta, editors, *Advances in Neural Information Processing Systems 21*, Cambridge, MA, 2009. MIT Press.
- [112] Nicol N. Schraudolph, Peter Dayan, and Terrence J. Sejnowski. Learning to evaluate Go positions via temporal difference methods. In N. Baba and L. C. Jain, editors, *Computational Intelligence in Games*. Springer-Verlag, 2001.
- [113] Nicol N. Schraudolph, Jin Yu, and Simon Günter. A stochastic quasi-Newton method for online convex optimization. In *Proc. 11th Intl. Conf. Artificial Intelligence and Statistics (AISTATS)*, San Juan, Puerto Rico, March 2007. Society for Artificial Intelligence and Statistics. <http://nic.schraudolph.org/pubs/SchYuGue07.pdf>.
- [114] Prakash P. Shenoy. Binary join trees for computing marginals in the Shenoy-Shafer architecture. *International Journal of Approximate Reasoning*, 17(2-3):239–263, 1997.

-
- [115] Prakash P. Shenoy and Glenn Shafer. Propagating belief functions with local computations. *IEEE Expert*, 1:43–52, 1986.
- [116] Prakash P. Shenoy and Glenn Shafer. Axioms for probability and belief-function propagation. In *Uncertainty in Artificial Intelligence*, pages 169–198. North-Holland, 1990.
- [117] S. Sherman. Combinatorial aspects of the Ising model for ferromagnetism. I. a conjecture of Feynman on paths and graphs. *Journal of Mathematical Physics*, 1:202–217, 1960.
- [118] N. J. A. Sloane. The on-line encyclopedia of integer sequences, 2009. <http://www.research.att.com/~njas/sequences/>.
- [119] David H. Stern, Thore Graepel, and David J. C. MacKay. Modelling uncertainty in the game of Go. In Lawrence K. Saul, Yair Weiss, and Léon Bottou, editors, *Advances in Neural Information Processing Systems 17*, pages 1353–1360, Cambridge, MA, 2004. MIT Press.
- [120] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in Monte Carlo simulations. *Physical Review Letters*, 58(2):86–88, Jan 1987.
- [121] Richard Szeliski, Ramin Zabih, Daniel Scharstein, Olga Veksler, Vladimir Kolmogorov, Aseem Agarwala, Marshall Tappen, and Carsten Rother. A comparative study of energy minimization methods for Markov random fields with smoothness-based priors. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 30(6):1068–1080, 2008. C++ source code available from <http://vision.middlebury.edu/MRF/code/>.
- [122] Marshall F. Tappen and William T. Freeman. Comparison of graph cuts with belief propagation for stereo, using identical MRF parameters. In *Proc. 9th IEEE Intl. Conf. Computer Vision (ICCV)*, pages 900–907, 2003.
- [123] B. Taskar, C. Guestrin, and D. Koller. Max-margin Markov networks. In S. Thrun, L. Saul, and B. Schölkopf, editors, *Advances in Neural Information Processing Systems 16*, pages 25–32, Cambridge, MA, 2004. MIT Press.
- [124] H. N. V. Temperley and M. Fisher. Dimer problem in statistical mechanics - an exact result. *Philosophical Magazine*, 6:1061–1063, August 1961.
- [125] G. Tesler. Matching in graphs on non-orientable surfaces. *Journal of Combinatorial Theory Series B*, 78:198–231, March 1961.
- [126] Creighton K. Thomas and A. Alan Middleton. Matching Kasteleyn cities for spin glass ground states. *Physical Review B*, 76(22):22406, 2007.
- [127] Robin Thomas. A survey of Pfaffian orientations of graphs. *International Congress of Mathematicians*, 3:963–984, 2006.
- [128] Carsten Thomassen. The graph genus problem is NP-complete. *Journal of Algorithms*, 10(4): 568–576, 1989.

- [129] G. Toulouse. Theory of the frustration effect in spin glasses: I. *Communications on Physics*, 2: 115, 1977.
- [130] Erik C. D. van der Werf, Hendrik Jaap van den Herik, and Jos W. H. M. Uiterwijk. Learning to estimate potential territory in the game of Go. In *4th International Conference on Computers and Games*, pages 81–96. Springer-Verlag, 2004.
- [131] Erik C. D. van der Werf, Hendrik Jaap van den Herik, and Jos W. H. M. Uiterwijk. Learning to score final positions in the game of Go. *Theoretical Computer Science*, 349(2):168–183, 2005.
- [132] Erik C. D. van der Werf, Mark H. M. Winands, Hendrik Jaap van den Herik, and Jos W. H. M. Uiterwijk. Learning to predict life and death from Go game records. *Information Science*, 175 (4):258–272, 2005.
- [133] J. Vannimenus and G. Toulouse. Theory of the frustration effect: II. Ising spins on a square lattice. *Journal of Physics C: Solid state physics*, 10:537–542, 1977.
- [134] R. Vilà and T. Cazenave. When one eye is sufficient: A static classification. In H. Jaap van den Herik, Hiroyuki Iida, and Ernst A. Heinz, editors, *Advances in Computer Games*, volume 263 of *IFIP*, pages 109–124. Kluwer, 2003.
- [135] S. V. N. Vishwanathan, Karsten Borgwardt, and Nicol N. Schraudolph. Fast computation of graph kernels. In B. Schölkopf, J. Platt, and T. Hofmann, editors, *Advances in Neural Information Processing Systems 19*, Cambridge MA, 2007. MIT Press.
- [136] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. MAP estimation via agreement on trees: Message-passing and linear programming. *IEEE Transactions on Information Theory*, 51(11):3697–3717, 2005.
- [137] Yair Weiss. Belief propagation and revision in networks with loops. Technical Report AI memo 1616, CBCL paper 155, MIT, Department of Brain and Cognitive Sciences, November 1997.
- [138] Arthur T. White and Lowell W. Beineke. Topological graph theory. In Lowell W. Beineke and Robin J. Wilson, editors, *Selected Topics in Graph Theory*, chapter 2, pages 15–49. Academic Press, 1978.
- [139] Aaron Windsor. Planar graph functions for the Boost graph library. C++ source code available from <http://boost-consulting.com/vault/index.php?directory=Algorithms/graph>, 2007.
- [140] David Wolfe. Go endgames are PSPACE-hard. In Richard Nowakowski, editor, *More Games of No Chance*, pages 125–136. Cambridge University Press, 2002.
- [141] Jennifer Wortman. Viral marketing and the diffusion of trends on social networks. Technical Report MS-CIS-08-19, University of Pennsylvania, Department of Computer and Information Science, May 2008.
- [142] Lin Wu and Pierre Baldi. A scalable machine learning approach to Go. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 1521–1528. MIT Press, Cambridge, MA, 2007.

-
- [143] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Generalized belief propagation. In T. K. Leen, T. G. Dietterich, and V. Tresp, editors, *Advances in Neural Information Processing Systems 13*, pages 689–695. MIT Press, 2001.